AeroMorse — Complete Build Guide

USB HID Keyboard & Mouse via Sip-and-Puff or Accessibility Switches

> **Software & this guide live here:** https://github.com/jlubin2001/AeroMorse — always download the firmware (`code.py` , `boot.py` , `morse_map.py` , `config.py`) from that repo. See **Step 9.3** for the download steps.

This guide is built around **one build — the one the project's author uses every day** — and three choices you can make differently: how you give input, how you see the display, and whether you hear audio feedback. If the standard build suits you, sections 2, 9 and 11 are all you need.

Contents

1. What AeroMorse does
2. The standard build
    - 2.1 A cheaper build with no screen on the board (tested)
3. Three choices: input, display, audio feedback
4. Choice 1 — Input
    - 4.1 Sip-and-puff (standard)
    - 4.2 Switches
        - Way 1 — a board that needs no soldering (tested)
        - Way 2 — on the standard #5691, three soldered wires (not tested)
        - One switch, or two switches with an "accept"
5. Choice 2 — Display
    - 5.1 The built-in screen (standard)
    - 5.2 A window on the computer
    - 5.3 A wireless display
    - 5.4 The window on the computer, in detail
        - On a Mac or a Linux computer (not tested yet)
        - Sound through the computer's speakers
        - Opening and closing the window with a Morse code
    - 5.5 Using more than one, or switching the built-in screen off
6. Choice 3 — Audio feedback
    - 6.1 None (standard)
    - 6.2 Through the computer's speakers
    - 6.3 A speaker on the device
7. Shopping list
    - Mounting hardware
    - Tools
8. Putting it together
9. Software
    - Step 9.1 — Install CircuitPython on the Feather
    - Step 9.2 — Install the required libraries
    - Step 9.3 — Copy the AeroMorse project files
        - Download the AeroMorse software (do this first)
        - 👉 https://github.com/jlubin2001/AeroMorse
    - Step 9.4 — Files for a wireless display (only if you have one)
        - Files on the receiver
        - Firmware on the receiver (Step 9.1 equivalent)
        - What the wireless display's drive looks like
        - Verifying the receiver
10. Settings for your choices
    - Input modes — what SWITCH_MODE does
11. First Power-On Test

- Quick test — sensor mode
- Quick test — switch mode
- If nothing appears

12. Troubleshooting
- Input — sip-and-puff sensor (#4414)
- Input — AT switches
- Input — timing
- On the computer
- Display
- Speaker
- Wireless display (ESP-NOW)
- Software / general

Appendix A — Groups and Group Cycling
Appendix B — Display Layout
Appendix C — Customising Macros
Appendix D — Other switch and speaker wiring
- Switches on a breadboard (Option B1)
  - Parts
  - Step-by-step breadboard walkthrough
  - Parts — exactly what to buy
  - Understanding the breadboard
  - Step 1 — Place the Feather in the breadboard
  - Step 2 — Place the 3.5 mm stereo jack (#1699)
  - Step 3 — Connect with jumper wires
  - Step 4 — Connect your AT switches
- Switches on a soldered jack board (Option B2)
- A piezo on a detachable plug (Option S2)
  - Parts
  - Wiring
  - Connection table
  - Why this works without signal conditioning
  - Already have a 3.5 mm-plug piezo or speaker?
  - Common configurations — how many #2915 jacks you need
  - Wiring guide for each role
  - Connecting either jack to the Feather
- A small speaker with a separate amplifier (Option S3)
- Connecting any of the speakers, with and without header pins
  - If your Feather has header pins (or sits in a breadboard)
  - If your Feather has no header pins (bare PCB pads)
  - Quick reference — all paths, all options

Appendix E — Configuration Reference
- Input — sensor / switches / thresholds
- Input mode — 1 / 2 / 3 switch
- Strong sip / strong puff
- Timing

- Code repeat (Darci-style hold-to-repeat)
- Audio
- Mouse (Group 2 movement speeds)
- Shift + arrows
- Repeat exclusions
- Display / wireless

Appendix F — Other displays

- Which displays are tested
- Using a separate display — display_setup.py (v1.26+)
- Understanding the columns
- FeatherWing displays (plug directly on Feather — easiest wiring, requires headers)
- Standalone / breakout displays (7 wires required)
- OLED displays (STEMMA QT plug-and-play, monochrome, no colour)
- Parts for a separate display
- Connecting a separate display
  - If you chose a FeatherWing display (#3651, #5872, or #3315)
  - If you chose the OLED #326
  - If you chose a standalone TFT breakout (#2050, #1770, etc.)
- Display driver library required per display chip
- A bigger wireless display
- A 3.5" screen on a #5477, step by step
  - F.1 When this build is the right pick
  - F.2 Parts list
  - F.3 Header pins on the #5477 — required
  - F.4 Hardware assembly
  - F.5 Software setup
    - Libraries on /lib
    - Project files
    - Add display_setup.py
    - The layout sizes itself
  - F.6 Configuration
  - F.7 Verifying it works

Appendix G — AutoHotkey: starting programs with a Morse code

- G.1 Get AutoHotkey
- G.2 Put the script in your work folder
- G.3 Make it start with Windows
- G.4 The three hotkeys
- G.5 The Morse codes
- G.6 Your own hotkeys, and changing the script

Appendix H — Enclosure and air-tube strain relief

- H.1 The author's enclosure — the ATMakers "HoseBox"
- H.2 A case for the wireless display
- H.3 No enclosure yet? Do the strain relief anyway
- H.4 Getting something 3D-printed without a printer

Appendix I — Keyboard layout: when symbols come out wrong

- I.1 Easiest: set the computer to a US keyboard
- I.2 Or: tell AeroMorse which layout the computer uses (v1.28+)
- I.3 What the setting does and does not change

Appendix J — More media, browser and app-launcher keys

Appendix K — Solderless build: Cytron Maker Feather AIoT S3 with two switches

- K.1 When this build is the right pick
- K.2 Parts
- K.3 What is different about this board
- K.4 Software
- K.5 config.py settings
- K.6 Wiring
- K.7 First test and troubleshooting
- K.8 Adding the pressure sensor later (not tested)

Appendix L — Other boards

- Why not every Feather works
- Bluetooth — not supported
- Boards that work
- A non-Adafruit board that works — and needs no soldering
- Metro form factor boards
- A solderless switch build on the Metro ESP32-S3
  - Using the pressure sensor on the Metro (tested)
- Boards that do NOT work for USB HID
- Recommended board: Adafruit ESP32-S3 Reverse TFT Feather #5691
- Boards with a screen built in
- The pressure sensor on a board with no STEMMA QT socket

Appendix M — Header pins

- Building with no header pins — what works
- Getting header pins soldered on

Appendix N — Seeing the device's error messages (Thonny)

- N.1 Download and install Thonny
- N.2 Confirm the Feather is plugged in
- N.3 Point Thonny at CircuitPython
- N.4 Leave the Shell panel visible
- N.5 Capturing a fresh boot trace (Ctrl+D)
- Useful Thonny shortcuts

1. What AeroMorse does

AeroMorse turns a small circuit board into a USB keyboard and mouse that you work with Morse code. A short sip is a dot and a short puff is a dash — or, with two switches, one switch is the dot and the other is the dash. It plugs into the computer with a USB-C cable and needs no drivers. It works on Windows, macOS, Linux, iPadOS, Android and ChromeOS. It is a **wired** device: Bluetooth is not supported (Appendix L explains why).

It is an open-source project directed by Jim Lubin, a ventilator-dependent quadriplegic who has used Morse code for computer access since 1989, and it is his own daily device. It was inspired by AirTalker.

What you get:

- **Keyboard** — letters, numbers, punctuation, function keys, modifiers.
- **Mouse** — movement, clicks, drag, scrolling.
- **Macros** — phrases and passwords on one code.
- **Media and app keys** — volume, play / pause, calculator and so on.
- **Scanning and Switch groups** — for iOS / Android Switch Control and for switch games.

These are arranged in ten groups; Appendix A explains them. Every code can be changed in one file, `morse_map.py`. Darci USB users can start from `morse_map_darci.py` and keep their familiar codes.

You do not need to know Morse code to begin. A printed cheat sheet, and a help window on the computer, show every code.

The parts for the standard build cost about $42 at Adafruit's prices in October 2026, not counting a USB cable, shipping, or an enclosure.

> **How this guide was made, and what is tested.** Jim directs the project and decides how it works; the firmware and this guide are written by Claude (Anthropic) under his direction. The **standard build in section 2 is the one Jim uses every day**. Each choice after that says plainly whether it has been built and tested. Hardware that nobody on the project has built is in the appendices, marked untested. If you build something, please report how it went in a GitHub issue.
>
> **Will this be someone's only way to use a computer?** Consider building two. The second is a spare that works the minute the first fails, and a safe place to try a change. See the Usage Guide, section 6.

2. The standard build

This is the build the project is designed around and tested on every day. It has **no soldering and no tools**: two boards, one cable between them, and a tube.

Part	Adafruit #	Price
ESP32-S3 Reverse TFT Feather — the board, with a small colour screen built in	#5691	$24.95
LPS33HW pressure sensor — senses the sip and puff	#4414	$12.50
STEMMA QT cable, 100 mm — joins the two boards	#4210	$0.95
Silicone tubing, 1 m — the sip-and-puff tube	#3659	$3.50
USB-C cable that carries **data** (not a charge-only cable)	any	—
Total		**$41.90**

Prices throughout this guide are Adafruit's single-item prices on 12 October 2026, before shipping and tax. They change; check the product page.

Putting it together takes four steps:

1. Push one end of the STEMMA QT cable into the socket on the Feather. It only fits one way; do not force it.
2. Push the other end into either socket on the pressure sensor.
3. Push the tube over the small raised port on the sensor.
4. Plug the USB-C cable into the Feather and the computer.

Then go to **section 9, Software**.

Two things to plan for before daily use, neither needed for a first test:

- **A strain relief for the tube.** The tube is held on only by friction, and every tug pulls on the sensor. Appendix H shows how to anchor it.
- **An enclosure.** Appendix H describes the one Jim uses and others.

2.1 A cheaper build with no screen on the board (tested)

The same sip-and-puff device for less money, also with **no soldering**. The board has no screen; what the device is doing shows in the **AeroMorse Display** window on the computer instead (§5).

Part	Adafruit #	Price
ESP32-S3 Feather, 4 MB flash / 2 MB PSRAM — the board, no screen	#5477	$17.50
LPS33HW pressure sensor	#4414	$12.50
STEMMA QT cable, 50 mm — joins the two boards	#4399	$0.95
Silicone tubing, 1 m	#3659	$3.50
USB-C cable that carries **data**	any	—
Total		**$34.45**

> ⚠️ **Pick the right version of the #5477.** Its Adafruit page lists more than one version of this Feather. Choose the one with **4 MB flash and 2 MB PSRAM**; AeroMorse needs the PSRAM.

Put it together exactly as above: cable from the Feather's STEMMA QT socket to the sensor, tube on the sensor's port, USB-C to the computer. The Feather's loose header pins are not needed — leave them in the bag.

In section 9, also change these lines in `config.py`:

```
DEVICE_NAME = "AeroMorse"   # any name you like
USE_DISPLAY = False         # this board has no screen
PC_DISPLAY  = True          # show the display in the window on the computer
```

Then run the **AeroMorse Display** program on the computer (§5). Sound can come through the computer too: `PC_SOUND = True` (§6.2).

> **Tested:** built and checked on 2026-11-09 with CircuitPython 10.3.1 and AeroMorse v1.33. The test board arrived with an older bootloader that would not take CircuitPython 10; if that happens to you, see "If CircuitPython 10 will not install" in Appendix L.

If this is the build you want, you can skip sections 3 to 6. They are for people who need something different.

---

3. Three choices: input, display, audio feedback

People need different ways to give input, to see what the device is doing, and to hear it. Each of these is a separate choice, and any combination works. The first row of each table is the standard build.

Input — how you make dots and dashes

Choice	Extra parts	Soldering	Tested
Sip-and-puff (standard)	none	none	✅ Jim's daily device
Switches, no soldering	a different board (Cytron) and two jacks	none	✅ built and tested
Switches on the standard board	a jack with screw terminals, 3 wires, a splitter cable	3 joints	⚠️ not tested

Display — how you see the code as you enter it

Choice	Extra parts	Soldering	Tested
Built-in screen (standard)	none	none	✅
A window on the computer — any size	none (a program on the computer)	none	✅ Windows, Jim's daily set-up · ⚠️ Mac and Linux not tested
A wireless display — a second small screen placed where you can see it	a second #5691	none	✅ Jim's daily set-up

Audio feedback — hearing each dot and dash

Choice	Extra parts	Soldering	Tested
None (standard) — the screen shows everything	none	none	✅
Through the computer's speakers	none (the same program)	none	✅ Windows · ⚠️ Mac and Linux not tested
A speaker on the device	STEMMA Speaker and its cable	3 joints on a #5691; none on the Cytron board	✅ on the Cytron board and the Metro · ⚠️ not tested on a #5691

The choices do not depend on each other. For example, Jim's own set-up is sip-and-puff, a wireless display **and** the window on the computer, with the built-in screen switched off and no speaker.

> **A word on header pins.** Feather boards arrive bare, with two strips of pins loose in the bag. **Nothing in this guide's main sections needs those pins soldered on.** They matter only for some of the untested hardware in the appendices (Appendix M).

---

4. Choice 1 — Input

4.1 Sip-and-puff (standard)

A small pressure sensor detects the change in pressure when you sip or puff on a tube. No hand or head movement is needed. The parts and the four steps are in section 2.

It is not hard breathing. The tube is closed at the sensor, so no air flows through it: a sip or a puff is a slight change of pressure made with the mouth, not a breath. The sensor responds to about 2 hPa, the pull it takes to raise a drink 2 cm up a straw. Jim is dependent on a ventilator to breathe and works his AeroMorse all day. The Reference (`AEROMORSE_REFERENCE.md`), under "How Input Works", says more.

Tube. The Adafruit silicone tubing (#3659, 2.5 mm inside) fits the sensor's port snugly and is the easiest choice. Aquarium airline tubing and medical sip-and-puff tubing (3/16" inside) also work; if the fit is loose, wrap a turn of silicone tape round the port first. Most people use 20 to 40 cm.

Setting: none — sip-and-puff is the default.

Adafruit's own guide to this sensor is good background reading on breath technique: https://learn.adafruit.com/st-lps33-and-circuitpython-sip-and-puff. Read it for ideas only. **Do not install its code** — it is for a different board.

4.2 Switches

Assistive-technology switches have a **3.5 mm mono plug**. Any of them works. Two switches is the usual arrangement — one for dot, one for dash — and one switch is also possible.

There are two ways to connect them. The first is the one the project has built and tested.

Way 1 — a board that needs no soldering (tested)

A different board, the **Cytron Maker Feather AIoT S3**, is sold with its pins already fitted, so every wire pushes on. **Each switch gets its own 3.5 mm jack**, so the switches plug straight in and no splitter cable is needed. The board has a buzzer built in. It has no screen; use the window on the computer (section 5.2).

The complete build — parts, wiring, settings — is in **Appendix K**.

One thing to know before ordering: Cytron ships from Malaysia, and to the USA the shipping and import charges came to about twice the price of the parts.

Way 2 — on the standard #5691, three soldered wires (not tested)

> ⚠️ **Nobody on the project has built this.** It is written from the parts' documentation. If you build it, please report how it went.

One 3.5 mm jack with screw terminals takes both switches. Nothing is soldered at the jack; three wires run from it to the Feather and are soldered to three pads there.

Part	Where
TRRS jack with screw terminals	Adafruit #2915, $2.50
3 short wires, about 15 cm	—
For two switches: a **3.5 mm stereo to dual mono cable**	sold online as "3.5mm stereo to dual mono"

The jack's four screw terminals are labelled for audio, not for switches. This is what each one is:

Label on the jack	Part of the plug	Feather pad	Is
Left	tip	D5	dot switch
Right	first ring	D6	dash switch
GND	sleeve	GND	ground
(no label)	second ring	—	leave empty

The labels are taken from Adafruit's description of the part and have not been checked against one. Look at yours before wiring.

Plugging the switches in:

You have	Plug in	Setting
Two switches	The stereo to dual mono cable goes into the jack; one switch goes into each of its two sockets. The cable's *left* socket is the dot and its *right* socket is the dash	`SWITCH_MODE = 2` (the default)
One switch	The switch goes straight into the jack — no cable	`SWITCH_MODE = 1`

One jack carries two switches because a stereo plug has two signal contacts and a mono plug has one; the cable gives each switch one of the two. This is the same arrangement Adafruit describes for its TRRS Trinkey: https://learn.adafruit.com/using-the-trrs-trinkey-as-an-assistive-technology-device/parts-list. That guide also shows a third switch, on the jack's remaining contact. AeroMorse does not read a third switch, so leave that out.

> **Do not plug a single mono switch straight into the jack while `SWITCH_MODE` is 2.** A mono plug's long sleeve touches the dash contact as well as ground, so the device would see the dash switch held down all the time. With `SWITCH_MODE = 1` only the dot contact is read and a single switch is fine.

Setting for either way: `USE_SENSOR = False` in `config.py`.

Which socket of the cable is the dot, and the warning about a single mono plug, follow from how the plugs are made. Like the rest of Way 2 they have not been tried on hardware.

One switch, or two switches with an "accept"

Besides the usual two-switch mode, AeroMorse can be worked with a single switch (a short press is a dot, a long press is a dash), or with two switches plus an explicit "accept" gesture. These are the `SWITCH_MODE` setting — see section 10.

Other ways to wire switches — a breadboard, a soldered jack board — are in Appendix D.

---

5. Choice 2 — Display

The display shows four things: the group you are in, the dots and dashes as you enter them, the last action, and a status line.

5.1 The built-in screen (standard)

The #5691's screen works as soon as the software is installed. There is nothing to buy or connect.

Setting, only if the text is upside down for the way you mount it: `DISPLAY_ROTATION = 0` puts the USB socket on the left of the screen, `180` puts it on the right.

5.2 A window on the computer

AeroMorse Display is a small program that shows the same four things in a window on the computer's own monitor. You can make it any size, put it anywhere, keep it on top of other programs and make it see-through. It needs no extra hardware — the information travels down the USB cable the device is already plugged in with.

This is the easy way to get **big text**.

1. In `config.py` set `PC_DISPLAY = True`.
2. On the computer, start `AeroMorse Display.exe`. It finds the device by itself.

On Windows it is a ready-made program. On a Mac or Linux computer it runs from Python instead (section 5.4); that has not been tested yet. It shows the AeroMorse plugged into that computer.

Section 5.4 has the details: the window's menu, resizing it, and opening it with a Morse code.

5.3 A wireless display

Use this when you cannot see the device itself — for example when the tube and sensor are behind you. A **second #5691** sits where you can see it and shows the same screen, with no cable between the two. Range is about 30 m indoors.

Part	Adafruit #	Price
A second ESP32-S3 Reverse TFT Feather	#5691	$24.95
Power for it: a USB charger or power bank…	any	—
…or a battery that plugs into the board:		
400 mAh, slim — about 3 to 5 hours	#3898	$6.95
500 mAh — about 4 to 6 hours	#1578	$7.95
1200 mAh — about 9 to 14 hours	#258	$9.95

There is nothing to assemble. The second board gets its own small set of files — Step 9.4 has them. Both boards need CircuitPython 9 or later.

Settings on the main device: `USE_WIRELESS_DISPLAY = True`, and `ESPNOW_CHANNEL` the same number on both boards.

5.4 The window on the computer, in detail

If the screen you want to see the AeroMorse on is **the computer's own monitor**, you need no second board at all. **AeroMorse Display** is a small Windows program that shows the device's screen — group, the dots and dashes as they build, the last action and the status line — in a window you can make any size and place anywhere.

1. On the device, set `PC_DISPLAY = True` in `config.py`.
2. On the computer, start `AeroMorse Display.exe` (from the repo). It finds the device by itself.

- **Any size, from 160 × 90 pixels up.** Drag the window's edge; the text grows to fill it. It will not go smaller than 160 × 90. **Resizing only works while *Title bar* is ticked** — the plain panel has no edge to drag. To change its size: right-click, tick *Title bar*, resize, then untick it again. The size is kept.
- **Right-click the window** for the menu: *Always on top*, *See-through* (so it can sit over other programs), *Title bar* (untick for a plain panel you move by dragging it), *Sound* (see below), *Show device*, *Close*. Your choices and the window's place are remembered.
- **It only listens.** The program never sends anything to the device, so it cannot stop or disturb it — unlike Thonny (Appendix N). The information travels over the USB cable the device is already plugged in with.
- **It cannot get lost off the screen.** The window opens where you left it. If that place is no longer on any screen — it was last on a second monitor that has since been unplugged — it opens near the top-left of the main screen instead, at the size you gave it.
- **It reconnects by itself** after `devicereset`, a replug or a restart. It remembers which AeroMorse it showed and waits for that one, even if another is plugged in.
- **More than one AeroMorse plugged in?** *Show device* in the menu lists each one by name — the `DEVICE_NAME` in its `config.py`, e.g. *AeroMorse Green (COM8)* — with a dot at the one shown. Pick another to show it instead. A device that is not plugged in is listed but greyed out.
- It works alongside the built-in screen and the wireless display, or instead of them (`USE_DISPLAY = False`).

Limits: it shows the AeroMorse plugged into *this* computer. It has no pressure bar. And while it is running, no other program (Thonny, a serial terminal) can open that device's port — close the window first.

Running from source instead: `python aeromorse_display.py` (needs `pyserial`). `--port COM8` listens to one port only; `--demo` shows made-up data with no device.

On a Mac or a Linux computer (not tested yet)

The same `aeromorse_display.py` runs on macOS and Linux. There is no ready-made program for them, so it runs from Python. The Windows parts were replaced with ones that work there, and the Linux parts were

checked without a window, but nobody has yet used it with a device on a Mac or a Linux computer.

1. **Python 3 with tkinter.**
   - Mac: install Python from python.org; it includes tkinter. (With Homebrew's Python, also `brew install python-tk`.)
   - Linux: Python is already there; add tkinter, e.g. on Ubuntu or Debian `sudo apt install python3-tk`.
2. **The two Python packages:** `pip3 install pyserial sounddevice` (`sounddevice` is only needed for the beeps).
3. **Linux only — permission to open the port.** Run `sudo usermod -aG dialout $USER`, then log out and back in. Until then the window says *Not allowed to open /dev/ttyACM0*.
4. **Start it:** `python3 aeromorse_display.py`, from the folder it is in. It finds the device by itself. To name the port yourself: `--port /dev/cu.usbmodem1101` (Mac) or `--port /dev/ttyACM0` (Linux).

Differences from Windows:

- **Right-click** works as usual. On a Mac a two-finger click or Control-click opens the menu too.
- **Settings are kept** in `~/Library/Application Support/AeroMorse` (Mac) or `~/.config/AeroMorse` (Linux).
- **See-through** needs a desktop that supports it. Most Linux desktops do; on some the window simply stays solid.
- **Opening it with a Morse code** (Appendix G) uses AutoHotkey, which is Windows only. On a Mac or Linux, a keyboard shortcut set up in the system's own settings can run the command in step 4.

Sound through the computer's speakers

AeroMorse Display can also **play the device's beeps on the computer** — a tone for as long as each dot or dash is held (higher for a dot, lower for a dash), a short blip when an action fires, and a lower blip on a group change. This gives audio feedback **without fitting a speaker to the device**, which matters to anyone who enters Morse by ear — for example when moving from a Darci USB.

1. In `config.py` set `PC_DISPLAY = True` and `PC_SOUND = True`.
2. Start AeroMorse Display, right-click the window and choose **Sound → Quiet / Medium / Loud**. A short beep confirms the volume. *Off* silences it again. The choice is remembered.

The pitches and blip lengths are the ones in `config.py` (`BEEP_DOT_FREQ`, `BEEP_DASH_FREQ`, `CONFIRM_FREQ`, `GROUP_FREQ`, `BEEP_CONFIRM_S`, `BEEP_GROUP_S`) — the same settings a fitted speaker uses, so changing them changes both.

Good to know:

- The sound comes from the computer, so it follows the computer's own volume and output device (speakers, headphones, Bluetooth). A Bluetooth speaker or headset adds a noticeable delay; wired speakers do not.
- A tone starts within a few hundredths of a second of the sip, puff or switch press and ends about as promptly. That is close to, but not quite as immediate as, a speaker on the device itself. If you key very

fast and rely on the sound for timing, a fitted speaker (§6) remains the better choice.
- The window does not need to be visible — only running. Make it small, or see-through, if you want the sound and not the picture.
- It plays alongside a speaker fitted to the device, if there is one.
- **Tested** on the author's daily device with a sip-and-puff sensor: clean tones, a dot clearly shorter than a dash. It has not yet been judged by someone who keys Morse by ear at speed. Reports are welcome.

Opening and closing the window with a Morse code

One Morse code can switch the window on and off, using the free **AutoHotkey** program on the computer. The steps — where to get AutoHotkey, a ready-made script, making it start with Windows, and the line for `morse_map.py` — are in **Appendix G — AutoHotkey**.

5.5 Using more than one, or switching the built-in screen off

The three displays work together in any combination. If you only ever watch the wireless display or the computer window, set `USE_DISPLAY = False` to switch the built-in screen off. That is not just tidiness: drawing the built-in screen takes the device's attention for a moment, and switching it off cured missed sips and puffs on Jim's device.

Other displays — a 3.5" screen that plugs onto the board, separate screens wired by hand, small OLED screens — are in Appendix F. None has been tested by the project, and the window on the computer does the same job with no hardware.

---

6. Choice 3 — Audio feedback

Some people enter Morse by ear and want a tone for each dot and dash. Others never miss it.

6.1 None (standard)

The screen shows every dot and dash. There is nothing to do. If no speaker is fitted you can set `USE_SPEAKER = False`, though leaving it on does no harm.

6.2 Through the computer's speakers

The same **AeroMorse Display** program (section 5.2) can play the beeps through the computer: a tone for as long as each dot or dash lasts, higher for a dot and lower for a dash, and a short blip when a code is accepted. No parts, no wiring.

1. In `config.py` set `PC_DISPLAY = True` and `PC_SOUND = True`.
2. Start AeroMorse Display, right-click the window and choose *Sound → Quiet, Medium* or *Loud*.

The window does not have to be visible, only running. The sound starts a few hundredths of a second after the sip or puff — close to a speaker on the device, but not quite as immediate. Use wired speakers;

Bluetooth adds a noticeable delay. Section 5.4 has more on the sound.

6.3 A speaker on the device

For sound that does not depend on the computer, or for someone who keys fast and relies on the tone for timing.

Part	Adafruit #	Price
STEMMA Speaker, with its own small amplifier and volume control	#3885	$5.95
Its cable — **sold separately**	#4046	$0.95

The cable plugs into the speaker. Its other end is three bare wires, which are soldered to three pads on the Feather:

Wire	Feather pad
Black	GND
White	A0
Red	3V

Setting: none — the speaker is on by default (`USE_SPEAKER = True`).

The Cytron board in section 4.2 has a buzzer built in and needs no speaker.

> ⚠️ **Tested on the Cytron board only.** There the speaker's three wires push onto the board's pins, and it has been built and heard working. Soldering it to a #5691, as described above, has **not** been tried by the project. If you do it, please report how it went.

Other speakers — a piezo on a detachable plug, a separate amplifier board — are in Appendix D.

---

7. Shopping list

Start with the standard build and add only what your choices need.

For	Buy	Adafruit #	Price
Every build	ESP32-S3 Reverse TFT Feather	#5691	$24.95
	USB-C **data** cable	any	—
Sip-and-puff (standard)	LPS33HW pressure sensor	#4414	$12.50
	STEMMA QT cable, 100 mm	#4210	$0.95
	Silicone tubing, 1 m	#3659	$3.50
Switches, no soldering (tested)	*see Appendix K — a different board replaces the #5691*	—	—
Switches on the #5691 (not tested)	TRRS jack with screw terminals	#2915	$2.50
	3 short wires	—	—
	for two switches, a 3.5 mm stereo to dual mono cable	sold online	—
Wireless display	a second #5691	#5691	$24.95
	a USB charger or power bank, or a battery	#3898, #1578 or #258	$6.95 to $9.95
Speaker on the device	STEMMA Speaker	#3885	$5.95
	its cable	#4046	$0.95
Window or sound on the computer	nothing	—	free
Recommended extras	Nylon screw and stand-off set, M2.5 — white	#3658	$14.95
	or the same set in black	#3299	$16.95
	An enclosure and a tube strain relief	Appendix H	—

What the common builds add up to (Adafruit parts only):

Build	Parts	Total
Standard: sip-and-puff, built-in screen	#5691, #4414, #4210, #3659	$41.90
Standard plus a wireless display on a charger	the above and a second #5691	$66.85
Standard plus a speaker	the above standard build, #3885 and #4046	$48.80

To check that a USB cable carries data: plug the board in with it. If a drive appears on the computer, it does.

Mounting hardware

One set is far more than one device needs — a few hundred pieces: M2.5 screws 4, 6 and 10 mm long, nuts, and hex stand-offs from 6 to 12 mm — and it covers every job in this guide: standing the Feather off the floor of an enclosure, screwing the sensor board down, and joining boards together (Appendix H). Nylon cannot short anything it touches. Black and white are the same parts; pick whichever suits your case. M2.5 fits the sensor board and the Feather's two larger mounting holes; the Feather's two smaller holes take M2 screws, which are not in this set.

Tools

The standard build needs no tools. The two soldered choices — switches on the standard board, and a speaker — need the tools below. If you do not own them, many makerspaces, library maker labs and repair shops will solder three wires for little or nothing.

Tool	Notes	Adafruit #	URL
Soldering iron — 40–60 W adjustable	Temperature-controlled irons are easier for beginners	#3685	https://www.adafruit.com/product/3685
Solder — 60/40 rosin-core 0.3 mm	Standard electronics solder; the thin gauge is easier for small pads	#145	https://www.adafruit.com/product/145
Flush cutters	For trimming wire leads flush after soldering	#152	https://www.adafruit.com/product/152
Helping hands	Holds small parts steady while soldering	#291	https://www.adafruit.com/product/291
Wire strippers	For stripping insulation from wire ends	any	hardware store

> **New to soldering?** Adafruit has a free beginner's guide: https://learn.adafruit.com/adafruit-guide-excellent-soldering

Useful for any build:

Tool	Purpose	Notes
Digital multimeter	Test switch continuity, check wiring	Any basic model; ~$10–20 at hardware stores
Small scissors	Cut sip-and-puff tubing to length	Any pair
Silicone tape	Seal tube to sensor if fit is loose	Hardware store; stretchy self-fusing tape
Velcro strips	Mount display or sensor to a surface	Any brand

---

8. Putting it together

Do the parts that match your choices. Nothing needs to be plugged into the computer until the end.

Sip-and-puff — the four steps in section 2.

Switches, no soldering — Appendix K.

Switches on the #5691 *(not tested)*

1. Strip both ends of three wires.
2. Screw one end of each into the **Left**, **Right** and **GND** terminals of the jack. Leave the unlabelled terminal empty.
3. Solder the other ends to the Feather's pads: Left to **D5**, Right to **D6**, GND to **GND**. The pad names are printed on the board.
4. Two switches: plug the stereo to dual mono cable into the jack, and a switch into each of its sockets. One switch: plug it straight into the jack and set `SWITCH_MODE = 1`.

Wireless display — nothing to assemble. Stand the second board where you can see it and give it power.

Speaker on the device

1. Plug the cable into the speaker.
2. Solder its three bare wires to the Feather's pads: black to **GND**, white to **A0**, red to **3V**.

Window or sound on the computer — nothing to assemble.

When the parts are together, go on to the software.

---

9. Software

Three steps, the same for every build: put CircuitPython on the board, copy a few libraries, copy the AeroMorse files. A fourth step is only for a wireless display.

You do not need any special program for this — the board appears on the computer as a drive, and you copy files to it. (A program called Thonny can show the device's error messages if something goes wrong. It is in Appendix N, and it is not needed for a normal install.)

Step 9.1 — Install CircuitPython on the Feather

1. Go to **https://circuitpython.org/downloads**
2. Search for your Feather board name (e.g. "ESP32-S3 Feather").
3. Download the latest **stable** `.uf2` file (not a pre-release).
4. Plug the Feather into your computer with the USB-C **data** cable.
5. **Double-tap** the small Reset button on the Feather quickly (two taps within about half a second).

- The NeoPixel LED on the Feather turns **green**.
- A drive named **FTHRS3BOOT** (or similar) appears on your computer.

6. Drag the `.uf2` file you downloaded onto that drive.
7. The Feather reboots automatically. After a few seconds a drive named **CIRCUITPY** appears. Done.

> If **CIRCUITPY** already appears when you plug in (without double-tapping), CircuitPython is already installed — skip straight to Step 9.2.
>
> If you see **FTHRS3BOOT** (or similar) every time you plug in without double-tapping, the Feather has no code loaded — this is normal, continue with Step 9.2.
>
> ⚠️ **If nothing happens when you drag a CircuitPython 10.x `.uf2` onto the drive** — it stays `FTHRS3BOOT` and the board never restarts — the board's bootloader is too old for 10.x. A #5691 with the 2023 TinyUF2 bootloader (0.12.3) accepted CircuitPython 9.2.9 but silently ignored a 10.x file. Either stay on **9.2.9** (AeroMorse runs on it), or update the TinyUF2 bootloader first and then flash 10.x. Updating the bootloader **can erase the CIRCUITPY drive**, so back the device up first. Use the library bundle that matches the CircuitPython you end up on (§9.2).

Step 9.2 — Install the required libraries

1. Go to **https://circuitpython.org/libraries**
2. Download the **Bundle** that matches your CircuitPython version.
   - To find your version: look at the file `boot_out.txt` on the CIRCUITPY drive. It will say something like `Adafruit CircuitPython 10.2.0` — download the bundle that matches your major version number (9.x or 10.x).
3. Open the downloaded `.zip` file. Inside is a folder called `lib`.
4. On the **CIRCUITPY** drive, open (or create) the `lib` folder.
5. Copy these items from the bundle's `lib` folder into CIRCUITPY's `lib` folder:

Always copy these — REQUIRED for every build (including the built-in TFT on #5691 / #5483 / #5300):

```
adafruit_hid/                ← entire folder
adafruit_bus_device/         ← entire folder
adafruit_register/           ← entire folder
adafruit_display_text/       ← entire folder  — needed for ALL builds
neopixel.mpy
```

> **Don't skip `adafruit_display_text/` if you have a built-in TFT.** Even though `#5691 / #5483 / #5300` don't need a separate display *driver* library (the display is already initialised via `board.DISPLAY`), every build still uses `adafruit_display_text` to render the group name, Morse buffer, last action, and status line. Missing this folder is the most common installation mistake — `code.py` fails immediately at import with `ImportError: no module named 'adafruit_display_text'`.

Copy these if using the sip-and-puff sensor:

```
adafruit_lps35hw.mpy
```

> **The two libraries that are easy to miss in the bundle** — `neopixel.mpy` and `adafruit_lps35hw.mpy`: - **`adafruit_lps35hw.mpy`** — the tricky one. Your sensor is the **LPS33HW** (#4414), but the CircuitPython driver is named **`lps35hw`** — the same driver covers both the LPS33HW and LPS35HW, Adafruit just named it after the 35. So don't go looking for an "lps33" file; the correct one is `adafruit_lps35hw.mpy`. Library home page: https://github.com/adafruit/Adafruit_CircuitPython_LPS35HW - **`neopixel.mpy`** — a **single bare file**, not a folder, and it has **no `adafruit_` prefix**, so it sorts to the bottom of the bundle's `lib/` folder away from everything else — easy to scroll right past. Library home page: https://github.com/adafruit/Adafruit_CircuitPython_NeoPixel
>
> Both ship inside the library bundle `.zip` you already downloaded — look in its `lib/` folder for the exact filenames above. You do not download them separately.

That is everything the standard build needs. A separate display from Appendix F needs one more driver file, named there.

> You only need the items listed above — you do not need to copy the entire bundle, which is very large.

Step 9.3 — Copy the AeroMorse project files

Download the AeroMorse software (do this first)

The AeroMorse firmware (`code.py`, `boot.py`, `morse_map.py`, `config.py` and the wireless-receiver files) is **not** part of the Adafruit library bundle you downloaded in §9.2. It lives in one place — the official GitHub repository:

> 👉 **https://github.com/jlubin2001/AeroMorse**

Always download from that repo. Copies posted elsewhere on the web may be older or modified versions; the repo is the single source of the latest, correct files.

To download (no Git account or software needed):

1. Open **https://github.com/jlubin2001/AeroMorse/releases/latest** in a browser.
2. Under **Assets**, click `AeroMorse-v1.33.zip` (the number is the version).
3. Unzip it. Everything you copy in the steps below comes from that folder. It also holds the Windows programs, the one-click tools and the PDF guides; keep it as your work folder (Usage Guide, section 1).

(The repo's green `< > Code` button → **Download ZIP** also works. The device files are in the root of that download, and the guides are in its `docs` folder.)

> **Which version do I have?** Every AeroMorse `.py` file has a version and release date in its header comment near the top — e.g. `AeroMorse code.py — version 1.33 (released 2026-10-28)`. Open the file in Thonny (or any text editor) to check. If a file you found somewhere else has no such header, or an older date than the repo, replace it with the repo copy. Keep `code.py`, `boot.py`, `morse_map.py` and `config.py` all at the **same** version. The wireless-display program (`receiver.py`) carries the same version number, so every AeroMorse file from one release says the same version. The main device's startup screen shows its version (e.g. `v1.9`) until the first sip, puff or switch press.
>
> **Filenames must be exact.** CircuitPython looks for `code.py`, `boot.py`, `morse_map.py`, and `config.py` by *literal* name. If you copy a file as `morse_map_darci.py` (or any other variant name) it will be ignored — `code.py` does `from morse_map import groups`, and that import resolves only to a file named `morse_map.py`. **Rename the file to `morse_map.py` *before* or *after* copying** — see the Darci callout below.

The repo root `code.py` is the active firmware — it drives the built-in 240×135 colour TFT of the #5691 Reverse TFT, #5483 and #5300 with no extra file, and a separate display through an optional `display_setup.py` (see Appendix F).

Copy these four files from the unzipped folder to the root of the CIRCUITPY drive (not inside any subfolder):

```
boot.py
code.py
config.py
morse_map.py
```

> **`config.py` holds every user-tunable setting** — sensor thresholds, switch mode, code repeat, audio pitches, etc. You edit `config.py` to change behaviour, not `code.py`. See section 10, and Appendix E for the full reference.
>
> **If you are a Darci USB user starting from `morse_map_darci.py`:** Copy `morse_map_darci.py` to the CIRCUITPY drive, then **rename it to `morse_map.py`** (right-click → Rename on Windows/Mac, or `mv` on Linux). Do not leave both files on the drive — CircuitPython will only load the one literally named `morse_map.py`, and a stale extra file just wastes flash.

The CIRCUITPY drive should now look like this:

```
CIRCUITPY/
├── boot.py
├── code.py
├── config.py          (user-tunable settings — see §10)
├── morse_map.py
└── lib/
    ├── adafruit_hid/                    (always — keyboard/mouse HID)
    ├── adafruit_bus_device/             (always — required by sensor + display libs)
    ├── adafruit_register/               (always — required by sensor + display libs)
    ├── adafruit_display_text/           (always — text labels on the screen)
    ├── neopixel.mpy                     (always — onboard NeoPixel)
    └── adafruit_lps35hw.mpy             (only if using sip-and-puff sensor)
```

> Built-in TFT boards (#5691 / #5483 / #5300) get nothing extra from the per-display list — `board.DISPLAY` is initialised by the firmware, so no driver `.mpy` is needed. They still need every "always" entry above.

Safely eject the CIRCUITPY drive before unplugging. The Feather reboots and runs the code automatically.

> The first boot after copying `boot.py` takes a few extra seconds. This is normal — the board is negotiating USB HID with the host. If nothing seems to happen, unplug and replug the USB cable once.

Step 9.4 — Files for a wireless display (only if you have one)

Skip this step if you are not using the ESP-NOW wireless display (section 5.3). The main board you set up above is already fully functional.

The receiver is a **second** board that runs entirely independently of the main board — its own CircuitPython firmware, its own `code.py`, its own `/lib` folder. It listens for ESP-NOW broadcasts from the main board and mirrors the four display fields on its own screen. It does **not** act as a keyboard or mouse; you do not load `morse_map.py` or `adafruit_hid` on it.

Files on the receiver

File on your computer	Copy to the receiver as
`boot.py`	`boot.py` (same file used on the sender)
`receiver.py`	`code.py` (rename when copying)
`receiver_config.py`	`receiver_config.py` — the display's own settings: wireless channel, screen brightness and rotation, no-signal / sleep / auto-restart times. Optional (defaults are built in), but copy it so you can change them. The main device does **not** use it

> **One `boot.py` for every board.** The same `boot.py` from the sender works unchanged on every receiver — it inspects the filesystem at boot and chooses its role automatically: - If `morse_map.py` is present → **sender**: enables USB HID (Keyboard, Mouse, ConsumerControl). - If `morse_map.py` is absent → **receiver**: leaves HID off so the host doesn't see the second board as a duplicate keyboard. - If `morse_map.py` is absent AND an empty `/hide` file is also present → **hidden receiver**: also disables the CIRCUITPY drive and serial port so the host sees nothing at all. Useful once the receiver is configured and you want a clean USB enumeration.
>
> **Do NOT copy `morse_map.py` to the receiver.** It's the file the boot detection uses to choose sender vs receiver — and the receiver doesn't decode any Morse anyway (it just renders what the main board sends).

Firmware on the receiver (Step 9.1 equivalent)

Install CircuitPython on the receiver the same way you did on the main board in §9.1. The receiver needs **at least CircuitPython 9** so the `espnow` module is included. The main board and the receiver do **not** need to be on the same CircuitPython major version — see §12 "Wireless display → CircuitPython versions on the two boards" for the details.

What the wireless display's drive looks like

```
CIRCUITPY/    (on the W1 receiver)
├── boot.py                        (= same boot.py as the sender)
├── code.py                        (= renamed copy of receiver.py)
└── lib/
    └── adafruit_display_text/     (only library needed)
```

Just **two files** at the root plus **one library folder** under `/lib`. Nothing else. The built-in TFT is initialised via `board.DISPLAY` so no display-driver `.mpy` is needed.

> `adafruit_hid/`, `adafruit_bus_device/`, `adafruit_register/`, `neopixel.mpy`, and `adafruit_lps35hw.mpy` are **not** required on the receiver — leaving them off saves flash and avoids confusion.

Verifying the receiver

1. Plug the receiver into a USB-C power source.
2. Within ~1 second of the **main board** booting, the receiver's display should mirror the four display fields (group, Morse buffer, last action, status).
3. If it stays blank or shows "No signal", open the receiver in Thonny and check the Shell — `ImportError` messages name the missing file. See §12 "Wireless display" for the full diagnostic table.

---

10. Settings for your choices

All settings are in one file, `config.py`, on the device's drive. Open it in Notepad, change the value, save; the device restarts with the new value.

For the standard build you change nothing. These are the only settings that follow from the three choices:

Choice	Setting	Value
Switches	`USE_SENSOR`	`False`
One switch, or two plus "accept"	`SWITCH_MODE`	`1` or `3` (normally `2`)
Window on the computer	`PC_DISPLAY`	`True`
Wireless display	`USE_WIRELESS_DISPLAY`	`True`
	`ESPNOW_CHANNEL`	the same number on both boards
Built-in screen not used	`USE_DISPLAY`	`False`
Built-in screen upside down	`DISPLAY_ROTATION`	`0` or `180`
Sound through the computer	`PC_SOUND`	`True` (with `PC_DISPLAY`)
No speaker on the device	`USE_SPEAKER`	`False` (optional)

Every other setting — how hard a sip must be, how long a pause ends a letter, mouse speed, beep pitch and so on — has a sensible value already and is explained in **Appendix E**. Tune those once the device is

working, one at a time.

Input modes — what `SWITCH_MODE` does

Mode	Behaviour	Group cycling
1	Only `ONE_SWITCH_INPUT` is active. Press ≤ `ONE_SWITCH_DOT_MS` (200 ms) = dot, longer = dash. Pause `ACCEPT_DELAY` = end-of-letter.	Long-press (≥ `LONG_PRESS`) cycles forward. **No backward cycle** — use a g0 Morse pattern.
2 *(default)*	Dot input = dot, dash input = dash. Pause `ACCEPT_DELAY` = end-of-letter.	Long-press dot = cycle back; long-press dash = cycle forward.
3	Dot input = dot, dash input = dash. **Explicit Accept**: long-press of `THIRD_SWITCH_GESTURE` commits immediately.	Long-press of the *other* gesture cycles forward. **No backward cycle** — use a g0 Morse pattern.

For a side-by-side comparison of how AeroMorse, Darci USB, morAce, and Adap2U each implement 1/2/3-switch input, see `MORSE_DEVICES_COMPARISON.md`.

---

11. First Power-On Test

1. Plug the Feather into your computer. **In sensor mode, do not sip or puff for the first few seconds** — the device measures the resting air pressure ("calibrating") as it starts, and a breath in the tube at that moment throws the measurement off.
2. Wait 3–5 seconds. The display shows the **start-up screen**:
   - the device name (`AeroMorse`, or your own `DEVICE_NAME`)
   - the AeroMorse version, e.g. `v1.33`
   - the CircuitPython version, e.g. `CP 9.2.9`
3. Open a plain-text editor on your computer (Notepad, TextEdit) and click inside it so it is focused.
4. Give **one sip, puff or switch press**. This first one only clears the start-up screen — it types nothing. The display now shows **[KEYBOARD]** and the device is ready.

No screen on your board, or `USE_DISPLAY = False`? There is nothing to see, but the device behaves the same way: the first sip / puff / press after start-up is swallowed, the next ones type. (A wireless display, if you use one, shows the same start-up screen.)

You do not need Thonny or a serial console for this test. If one happens to be connected it shows `Calibrating — do not sip or puff ...` followed by `Calibration complete — ready for input.`

Quick test — sensor mode

Gently sip (dot) · pause · gently puff (dash) · pause · puff (dash), then stop. After about a third of a second (`ACCEPT_DELAY`) the letter **W** (`.--`) appears in the text editor. If a speaker is fitted, it beeps once for each sip and puff.

Quick test — switch mode

Press dot switch · dot switch · dash switch (`. . -`), then stop. The letter **U** (`..-`) appears.

If nothing appears

- Make sure the text editor window is focused (click in it).
- Unplug and replug the USB cable — the first connection after installing `boot.py` sometimes needs a fresh plug.
- In sensor mode: make sure the tube is pushed onto the sensor and you are breathing into it, not blowing from the side.
- In switch mode: confirm `USE_SENSOR = False` is set in `config.py`.
- Remember the first sip / puff / press after start-up only clears the start-up screen; enter the test code again.

---

12. Troubleshooting

Grouped by hardware option, in the same order as §4 – §6 (Input → Display → Speaker → Wireless display → Software / general).

Input — sip-and-puff sensor (#4414)

"Calibrating" message hangs at startup

- The LPS33HW was not found. Check the STEMMA QT cable between the Feather and the sensor — both connectors must click in firmly.
- Confirm `adafruit_lps35hw.mpy` is in `/lib` on the CIRCUITPY drive.

False pressure triggers (random dots/dashes while not sipping or puffing)

- Raise `THRESH_SIP` and `THRESH_PUFF` to 3 or 5 in `config.py` if it happens immediately on touching the tube. Combined with a low `DEBOUNCE_SAMPLES` change.
- Check the sip-and-puff tube is not kinked or pinched.
- Confirm auto-zero is on: `BASELINE_DRIFT_S` should be non-zero (default `30`). If it's `0`, the boot-time baseline is fixed and will go stale over minutes/hours.

Pressure bar slowly grows red/green over time, then fires random codes

- Classic symptom of **ambient pressure drift** (weather, HVAC, temperature). The LPS33HW measures absolute atmospheric pressure, so a slow real-world change in the room shows up as a slow drift away from the boot-time baseline — eventually crossing the threshold and firing as a phantom sip or puff.
- Confirm `BASELINE_DRIFT_S` in `config.py` is non-zero (default `30`). If it's `0`, auto-zero is disabled — set it to `30` and reboot (Ctrl+D in Thonny). The pressure bar should stay near zero even after the device sits idle for hours.
- If even with auto-zero on the bar still drifts visibly, lower `BASELINE_DRIFT_S` to `15` to track faster. Don't go below ~10 or slow deliberate sips may start getting absorbed into the baseline.
- See Appendix E "`BASELINE_DRIFT_S`" for the full tuning rationale.

Input — AT switches

AT switches not registering

- Confirm `USE_SENSOR = False` in `code.py`.
- Test the switch with a multimeter in continuity mode: pressing the switch should beep, confirming Tip connects to Sleeve.
- Confirm D5 (dot) and D6 (dash) wires are connected to the correct jack pins.

Input — timing

Pattern fires too early (cuts off long patterns)

- Raise `ACCEPT_DELAY` (default `0.3`) to `0.4` or `0.5` in `config.py`.

Groups cycle when you did not mean to

- Raise `LONG_PRESS` from `1.0` to `1.5` or `2.0` to require a longer hold.

Typing feels slower than my previous Morse device

Experienced Morse users migrating from a switch-based device (Darci USB, morAce, a homebrew paddle) often find they have to deliberately slow down on AeroMorse or the error rate climbs. This is a **tuning** problem, not a limitation — the defaults are deliberately conservative, set for a first-time user who benefits from generous timing. A fluent user should retune.

Understand the failure mode first, because it tells you which knob to turn. With `SWITCH_MODE = 2` a character commits after `ACCEPT_DELAY` of idle. So:

- **Letters running together** (you meant `E` `T`, you got `A`) means you started the next character before `ACCEPT_DELAY` elapsed. The timeout is too long *for your speed*. **Lower `ACCEPT_DELAY`.**
- **Letters splitting apart** (you meant `C`, you got `N` then `T`) means you paused mid-character longer than `ACCEPT_DELAY`. **Raise `ACCEPT_DELAY`.**

Being forced to slow down is the first case. Work through these in order — the first two matter most:

Setting	Default	Try	Effect
`ACCEPT_DELAY`	`0.3`	`0.20`, then `0.15`	Directly sets the pause you must leave between characters. The single biggest throughput lever.
`USE_DISPLAY`	`True`	`False`	**Only if you watch a wireless display, not the built-in screen.** Drawing on the built-in screen stops the firmware reading the sensor for tens of milliseconds each time the text changes — long enough to miss or merge a quick sip/puff. With it off the screen is blanked and those pauses disappear. On the author's own device this was the change that ended missed dots and dashes.
`DEBOUNCE_SAMPLES`	`3`	`2`	Confirms each edge one loop pass sooner (a few ms). Don't go to `1` unless you have a very clean signal.
`THRESH_SIP` / `THRESH_PUFF`	`2`	`2`, or lower	Lower thresholds trigger earlier in the breath. Only lower these if you are *not* getting false triggers; use `test_pressure.py` to check your headroom.

Change **one setting at a time** and type a familiar paragraph after each. Latency changes are easy to misjudge by feel, and stacking three edits at once makes it impossible to tell which one helped.

> **Settings removed in v1.25.** `SENSOR_FILTER_ENABLED`, `SENSOR_FILTER_HEAVY` and `POINTS_TO_AVERAGE` used to be listed here as speed levers. They never did anything: the firmware wrote the filter settings to names the sensor library doesn't have, so the sensor's filter was always off, and the running average was never used. Earlier editions of this guide that told you to change them were wrong. An old `config.py` that still has those lines keeps working; they are ignored.

Repeated symbols collapse — `--` reads as `-`, `..` reads as `.`

A distinct fault with a distinct fix. The signature: the wrong character is always the intended one with a **run of identical symbols shortened by one**, never the wrong symbol and never a dropped isolated symbol. Examples from the default map:

Intended	Pattern	Received	Pattern	Collapse
`c`	`- - - .`	`g`	`- - .`	3 dashes → 2
space	`. . - -`	`w`	`. - -`	2 dots → 1
`LEFT_SHIFT`	`- - . . . -`	`+`	`- . . . -`	2 dashes → 1

Cause. To register two dots in a row the firmware must see *rest → dot → rest → dot*. Pressure has to fall back inside the resting band *between* the two dots and stay there long enough for `DEBOUNCE_SAMPLES` to confirm it. Three things eat that brief dip:

- **your own breath**: in a quick double sip or double puff the pressure often only falls part of the way back before the second one starts, and never gets inside the idle band at all;
- **moments when the firmware is not reading the sensor**: while it redraws the built-in screen (tens of milliseconds whenever the text changes) the dip can come and go unseen. Measured on the author's device before v1.22 these pauses were 60–130 ms on every sip and puff, and they were the main cause of merged and dropped symbols;
- `DEBOUNCE_SAMPLES`, which needs that many consecutive agreeing readings (one per loop pass, a few ms each) before it will believe the IDLE.

Type faster and your gaps between symbols shrink, so the pair merges into one longer symbol.

This is why the fault often **appears right after you lower** `ACCEPT_DELAY` — nothing is wrong with the new value, you simply started keying faster and crossed the point where adjacent symbols can no longer be told apart.

Fix, in order:

0. *(v1.18+)* `REPEAT_SPLIT_PCT = 60` — "split on a dip". A sip/puff that drops below 60 % of its peak and then climbs again by 1 hPa is counted as two, even if the pressure never got back inside the idle band. It leaves the triggers and timing alone, so try it first if your other settings already suit you. `0` switches it off.
1. `USE_DISPLAY = False` — **if you watch a wireless display and not the built-in screen.** Removes the screen-redraw pauses, so the dip between symbols is seen. (v1.25 also made the pressure bar far cheaper to draw for those who keep the screen on.)
2. `DEBOUNCE_SAMPLES = 3` → `2` — fewer samples needed to confirm the brief IDLE.
3. Steps 1 and 2 are usually enough on their own. Only if they are not, consider raising `THRESH_SIP` / `THRESH_PUFF` — but read the warning below first, because on sip-and-puff this often makes things **worse**, not better.

> **Raising the thresholds is a trap for sip-and-puff users.** In theory it should help: the thresholds set the *width of the idle band*, so a wider band means pressure re-enters idle sooner on the way back from each symbol, separating a repeated pair more cleanly. In practice it backfires unless you have a lot of headroom, because the same change also makes every symbol take **longer to cross the threshold on the way up**, and marginal symbols stop registering at all. Breath pressure is modest and slow-changing compared with a switch closure, so most sip-and-puff users have little headroom to spend — the onset penalty outweighs the gap benefit and the error rate climbs. Field result from the project's primary user: raising the thresholds from `2` to `3` made it noticeably worse; what fixed the fault was step 1, switching off the built-in screen he never looked at.
>
> Only try this if `test_pressure.py` shows your peaks are **several times** your current threshold, and change it back promptly if the error rate does not improve.
>
> Do **not** reach for `ACCEPT_DELAY` for this fault. That setting governs the gap *between characters*; this failure happens *within* a single character, and lowering or raising it will not help.

If you still can't get there: consider `SWITCH_MODE = 3` (§"Input modes"). The third-switch Accept gesture commits the character **instantly**, so `ACCEPT_DELAY` stops being a speed limit altogether — you never wait on a timeout. It costs one extra gesture per character, which is a good trade for users who are fast enough that the inter-character pause dominates their typing time. This is also the closest match to Darci USB's end-of-character behaviour.

An extra dash right after a sip (`C` comes out as `=`, `9` as `_`)

The opposite fault: a symbol is *added*. `C` (`---.`) becomes `=` (`---.-`) and `8` (`---..`) becomes `---.-.` — a puff appears straight after the last sip. When a sip ends, the pressure can bounce above normal for a moment, and that bounce is read as a puff.

(v1.33+) Set `SETTLE_MS` in `config.py`. For that many milliseconds after a sip or puff ends, the opposite kind is ignored. On the author's test device every bounce was over within 30 ms, while a real puff right after a sip lasted 70 ms or more, so `SETTLE_MS = 50` stopped the extra dashes without losing real ones. Start at 50–80; raise it if it still happens, lower it if a quick puff after a sip goes missing. `0` (the default) switches it off.

To see the bounce for yourself, set `TRACE_MS = 400`. After each sip ends the device then prints `TRACE` lines to its USB log: milliseconds since the sip ended, the pressure, and `*` where a puff was accepted. Read it with a serial monitor that only listens. Set it back to `0` when done.

On the computer

Some symbols come out wrong (`@` gives `"`, `#` gives `£`, letters swapped)

The computer is set to a keyboard layout other than US. Either switch the computer to a US keyboard, or set `KEYBOARD_LAYOUT` in `config.py` — see **Appendix I**.

Pop-ups on the computer when the device starts or restarts

AeroMorse shows up on the computer as a keyboard, a mouse **and a small USB drive called `CIRCUITPY`** — that drive is how you edit `config.py` and `morse_map.py`. Every time the device starts, restarts (`devicereset`, saving a file, replugging) the drive reconnects, and any program that watches for USB drives reacts with a pop-up near the clock. Typical ones on Windows:

Pop-up	Who shows it	What it wants
AutoPlay — "Select to choose what happens with removable drives"	Windows	To know what to do when a USB drive appears
"New device detected" / an offer to scan the drive	Your antivirus (ESET and others)	To scan the drive
An offer to **back up the device**	Google Drive (other cloud-backup tools do the same)	To copy the drive's contents to the cloud
"There's a problem with this drive — scan and fix"	Windows	Appears if the drive vanished mid-restart without being ejected

They are harmless and safe to ignore. None of them changes anything unless you click it, and they have no effect on typing. If you would rather not see them, switch each one off where it comes from (menu names vary a little between versions):

- **AutoPlay:** Windows *Settings* → *Bluetooth & devices* → *AutoPlay* → turn off **Use AutoPlay for all media and devices**.
- **Antivirus:** look in its settings for **Removable media** (sometimes under device control or detection) and set the action on inserting a drive to **do not scan / do not ask**. In ESET it is under *Setup* → *Advanced setup*. Leave the rest of the antivirus as it is.
- **Google Drive:** Drive icon near the clock → gear → *Preferences* → gear again → **USB devices & SD cards** → untick the prompt to back up. The pop-up itself may also offer *Don't ask again for this device*. This one is worth switching off regardless, so the contents of your device (your key map and your encrypted secrets file) are never copied to the cloud.

Do **not** try to stop the pop-ups by hiding or disabling the `CIRCUITPY` drive — without it you could no longer edit or update the device.

Display

Display is blank or white

- **Built-in screen (#5691):** check `USE_DISPLAY` in `config.py` — `False` deliberately blanks the screen and switches its backlight off. Also check `DISPLAY_BRIGHTNESS` is not set very low.
- **Any other display:** it needs a `display_setup.py` file on the drive (Appendix F "Using a separate display"; Appendix F for the 3.5" FeatherWing). Without one, `code.py` looks only for a screen built into the board. If the file is there and the screen is still blank, the start-up log says why — look for a line starting `display_setup.py`.
- For an OLED: check both ends of both STEMMA QT cables are fully clicked in.
- In the OLED block of `display_setup.py`, try changing `device_address=0x3C` to `device_address=0x3D` if the OLED stays blank — some OLEDs use the alternate address.

Speaker

No sound from the STEMMA Speaker (or any speaker option)

- Verify wiring: Audio → A0, VIN → 3V, GND → GND.
- Confirm the #4046 cable is fully seated in the #3885 speaker's JST PH socket — this is the most common cause of silent speakers.
- Open the serial console in Thonny (Appendix N). If you see `WARNING: pwmio module not available`, you are running an unusual minimal CircuitPython build — `pwmio` is present in every standard build for every supported board. Re-flash with the standard `.uf2` from circuitpython.org/downloads.
- The audio is a square wave (not a sine), so the tone will sound slightly buzzy — that is expected and not a defect.

Wireless display (ESP-NOW)

Wireless display shows "No signal"

- Confirm both boards are powered and running CircuitPython 9.x or later.
- Check the serial console on the main board. It should print `ESP-NOW: wireless display active (broadcast)` at startup.
  - `ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False` → set `USE_WIRELESS_DISPLAY = True` in `config.py` (§10). It ships `False` since most builds have no receiver.
  - `ESP-NOW: module not available on this board` → the board is not an ESP32-family chip. Wireless display requires ESP-NOW, which is ESP32-only.
  - `ESP-NOW: init failed (...)` → see entry below.
- Range is approximately 30 m indoors. Move the boards closer to test.
- Channel mismatch: see "How the ESP-NOW channel is selected" below.

Wireless display resets itself / you see it reboot

- **This is normal, intended behaviour — not a fault.** The ESP32 WiFi radio on the receiver occasionally wedges: packets stop arriving even though both boards are fine. The receiver detects this and **soft-resets itself after 30 s with no signal** so nobody has to unplug it. The timeline on the receiver is:
  - **0–10 s** no packets → nothing shown (brief gaps are normal).
  - **10 s** → the display switches to **"No signal"**.
  - **30 s** → the receiver **soft-resets** (`microcontroller.reset()`); `boot.py` + `code.py` rerun and, if the sender is up, the mirror is back within a couple of seconds.
- **The receiver only auto-resets once it has actually received signal since powering on.** If the sender has never been present this boot — for example the receiver is left plugged into a laptop that is powered **off overnight** — the receiver shows "No signal" and then waits quietly. It does **not** reboot every 30 s all night. When the sender comes back (laptop powers on, main board boots), the receiver is still listening and starts mirroring again on its own with no reset needed.
- **If you see it reboot once when the host computer shuts down**, that is expected: the receiver had signal earlier, so it makes one recovery attempt, then the fresh boot has no signal and it goes quiet.

- **If the receiver reboot-loops continuously while the sender IS up and running**, that points at a persistent link problem rather than an occasional wedge — check the channel match and range (below), and watch the sender's serial console to confirm it is actually broadcasting.
- To change or disable this, set `AUTO_RESET_S` in the display's `receiver_config.py`. Raise the value to reset less eagerly, or set it very high to effectively disable auto-reset.

How the ESP-NOW channel is selected

In CircuitPython 9.x `wifi.radio.channel` is not settable (and on 9.2.9 it isn't even readable). The only reliable way to pin the radio to a specific channel is the **start_ap / stop_ap dance** — start a soft AP on the desired channel, then immediately stop it. The radio stays on that channel. Both boards do this with the same channel:

```
wifi.radio.start_ap(" ", "", channel=ESPNOW_CHANNEL, max_connections=0)
wifi.radio.stop_ap()
espnow.ESPNow()
```

The channel is controlled by `ESPNOW_CHANNEL` in `config.py` (sender) and `ESPNOW_CHANNEL` in `receiver_config.py` (display) — these **must match**. Default is channel 1.

> Do **not** use `wifi.radio.connect()` to join a network for channel selection — `connect()` enables WiFi power-save, which silently drops most incoming ESP-NOW frames.

Sender peer setup — the dummy unicast workaround

CircuitPython issue #9380 causes `send()` to a broadcast peer to fail with `ESP_ERR_ESPNOW_NOT_FOUND (0x3069)` when **only** the broadcast peer is registered. The fix is to register a dummy unicast peer first, then the broadcast peer:

```
e.peers.append(Peer(mac=b'\x02\x00\x00\x00\x00\x01', channel=ESPNOW_CHANNEL))
broadcast = Peer(mac=b'\xff\xff\xff\xff\xff\xff', channel=ESPNOW_CHANNEL)
e.peers.append(broadcast)
e.send(msg, broadcast)
```

The receiver registers **no peers at all** — incoming ESP-NOW frames fire the receive callback regardless of peer registration.

To change the channel (e.g., heavy 2.4 GHz interference on channel 1, common ones to try are 6 and 11):

1. Edit `ESPNOW_CHANNEL` in `config.py` on the main board.
2. Edit `ESPNOW_CHANNEL` in `receiver_config.py` on the display to the same value.
3. Save both — each board restarts by itself. Valid 2.4 GHz channels are 1 – 13 (1, 6, 11 are the non-overlapping ones).

Wireless display is frozen / not updating

- The receiver updates whenever a packet arrives (10× per second). If the display updates briefly then freezes, the main board may have restarted. Check the main board's serial console for errors.

Main board serial console shows `ESP-NOW: init failed`

- The `wifi` module initialisation failed. Try power-cycling the main board. Rarely, the WiFi radio needs a cold boot (unplug power completely rather than pressing Reset).

Repeating `ESP-NOW send error: ESP-NOW error 0x3069`

- 0x3069 is `ESP_ERR_ESPNOW_NOT_FOUND` (peer not registered). On CircuitPython 9.x this happens when the sender has registered only a broadcast peer — see #9380. The fix is built into AeroMorse's `code.py`: a dummy unicast peer (`02:00:00:00:00:01`) is registered before the broadcast peer. If you see this error, confirm you have the current `code.py` — the dummy-peer line should appear immediately before the broadcast peer append.

`ESP-NOW: init failed (ESP-NOW error 0x3065)` on CircuitPython 9.0.x

- 0x3065 decodes as `ESP_ERR_ESPNOW_NOT_INIT` — the underlying ESP-NOW subsystem wasn't ready when `ESPNow()` was constructed. This is a known bug in CircuitPython 9.0.x that was fixed in 9.1+. Two fixes:
  - **Easy:** set `USE_WIRELESS_DISPLAY = False` in `code.py` (§10) to skip ESP-NOW entirely. Acceptable if you don't need the wireless display.
  - **Best:** upgrade CircuitPython on the Feather to **9.2.x or 10.x** from circuitpython.org/downloads. The fix is included in those versions; leave `USE_WIRELESS_DISPLAY = True`.

Wireless display acts as a second keyboard on Windows

- The receiver has `morse_map.py` on it. The shared `boot.py` checks for that file to decide its role: present = sender (USB HID enabled), absent = receiver (HID disabled). If you copied `morse_map.py` onto the receiver by mistake, delete it and reboot the receiver — `boot.py` will then leave USB HID off and the host will stop seeing the receiver as a second keyboard.

CircuitPython versions on the two boards

- ESP-NOW sends plain bytes with no awareness of CircuitPython versions. Each board runs its own CircuitPython and its own `/lib` folder. You can have CP9 on the main board and CP10 on the receiver, or any combination — no steps are needed to make them work together. The only requirement is that **each board runs at least CP9** (earlier versions did not include the `espnow` module). There is no need to match versions, update one when you update the other, or do anything differently when the versions differ.
- **However:** if you copy the `/lib` folder from the main board to the receiver, both boards must be on the **same major version** (both CP9 or both CP10) — `.mpy` library files compiled for one major version will not load under the other. If the boards are on different major versions, download the matching library

bundle for each board separately from circuitpython.org and copy the files individually. The receiver only needs one folder from the bundle: `adafruit_display_text/`.

Software / general

REPL shows `ImportError`

- A library file is missing from `lib/`. Re-read §9.2 and copy the missing file from the CircuitPython bundle. The most common one to miss is `adafruit_display_text/` — see the callout in §9.2.

Nothing happens on the computer at all

- Unplug and replug the USB cable.
- Confirm the cable is a data cable (a drive should appear when plugged in).
- Confirm `boot.py` is on the root of the CIRCUITPY drive.
- Confirm `morse_map.py` is named **exactly** `morse_map.py` (not `morse_map_darci.py` or similar) — `code.py` imports it by literal name. See Step 9.3.

Appendix A — Groups and Group Cycling

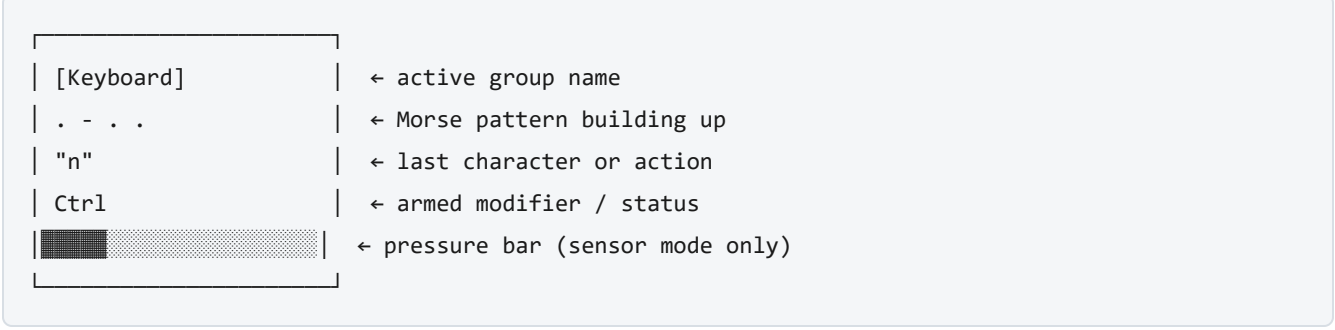
AeroMorse has ten groups: g0 (always-available) plus g1–g9. Group 0 is checked in the background on every keystroke (its 8-symbol patterns work in any group). Groups 1–9 cycle with long-press, or you can jump straight to any group with its 8-symbol Group 0 toggle code.

Group	Contents	8-symbol jump code	How to reach by cycling
0	Group-jump patterns (8 symbols)	—	Always active
1	Letters, numbers, punctuation, function keys	`........`	Power-on default
2	Mouse movement, clicks, Windows shortcuts	`--------`	Long-puff cycling
3	Macro text strings	`.-------`	Long-puff cycling
4	Scanning — Enter, Space and F3–F12 on the 12 shortest codes (Switch Control on iOS / Android)	`.......-` or `---.....`	Long-puff cycling
5	Media — USB HID Consumer Controls (volume / play-pause / mute / track / brightness / eject) on the 12 shortest codes	`......--`	Long-puff cycling
6	Placeholder	`.....---`	Long-puff cycling
7	Placeholder	`...-----`	Long-puff cycling
8	Placeholder	`..------`	Long-puff cycling
9	**Switch** — sip holds Enter, puff holds Space (games / switch apps; see `AEROMORSE_SWITCH_MODE_GUIDE.md`). Returns to Keyboard after 20 s idle. (Group 7 before v1.24)	`.-------`	Only with `.-------` — long-press cycling skips it; leaves by itself

Long sip (hold ≥ `LONG_PRESS` seconds) → cycle backward (…3→2→1→9→8…) **Long puff** (hold ≥ `LONG_PRESS` seconds) → cycle forward (1→2→3→4→…→9→1…)

With 10 groups, cycling all the way around takes a while — the 8-symbol Group 0 jump codes are the fast path to any group from anywhere. The group-toggle codes use a "count of trailing dashes" scheme: 8 dots = g1, then add trailing dashes to reach the higher groups (see `morse_map.py` Group 0 comments for the full list). Group 4 (Scanning) sends F1–F12, making AeroMorse usable as a Switch Control scanning input on iOS and Android, where function keys act as switch actions.

Appendix B — Display Layout



Status line shows: **Ctrl / Shift / Alt / GUI** (sticky modifier armed), **SLOW / FAST / DRAG / RPT** (mouse state). SLOW and FAST show only in groups that move the mouse (v1.33+); the speed is kept, so it is still set when you come back to Group 2.

Appendix C — Customising Macros

Open `morse_map.py`. Find Group 3 near the bottom. Replace the empty strings with your own phrases:

```
g3[3][0b100]  = 'Jane Smith'      # -..   same pattern as letter D
g3[4][0b0010] = 'Thank you!'      # ..-.  same pattern as letter F
g3[3][0b010]  = 'Best regards,'   # .-.   same pattern as letter R
```

Save the file — the Feather reloads automatically.

Appendix D — Other switch and speaker wiring

The main guide gives one way to connect switches without a different board (section 4.2, the jack with screw terminals) and one speaker (section 6.3, the STEMMA Speaker). This appendix holds the other ways. It names the options by letter: B1, B2 and B3 for switches, S1, S2 and S3 for speakers. **B3** is the screw-terminal jack of section 4.2 and **S1** is the STEMMA Speaker of section 6.3.

A note on the #2915 jack's labels. The text below calls its screw terminals Tip, Ring and Sleeve. On the part they are marked for audio: **Left** is the tip, **Right** is the first ring, **GND** is the sleeve, and the unlabelled one is the second ring.

> ⚠️ **Not tested by the project.** Unless a part of this appendix says otherwise, nobody on the project has built what it describes. It is written from the parts' documentation. The build the project uses and tests every day is the standard build in section 2. If you build one of these, please report how it went.

Switches on a breadboard (Option B1)

A build using a half-size breadboard, a single TRRS jack (#1699), and three jumper wires. Lowest-cost path and reversible, but bulky and best suited to bench testing rather than long-term use.

> ⚠️ **This option needs a Feather with header pins.** The Feather stands *in* the breadboard on its two rows of pins, so "solderless" is true only if those pins are already on the board. Most Feathers ship as a bare board with a loose pin strip in the bag — the pins then have to be soldered on first (about 28 joints), or the board bought "with headers". A bare Feather cannot be used on a breadboard at all.
>
> **Not for the recommended #5691.** Its screen is on the underside, so once it is pushed into a breadboard the screen faces down into the breadboard and cannot be read. On a breadboard use the #5483 (screen faces up) instead.
>
> **No header pins, or a #5691?** Use **Option B3** below — three wires, and no breadboard.

Detailed step-by-step breadboard instructions are in Appendix D — Breadboard Wiring Walkthrough at the end of this guide.

For most builders Option B3 (#2915 Terminal Block) is simpler and tidier — try that first.

Parts

Qty	Item	Adafruit #	URL
1	Half-Size Breadboard	#64	https://www.adafruit.com/product/64
1	Breadboard-Friendly 3.5 mm Stereo Jack	#1699	https://www.adafruit.com/product/1699
3	Short male-to-male jumper wires	—	—

Alternative — two jacks, no Y-splitter: Instead of one jack + a Y-splitter, you can use **two #1699 jacks** — one per switch. Each switch plugs directly into its own dedicated jack. This is slightly simpler to wire and removes the Y-splitter from the parts list.

Qty	Item	Adafruit #	URL
1	Half-Size Breadboard	#64	https://www.adafruit.com/product/64
2	Breadboard-Friendly 3.5 mm Stereo Jack	#1699	https://www.adafruit.com/product/1699
4	Short male-to-male jumper wires	—	—

Wiring with two jacks:

Jack	Leg	Feather pin
Jack 1 (dot switch)	TIP	D5
Jack 1 (dot switch)	SLEEVE	GND
Jack 2 (dash switch)	TIP	D6
Jack 2 (dash switch)	SLEEVE	GND

The RING leg on each jack is left unconnected. Both SLEEVE legs can share the same GND pin on the Feather, or use two separate GND holes (the Feather has several GND pins — any of them work).

Step-by-step breadboard walkthrough

Detailed step-by-step instructions for **Option B1 — Breadboard**, above. Skip this appendix if you chose Option B2 (TRRS breakout) or Option B3 (#2915 Terminal Block).

> ⚠️ **Before you start: your Feather must have header pins soldered on.** It stands in the breadboard on those pins. If yours came as a bare board (most do), the pins have to be soldered first — see Appendix M — or choose Option B3, which needs no breadboard. The #5691 Reverse TFT is also a poor fit here: on a breadboard its screen faces down.

Parts — exactly what to buy

Qty	Item	URL	Notes
1	Half-Size Breadboard	https://www.adafruit.com/product/64	Any equivalent generic breadboard also works
1	Breadboard-Friendly 3.5 mm Stereo Jack	https://www.adafruit.com/product/1699	**1 jack only** — the stereo jack handles both switches
1 pack	Male-to-male jumper wires	https://www.adafruit.com/product/153	You will use 3 of them
1	3.5 mm mono Y-splitter	any store	Needed only if you use 2 switches

With header pins already on the Feather: no soldering iron, no solder, no stripped wires — all connections are push-in.

Understanding the breadboard

A breadboard is a plastic block full of small holes. Metal clips inside connect certain holes together so you can make circuits by pushing wires into holes instead of soldering.

```
   ← column numbers →
    1  2  3  4  5  6  7  8  9 ...
a  [ ][ ][ ][ ][ ][ ][ ][ ][ ]
b  [ ][ ][ ][ ][ ][ ][ ][ ][ ]   ← holes a–e in the SAME column
c  [ ][ ][ ][ ][ ][ ][ ][ ][ ]      are all connected to each other
d  [ ][ ][ ][ ][ ][ ][ ][ ][ ]      (5 holes share one clip inside)
e  [ ][ ][ ][ ][ ][ ][ ][ ][ ]
   ──────────────────────────── ← CENTER GAP (nothing crosses here)
f  [ ][ ][ ][ ][ ][ ][ ][ ][ ]   ← holes f–j in the SAME column
g  [ ][ ][ ][ ][ ][ ][ ][ ][ ]      are connected to each other
h  [ ][ ][ ][ ][ ][ ][ ][ ][ ]      (separate from the a–e group)
i  [ ][ ][ ][ ][ ][ ][ ][ ][ ]
j  [ ][ ][ ][ ][ ][ ][ ][ ][ ]
```

Key rule: Holes sharing the same column number AND same side (a–e, or f–j) are connected. Push a wire into column 5 row c and a pin into column 5 row e — they are now electrically connected without any other wire needed.

The center gap is a physical break — nothing crosses it automatically. That is where the Feather board will sit, bridging the two sides.

Step 1 — Place the Feather in the breadboard

The Feather is a long narrow board with a row of pins along each long edge. Press it firmly into the breadboard so it sits across the center gap like a bridge. One row of pins will be in the **e** row, the other in the **f** row.

```
        USB-C port
           ↑
   ┌──────────────────┐
   │                  │
e  ●  ●  ●  ●  ●  ●  ●  ●   ← left-edge pins press into row e
   ═══ FEATHER BOARD ═══
f  ●  ●  ●  ●  ●  ●  ●  ●   ← right-edge pins press into row f
   │                  │
   └──────────────────┘
```

The board should sit level and firm. Every pin in row e now has 4 open holes (a, b, c, d in the same column) you can use to connect to it. Same for row f.

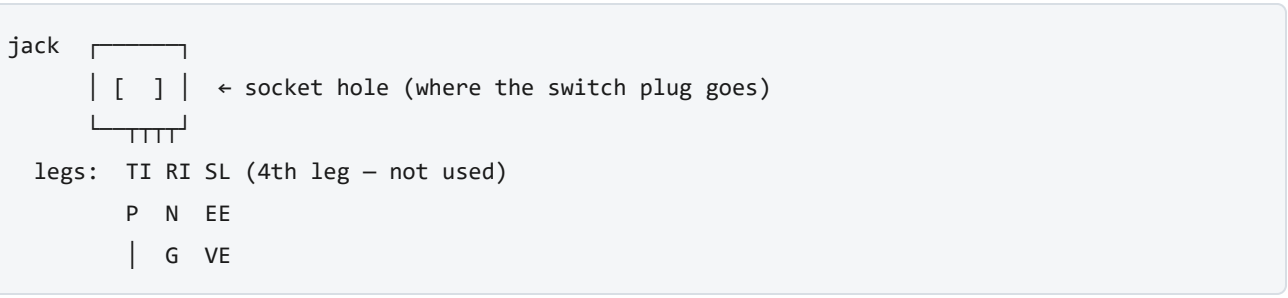
Finding pin D5, D6, and GND on the Feather: Pin names are printed on the *underside* of the Feather board. Before pressing it into the breadboard, flip it over and look. You will see small text next to each pad.

On the #5691, D5 and D6 are on the right-side column of pins; GND appears on both sides. Make a small note of which column numbers they land in once the board is inserted.

> **Tip:** Photograph the underside before inserting so you can refer back to the pin names while looking at the top.

Step 2 — Place the 3.5 mm stereo jack (#1699)

The jack has 4 short metal legs underneath it. Press it into an empty area of the breadboard, several columns away from the Feather so there is room to work. Each leg goes into a different row.



The three legs you will use are labelled on the jack's product page:

- **TIP** — dot switch (switch 1)
- **RING** — dash switch (switch 2)
- **SLEEVE** — ground (common return for both switches)

Because each leg is in its own row, you have 4 open holes in that row to connect to.

Step 3 — Connect with jumper wires

The jumper wires from Adafruit #153 have a male pin on each end that pushes into a breadboard hole. You are going to connect three pairs of holes:

From (jack side)	To (Feather side)
Any open hole in the TIP leg's row	Any open hole in D5's column, row a–d
Any open hole in the RING leg's row	Any open hole in D6's column, row a–d
Any open hole in the SLEEVE leg's row	Any open hole in GND's column, row a–d

Push one end of a jumper wire into the jack's TIP row, and the other end into D5's column. The wire can be any length — just pick one short enough to lie flat without flopping around. Repeat for RING→D6 and SLEEVE→GND.

```
Example layout (column numbers illustrative):

col:  5   6   7  ...  18  19  20
a    [ ] [ ] [ ]     [ ] [ ] [ ]
b    [ ] [ ] [ ]     [ ] [ ] [ ]
c    [J] [J] [J]     [ ] [ ] [ ]   ← jack legs in cols 5, 6, 7 at row c
d    [──────────────────────] [ ]   ← jumper wires run across
e    [D5][D6][GN]    [ ] [ ] [ ]  ← Feather pins in same cols at row e
     ═══════════ GAP ═══════════
f    [D5][D6][GN]    [ ] [ ] [ ]  ← (Feather right-edge, mirror of above)
```

The exact column numbers do not matter — what matters is:

- The jack leg and the Feather pin you want to connect are at **opposite ends of the same jumper wire**
- No two wires accidentally share a hole

Step 4 — Connect your AT switches

You need **one Y-splitter** (3.5 mm mono-to-stereo) to plug two standard AT switches into the single stereo jack:

- Switch 1 (dot) → plugs into the **Tip** side of the splitter
- Switch 2 (dash) → plugs into the **Ring** side of the splitter

If you only have one switch, plug it directly into the jack — it will be your dot (.) switch.

Switches on a soldered jack board (Option B2)

Solders three short wires to an Adafruit TRRS Jack Breakout board (T, R1, S — Ring 2 is unused). The other ends go to the Feather: **soldered to its pads** (3 more joints), or — only if the Feather has header pins — pushed onto the pins using wires with a female socket on that end.

Parts needed:

- Adafruit TRRS Jack Breakout #5764 https://www.adafruit.com/product/5764
- 3 short wires ~8 cm (different colours help)
- Soldering iron + solder

TRRS pad	Feather pin	Wire colour suggestion
T (Tip)	D5	Red
R1 (Ring 1)	D6	Blue
S (Sleeve)	GND	Black
R2	*(no wire — pad left bare)*	—

> **Both switch options work with the same `code.py`** — set `USE_SENSOR = False` in the configuration section.

Steps:

1. Solder a short wire to each pad: T (red), R1 (blue), S (black). Leave R2 bare.
2. Connect the other ends to the Feather — solder each to its pad, or, if the Feather has header pins, use wires with a female socket end and push them onto the pins:

Wire colour	Feather pin
Red (T)	D5
Blue (R1)	D6
Black (S)	GND

3. Plug AT switches directly into the TRRS jack:
   - Dot switch: tip-to-sleeve connection = D5 goes low = dot
   - Dash switch: ring1-to-sleeve connection = D6 goes low = dash

A piezo on a detachable plug (Option S2)

A passive piezo wired via a **3.5 mm jack-and-plug pair**, so the piezo end can be unplugged and swapped. Quieter than Option S1, but every connection on the jack and the piezo is a screw terminal. **The only possible soldering is at the Feather:** with header pins, two jumper wires push on and nothing is soldered; with a bare Feather, two wires are soldered to the A0 and GND pads. Any speaker that already terminates in a 3.5 mm mono plug also plugs straight into the same #2915 jack with no adapter.

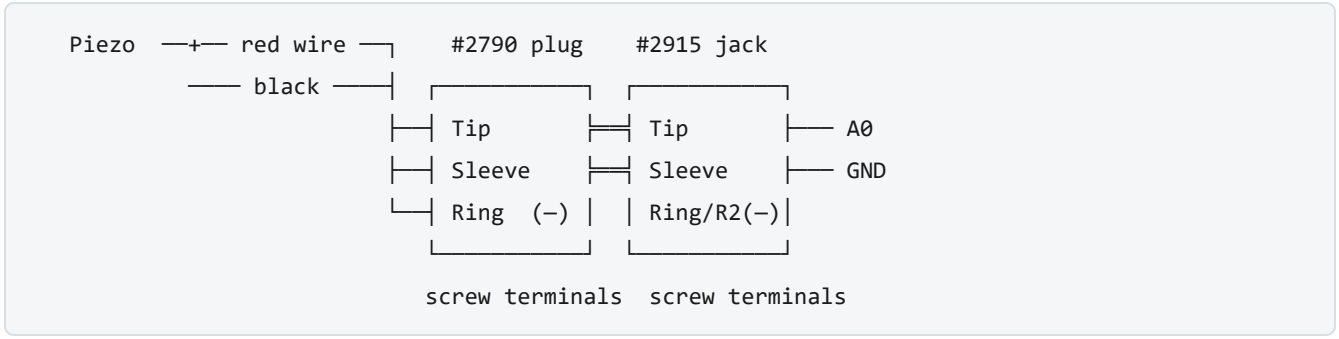
> **Important:** Buy a *passive* piezo, not an *active* one. An active buzzer has a built-in oscillator and will only click, not produce a tone. The two piezos below are confirmed passive and work directly from the Feather's A0 pin via PWM — no resistor, no capacitor, no amplifier (a piezo is a high-impedance capacitive load).

Parts

Qty	Item	Adafruit #	Price	Role
1	TRRS Jack Terminal Block	#2915	$2.50	Jack on the Feather side (screw terminals)
1	3.5 mm Stereo Audio Plug Terminal Block	#2790	$2.50	Plug on the piezo side (screw terminals)
1	Small Enclosed Piezo with Leads	#1740	$0.95	Compact piezo, good for tucking in an enclosure
—	*or* Large Enclosed Piezo with Leads	#1739	$0.95	Slightly larger housing, a little louder
2	Female-to-female jumper wires (if Feather has headers)	#266	—	Connect #2915 to Feather A0 / GND

Total: ~$6 if you already own jumpers.

Wiring



Connection table

Side	Terminal	Connects to	Notes
#2790 plug (piezo side)	Tip	Piezo "+" lead (red)	Polarity match
	Sleeve	Piezo "−" lead (black)	
	Ring	—	Leave empty
#2915 jack (Feather side)	Tip	Feather **A0**	Audio signal
	Sleeve	Feather **GND**	Audio return
	Ring	—	Leave empty (mono plug)
	Ring 2	—	Leave empty

Why this works without signal conditioning

The piezo is a capacitive, high-impedance load — direct PWM drive from A0 (square wave at 800–1200 Hz) makes the piezo flex audibly with no need for a series resistor, AC-coupling capacitor, or amplifier. Same electrical path as the bare-lead piezo of older Option S2 builds; the jack-and-plug pair just adds detachability.

Already have a 3.5 mm-plug piezo or speaker?

If your speaker already terminates in a 3.5 mm mono plug, skip the #2790 plug — the existing plug goes straight into the #2915 jack. You still need the jack itself wired to A0 / GND on the Feather.

Common configurations — how many #2915 jacks you need

Build	#2915 jacks needed	What each one is wired for
Sip-and-puff + on-board STEMMA Speaker (#3885)	**0**	Sensor uses STEMMA QT; speaker uses JST PH (#4046 cable). No 3.5 mm jack involved.
Sip-and-puff + removable 3.5 mm-plug piezo (Option S2)	**1** (Audio)	Tip → A0, Sleeve → GND
AT switches + on-board STEMMA Speaker (#3885)	**1** (Switches)	Tip → D5, Ring → D6, Sleeve → GND
AT switches + removable 3.5 mm-plug piezo	**2** (Switches + Audio)	Jack 1: Tip → D5, Ring → D6, Sleeve → GND · Jack 2: Tip → A0, Sleeve → GND
AT switches + PAM8302 amp + speaker (Option S3)	**1** (Switches)	Tip → D5, Ring → D6, Sleeve → GND. PAM8302 wires to A0/3V/GND directly.

Wiring guide for each role

Switches jack (one #2915, mono or TRS-Y plugs in):

#2915 terminal	Feather pin	Plug type used	What it does
Tip	**D5**	mono switch *or* TRS Y-splitter ring 1	Dot input
Ring	**D6**	TRS Y-splitter ring 2	Dash input
Ring 2	—	(leave empty)	—
Sleeve	**GND**	mono switch sleeve / TRS sleeve	Ground / switch return

> A single mono switch plugged into the same jack gives **dot only** (D5 alone). For both dot and dash, use a TRS Y-splitter (mono-to- stereo) with one switch on each branch.

Audio jack (one #2915, 3.5 mm-plug piezo plugs in):

#2915 terminal	Feather pin	Plug type used	What it does
Tip	**A0**	piezo's Tip lead	Audio (PWM signal)
Ring	—	(leave empty)	—
Ring 2	—	(leave empty)	—
Sleeve	**GND**	piezo's Sleeve lead	Audio return

> If your piezo doesn't already have a 3.5 mm plug, pair it with the #2790 plug-side terminal block — that gives you a solderless chain from the jack to the piezo (see the parts table at the top of Option S2). The two wires to the Feather are solderless only if the Feather has header pins.

Connecting either jack to the Feather

Two short wires per jack from the screw terminals to the matching Feather pin (or pad, if your Feather has no headers). The Appendix D speaker assembly already covers the headers-vs-pads choice — same procedure for the switches jack.

For builders running **both** jacks (the AT-switches + audio combination): wire each jack to its set of pins independently. The Tip terminal of the switches jack goes to D5 *only*. The Tip terminal of the audio jack goes to A0 *only*. No shared wires between the two jacks except GND, which both jacks return to.

A small speaker with a separate amplifier (Option S3)

For louder output or a specific speaker size:

- **Adafruit Mono 2.5 W Class D Amplifier PAM8302 #2130** https://www.adafruit.com/product/2130
- Any small 4 Ω or 8 Ω speaker (3 W or less)
- Amplifier A+ → A0, A− → GND, VIN → 3V, GND → GND
- Speaker connects to the amplifier output terminals

> **This option involves soldering.** The amplifier board's own pin strip and screw terminal are supplied loose, and its four wires to the Feather are soldered to the pads unless the Feather has header pins. Choose Option S1 or S2 for less soldering.

Connecting any of the speakers, with and without header pins

All three speaker options end at the same three Feather signals: **A0** (audio), **3V** (power, only for S1 and S3), and **GND**. How those three signals are connected depends entirely on whether your Feather has header pins.

If your Feather has header pins (or sits in a breadboard)

STEMMA Speaker #3885 + #4046 cable — the #4046 cable ends in three tinned bare wires (black / white / red). The tinned tips are stiff enough to push directly into a breadboard, but for the most reliable contact, either (a) solder a short pin or male header onto each wire tip, or (b) crimp/solder each wire into a female jumper that slides onto the Feather's header pin. Match black → GND, white → A0, red → 3V.

#4046 cable wire	Feather pin
Black (GND)	GND
White (Audio / signal)	A0
Red (VIN)	3V

Option S2 piezo (#2915 + #2790 + #1740/#1739) — the piezo's bare leads go into the #2790 plug's screw terminals (red/+ → Tip, black/− → Sleeve). The #2790 plug then plugs into the #2915 jack. On the Feather side, two short F-F jumper wires from the #2915 terminals to header pins: Tip terminal → **A0**, Sleeve terminal → **GND**. No resistor, no cap.

PAM8302 amplifier (#2130) + speaker — same pin mapping as the STEMMA Speaker (A0, 3V, GND), but soldered to the amp's A+ / VIN / GND pads first. Speaker connects to the amp's two output terminals (polarity not critical).

If your Feather has no header pins (bare PCB pads)

The tinned ends of the #4046 cable have nothing to plug into on a header- less Feather. You have two practical paths — pick one. Both require a small soldering job **on the Feather**, but neither is harder than soldering header pins.

Path 1 — Solder the speaker wires directly to the Feather (recommended)

Cleanest, lowest profile, and the most permanent.

1. Identify the **A0**, **3V**, and **GND** pads on the Feather (printed on the underside).
2. Plug the #4046 cable's JST PH socket end into the #3885 speaker. The other end of #4046 already has three tinned bare wires — no cutting needed. (For the Option S2 piezo build, strip two short wires for the #2915 → Feather link; the piezo itself stays attached to the #2790 plug via screw terminals. For PAM8302, solder the wires to the amp's A+ / VIN / GND pads first.)
3. Trim each tinned wire to ~15 cm if longer than you need, and re-tin the cut ends with a touch of solder.
4. Tin each Feather pad with a touch of solder.
5. Press each tinned wire onto its matching pad and reflow with the iron:
   - Black (GND) → GND pad
   - White / Audio → A0 pad
   - Red (VIN) → 3V pad *(#3885 and PAM8302 only — piezo skips this)*
6. Tug each wire gently to confirm it is fixed.

> A 30 W iron, 0.5 mm solder, and a steady hand are plenty. The pads are larger and more forgiving than header-pin holes.

Path 2 — Solder header pins to the Feather first

Use this path if you also want headers for the AT switch jumper wires, or to make the speaker removable later.

1. Solder header pins onto the Feather's A0, 3V, and GND positions only — you don't need a full row of pins, just the three the speaker needs.
2. Then follow the "header pins" instructions above:
   - #3885 STEMMA Speaker — male pins plug into a breadboard, or use three female-to-female jumpers from the speaker cable to the Feather pins.
   - Option S2 piezo (#2915 + #2790 + piezo) — two F-F jumpers from the #2915 jack's Tip and Sleeve screw terminals to the Feather's A0 and GND pins. The piezo itself attaches to #2790's screw terminals.
   - PAM8302 — wire the amp board to A0 / 3V / GND via female jumpers.

> A local makerspace, library maker lab, or electronics repair shop will usually solder a few pins for free or a few dollars. See Appendix M for more on getting headers done if you don't own an iron.

Path 3 — Skip the speaker entirely

The speaker is optional. The OLED / TFT display already shows every dot, dash, group, and last action — many users do not miss the audio at all. Set up the rest of the device first; add the speaker later if you decide you want it.

Quick reference — all paths, all options

Feather has headers?	#3885 + #4046 cable	Option S2 piezo (#2915 + #2790 + #1740/#1739)	PAM8302 + speaker
Yes (breadboard or pins)	Tin/cap 3 wire ends, push into breadboard rows for A0/3V/GND, or use F-F jumpers	2 F-F jumpers from #2915 (Tip / Sleeve) → A0 / GND pins. Piezo wired to #2790 by screw terminals. **Zero soldering.**	Wire amp to A0/3V/GND via breadboard or F-F jumpers
No (bare pads)	Solder #4046 tinned wires directly to A0/3V/GND pads	Solder 2 wires from #2915 (Tip / Sleeve) screw terminals to Feather A0 / GND pads. Piezo side still screw-terminal.	Solder amp board to A0/3V/GND pads

---

Appendix E — Configuration Reference

Per-setting deep explanations for every entry in `config.py`. Section 10 lists only the settings that follow from your choice of input, display and audio. This appendix covers every setting: what it does and how to tune it.

Settings are grouped here in the same order as in `config.py` itself.

Input — sensor / switches / thresholds

`USE_SENSOR` (default `True`). `True` selects Option A in the build guide (LPS33HW pressure sensor #4414) — sip is a dot, puff is a dash. `False` selects Option B (two AT switches on D5 / D6, or whatever you set `DOT_PIN` / `DASH_PIN` to). The Feather still presents the same USB HID Keyboard / Mouse / ConsumerControl interfaces either way; only the input transducer changes.

`THRESH_SIP` (default `2`), **`THRESH_PUFF`** (default `2`). hPa delta from the calibrated baseline at which a sip or puff is detected. A sip pulls pressure DOWN, a puff pushes it UP; the symbol fires once the delta exceeds the threshold.

- Getting false triggers (sips firing when you're not sipping)? Raise both to `5` or `8`.
- Light sips going unrecognised? Drop both to `2` or even `1`.
- The two thresholds tune independently — many users find their puffs are naturally stronger than their sips and need a higher `THRESH_PUFF` to balance.
- Works in concert with `DEBOUNCE_SAMPLES` — see below.

> **Use `test_pressure.py` to find your personal thresholds.** Copy `test_pressure.py` to the CIRCUITPY drive, open the Thonny Shell, press **Ctrl-C** to interrupt `code.py` and reach the `>>>` prompt *(do not reset — that re-runs code.py)*, then type `import test_pressure`. The script calibrates a baseline for 1 second, then prints a live pressure-delta bar chart for 30 seconds while you sip and puff normally. At the end it suggests `THRESH_SIP` and `THRESH_PUFF` values based on 60 % of your observed peaks — copy those into `config.py`. To run it again in the same session: `exec(open('test_pressure.py').read())` (CircuitPython has no `importlib.reload`).

`DEBOUNCE_SAMPLES` (default `3`). *Sensor mode only.* Number of consecutive readings that must agree on a new state before the firmware accepts a state change. One reading is taken per pass of the main loop (a few milliseconds; the sensor itself delivers a new value every ~13 ms), so `DEBOUNCE_SAMPLES = 3` means a change must hold for roughly 10–15 ms before it takes effect. Filters out brief pressure wobble that would otherwise turn a single dot into dot-dot.

- The big win: with debounce, you can use **low** `THRESH_*` values (2 or even 1) and still get rock-solid detection of every dot and dash. That gives you fast response (no need to press hard) AND wobble immunity.

- Set to `1` to disable. Switch mode reads digital pins which are already clean, so this value is ignored when `USE_SENSOR = False`.

> **Removed in v1.25:** `POINTS_TO_AVERAGE`, `SENSOR_FILTER_ENABLED` and `SENSOR_FILTER_HEAVY`. None of them ever had an effect — the running average was not used, and the filter settings were written to names the sensor library doesn't have, so the LPS33HW's low-pass filter was always off. The readings the firmware acts on are the sensor's raw 75 Hz output; wobble is handled by `DEBOUNCE_SAMPLES`.

`BASELINE_DRIFT_S` (default `30`). *Sensor mode only.* Time constant in seconds for auto-zero drift correction. The LPS33HW sensor measures **absolute** atmospheric pressure, not pressure- relative-to-anything, so the boot-time baseline goes stale as real ambient pressure changes — from weather systems, HVAC cycling, doors opening, temperature changes, anything. Without correction, after 30–60 minutes the device often starts firing phantom sips or puffs as the stale baseline drifts further and further from current ambient.

- While the state machine is **IDLE** (no active sip/puff), baseline drifts toward the current raw reading. The figure is nominal: the correction is applied once per loop pass with a factor of ~0.00044 (`1 / (BASELINE_DRIFT_S × 75)`), and the loop runs 3–4 times faster than 75 passes a second, so with the default `30` a step change in ambient is in practice ~95 % corrected in roughly 20–30 s, not 90.
- During a sip or puff the baseline is **frozen** — a held sip or puff cannot be absorbed into the baseline, so a long one still detects normally.
- Set to `0` to disable auto-zero entirely (return to the original fixed-baseline behaviour). Useful if your environment has perfectly stable atmospheric pressure (rare) or for debugging.
- Tune lower (`15`) for environments with fast pressure changes (close to an HVAC vent); higher (`60` or `120`) if you find the baseline drifting away during slow deliberate sips.

`DOT_PIN` / **`DASH_PIN`** (defaults `board.D5`, `board.D6`). GPIO pins for the AT-switch jacks. Only used when `USE_SENSOR = False`. Wired via 3.5 mm TRRS jack #1699 (B1/B2) or terminal block #2915 (B3) — see section 4.2.

Input mode — 1 / 2 / 3 switch

See **§10 "Input modes — what `SWITCH_MODE` does"** for the mode behaviour table.

`SWITCH_MODE` (default `2`). `1` = single-switch timed (one input; short = dot, long = dash, pause = end-of-letter). `2` = paddle / two-switch (dot input = dot, dash input = dash). `3` = paddle + explicit Accept (long-press of `THIRD_SWITCH_GESTURE` commits immediately).

`ONE_SWITCH_INPUT` (default `"dot"`). *Mode 1 only.* Which physical input is the sole switch. `"dot"` uses the sip (sensor mode) or D5 jack (switch mode); `"dash"` uses puff / D6.

`ONE_SWITCH_DOT_MS` (default `200`). *Mode 1 only.* ms boundary between dot and dash. Press ≤ this is a dot, longer is a dash. Group cycle still kicks in at `LONG_PRESS` regardless.

`THIRD_SWITCH_GESTURE` (default `"long_dash"`). *Mode 3 only.* Which long-gesture serves as the explicit Accept. `"long_dash"` makes a long puff (or long-D6) the Accept; `"long_dot"` makes a long sip (or long-D5) the Accept. The OTHER long-gesture cycles groups forward.

Strong sip / strong puff

A distinct gesture from regular dot/dash and from long-press cycle/Accept. Useful for jumping groups or firing a dedicated action without leaving the current group.

Detection in sensor mode: while a sip/puff is held, the firmware tracks peak pressure delta. As soon as it crosses `THRESH_SIP_STRONG` or `THRESH_PUFF_STRONG`, the configured `STRONG_*_ACTION` fires immediately, the pending Morse pattern is cleared, and no dot/dash is emitted for that press.

Detection in switch mode: switches have no intensity, so the equivalent gesture is a **long-press** of the corresponding switch — the dot switch fires `STRONG_SIP_ACTION`, the dash switch fires `STRONG_PUFF_ACTION`. The long-press is detected at the existing `LONG_PRESS` duration. Setting either action **overrides** the `LONG_PRESS_CYCLES_GROUP` cycle-on-long-press behaviour for that switch. `THRESH_*_STRONG` values are ignored in switch mode.

`STRONG_SIP_ACTION` / `STRONG_PUFF_ACTION` (shipped `"group 2"` / `"group 1"`). Command strings fired when the gesture is detected. Same syntax as g0 toggle codes — most commonly `"group N"` to jump to a specific group. The shipped values jump a strong sip to Group 2 (Mouse) and a strong puff to Group 1 (Keyboard). Set either to `""` to disable that gesture. Honoured only in `SWITCH_MODE = 2`.

`THRESH_SIP_STRONG` / `THRESH_PUFF_STRONG` (defaults `15`). *Sensor mode only.* hPa thresholds. Set noticeably higher than the regular `THRESH_SIP` / `THRESH_PUFF` (3× is a sensible start) so a normal sip can't accidentally trigger a strong gesture.

Timing

`ACCEPT_DELAY` (default `0.3`). Idle pause (seconds) after the last symbol before the pattern commits.

- Patterns committing before you finish? Raise to `0.5` or `0.7`.
- Patterns feeling sluggish? Lower toward `0.2` or `0.15`.
- Sip-and-puff users with a comfortable breath rhythm typically land somewhere in **0.3–0.7**; fast, experienced users may prefer `0.2`.
- In `SWITCH_MODE = 3` this is a safety-net timeout; the third-switch gesture commits instantly regardless of `ACCEPT_DELAY`.

`LONG_PRESS` (default `1.0`). Hold time (seconds) for the long-gesture (cycle / Accept / strong). Raise (e.g. `1.5`) if accidentally triggering on slow symbols; lower (e.g. `0.7`) if a deliberate long-press feels too slow.

Code repeat (Darci-style hold-to-repeat)

`CODE_REPEAT` (default `False`). When `True`, holding a sip or a puff (or the dot or dash switch) emits a stream of dots or dashes at the configured repeat interval — one symbol on press, then one every `DOT_REPEAT_MS` / `DASH_REPEAT_MS` while held. Release ends the stream. `ACCEPT_DELAY` of idle still commits the accumulated pattern. This is the Darci USB "code repeat" behaviour. Honoured only in `SWITCH_MODE = 2`.

`DOT_REPEAT_MS` (default `200`) / **`DASH_REPEAT_MS`** (default `600`). ms per auto-repeated dot / dash. Dash conventionally ≈ 3 × dot but tune to your rhythm.

`CODE_REPEAT_MAX` (default `8`). Cap on symbols per held stream. Prevents buffer overflow if a hold goes on too long.

`LONG_PRESS_CYCLES_GROUP` (default `True`). Independent of `CODE_REPEAT`. `True` = holding a sip or puff for ≥ `LONG_PRESS` cycles groups (sip = back, puff = forward). `False` = long-press does nothing; switch groups via g0 Morse patterns instead. **Recommended `False` alongside `CODE_REPEAT = True`** so a sustained symbol stream doesn't accidentally trigger a group cycle.

Audio

`USE_SPEAKER` (default `True`, v1.29+). `False` switches the device's own speaker or buzzer off: no sidetone and no blips from the device. It does not affect sound through the computer (`PC_SOUND` with the AeroMorse Display program, §5), so you can have the beeps from the computer only, from the device only, from both, or from neither. Use it for a quiet room, or for a board whose built-in buzzer you do not want. A `config.py` without this line behaves as `True`.

`AUDIO_PIN` (default `board.A0`). PWM output pin for the speaker. All speaker options (S1 STEMMA Speaker, S2 jack-and-piezo, S3 PAM8302 amp) wire to A0 / GND. The output is a 50% duty square wave at the chosen frequency — see Build Guide §6.

`BEEP_DOT_FREQ` (default `1200`) / **`BEEP_DASH_FREQ`** (default `800`). Hz — dot (sip) and dash (puff) sidetone frequencies. Different pitches make it easier to hear what symbol just fired. Raise both proportionally if a piezo's resonance peak is higher; the STEMMA speaker is reasonably flat across these frequencies.

`CONFIRM_FREQ` (default `1050`) / **`GROUP_FREQ`** (default `550`). Hz — short blips fired when an action commits (`CONFIRM_FREQ`) and when groups change (`GROUP_FREQ`).

`BEEP_CONFIRM_S` (default `0.06`) / **`BEEP_GROUP_S`** (default `0.14`). Seconds — duration of the confirm and group-change blips.

Mouse (Group 2 movement speeds)

Effective pixels per step = `raw direction × MOUSE_SPEED_* × MOUSE_SPEED_FACTOR`, and then Windows multiplies that by its own pointer speed (Settings → Bluetooth & devices → Mouse → *Mouse pointer speed*). The defaults are set for Windows' **default pointer speed**: **10** on Windows 11's 1–20 slider, the middle. (The

older Control Panel slider, *Mouse Properties → Pointer Options*, has 11 steps; there the default is the 6th.) On a PC where the pointer speed is turned up to the fastest, the same settings move about 3.5 times as far — use `MOUSE_SPEED_FACTOR = 2` there.

`MOUSE_SPEED_NORMAL` (default `2`) — default speed when group 2 is entered.

`MOUSE_SPEED_SLOW` (default `1`) — toggled via the `mslow` command.

`MOUSE_SPEED_FAST` (default `3`) — toggled via the `mfast` command if you map it.

`MOUSE_SPEED_FACTOR` (default `7`) — overall scale. `7` on a PC at Windows' default pointer speed feels the same as `2` on a PC with the pointer speed at the fastest, which is how AeroMorse was tuned (and before v1.31 the default was `2`). With *Enhance pointer precision* on — Windows' default — the pointer also speeds up for fast moves and slows down for slow ones, so the feel is close rather than exact.

`MOUSE_SCROLL_FACTOR` (default `2`, v1.31+) — scale for the scroll codes. Windows' pointer speed does not change scrolling, so it has its own setting. Without it in `config.py`, scrolling uses `MOUSE_SPEED_FACTOR`, as before v1.31.

`MOUSE_REPEAT_DELAY` (default `0.040`). Seconds between repeat ticks when mouse-repeat is active (40 ms = 25 ticks/sec).

`MOUSE_CLICK_MOD_DELAY` (default `0.030`). Seconds to hold an armed modifier before and after a mouse click. The keyboard and the mouse are two **separate USB HID interfaces**, so the host is free to process their reports out of order. Without a settle delay the click frequently lands before the modifier registers and the host sees a plain click — the reason **Ctrl+click**, **Shift+click**, and **Alt+click** used to do nothing. If modified clicks are still unreliable on your host, raise this to `0.05`.

`MOUSE_CLICK_KEEPS_MODS` (default `True`). Controls whether an armed modifier survives a mouse click.

- `True` — the modifier stays armed after the click, so **Ctrl+click multi-select** works the way it does for an able-bodied user: arm Ctrl **once**, click each file you want, then toggle Ctrl back off with the same pattern that armed it. The status line keeps showing `RCtrl` the whole time so you can see it is still held.
- `False` — one-shot behaviour: the modifier clears after a single click, so every modified click costs two patterns (arm, then click).

> **Selecting several files in Windows File Explorer.** In Group 2, arm **Right Ctrl** (`-.-.`), then send **left click** (`.-`) once per file — the modifier stays armed between clicks. When the selection is complete, enter `-.-.` again to release Ctrl — or just **right-click** one of the chosen files to copy or move them: the right-click lets go of Ctrl for you (see the next setting).
>
> `RIGHT_CLICK_CLEARS_MODS` (default `True`, v1.33+). A right-click is sent without the armed Ctrl / Shift / Alt / Win and turns them off, so after picking files with Ctrl or Shift you can go straight to the right-click menu. (Shift+right-click would open Windows' longer menu instead.) `False` = a right-click uses the armed keys like a left-click.
>
> Explorer also has a **mouse-only** alternative that needs no modifier at all: turn on **View › Show › Item check boxes** (Windows 11) or **View › Item check boxes** (Windows 10). Each item then gets a checkbox, and plain single clicks on the checkboxes accumulate a multi-selection. This is often far less effort than modified clicking and it survives an accidental un-modified click, which would otherwise collapse the whole selection.

`MOUSE_CLICK_HOLD` (default `0.060`). Seconds the mouse button is held down for each `mclick`. The firmware does an explicit press → hold → release rather than a single instantaneous click, because a **zero-duration click is ignored by Windows scrollbar tracks** and a number of other controls — they need the button held for a few tens of milliseconds to register. This is the reason a `mdrag` toggle would scroll (it holds the button down) while a plain click on the same scrollbar did nothing. ~60 ms is reliable; raise it if scrollbar or button clicks still don't take, lower it toward `0.03` if clicks feel sluggish.

> **Scrollbar tip.** Clicking the scrollbar **track** (the empty area above or below the thumb) scrolls one page; clicking the little **arrows** at the ends scrolls one line. Clicking the **thumb** itself does *not* scroll — you drag it, which on AeroMorse is `mdrag left` (`-.`) to grab, a few `mmove` steps, then `-.` to release. The mouse wheel patterns (`mwheel up` / `mwheel down`) are usually the least effort for scrolling and don't depend on cursor position over the bar at all.

`MOUSE_CLICK_GAP` (default `0.040`). Seconds between the two clicks of a double-click (`mclick left 2` / `mclick right 2`). Only used when the click count is 2. The full double-click therefore takes `HOLD + GAP + HOLD` ≈ 160 ms, well within Windows' default 500 ms double-click window. Raise it only if double-clicks are being seen as two single clicks.

Shift + arrows

`SHIFT_SELECT` (default `True`, v1.30+). A sticky modifier normally lets go after the one key that follows it. With this on, Shift is the exception when a *navigation* key follows — an arrow, Home, End, Page Up or Page Down: the armed modifiers stay on, so Shift, arrow, arrow, arrow … keeps extending the highlighted text, and Shift, arrow, `repeat` extends it continuously. Arming Ctrl together with Shift selects a word at a time. The modifiers let go by themselves at the next key that is not a navigation key — Ctrl (so Ctrl then C is a plain copy), Delete, a letter — or when Shift is sent again. Everything else is unchanged: Shift then a letter is

still one capital, and Ctrl then an arrow is still a single word jump. `False` restores the behaviour before v1.30, where Shift let go after one key. A `config.py` without this line behaves as `True`.

Repeat exclusions

`NO_REPEAT_KEYS` (default `("PAGE_UP", "PAGE_DOWN")`). A tuple of `Keycode` names (as strings) that must never auto-repeat. After you fire one of these, pressing `repeat` does nothing instead of re-firing the key. Page keys are excluded by default because an accidental repeat scrolls far past where you were and you lose your place. Add any others you don't want to repeat, e.g. `("PAGE_UP", "PAGE_DOWN", "HOME", "END", "ESCAPE", "TAB")`.

- *(v1.20+)* An entry can also be a **key combination**: join the names with `+`, e.g. `"ALT+TAB"`, `"GUI+TAB"` or `"RIGHT_CONTROL+RIGHT_ALT+LEFT_ARROW"`. It blocks exactly that combination (keys in any order). A single-key entry such as `"TAB"` only blocks that key pressed on its own, so `SHIFT+TAB` still repeats unless you list it, and `LEFT_ARROW` alone still repeats.
- The **arrow keys are intentionally not** in this list — Up/Down arrow held with `repeat` is a normal, fine-grained way to scroll, and you stop it with any sip/puff.
- Names are matched against `adafruit_hid.Keycode`. A name that isn't a valid keycode is ignored with a warning at boot rather than crashing, and an older `config.py` without this setting falls back to the page-key default.

Display / wireless

`DISPLAY_ROTATION` (default `0`). Display orientation in degrees. `0` = USB-C port on the LEFT side of the display, `180` = USB-C port on the RIGHT. `90` / `270` are also valid but place the text vertically — useful if you've mounted the Feather sideways.

`USE_WIRELESS_DISPLAY` (default `False`). `True` enables the WiFi radio at boot and broadcasts the display state via ESP-NOW for a wireless display to mirror. Default is `False` because most builds are a single board — **set it `True` only when you actually have a receiver paired.** Adds roughly 80–100 mA to the current draw while running. Has no effect on non-ESP32 boards — the `espnow` import fails there and the radio stays off regardless.

`ESPNOW_CHANNEL` (default `1`). The 2.4 GHz WiFi channel (1–13) the sender broadcasts on. **It must match `ESPNOW_CHANNEL` in the display's `receiver_config.py`** or the receiver hears nothing. Only relevant when `USE_WIRELESS_DISPLAY = True`. Change it only if channel 1 is congested in your area. See §12 "How the ESP-NOW channel is selected" for the full mechanism and why the channel is pinned the way it is.

`KEYBOARD_LAYOUT` (default `"US"`, v1.28+). Which keyboard layout the computer is set to, so that characters written in `morse_map.py` are typed with the right keys. `"US"` uses the table built into the firmware and needs no extra file. Any other value names a layout file in `/lib` — `"win_uk"` loads `keyboard_layout_win_uk` (the full file name is accepted too). If the file cannot be loaded the US table is used and the reason is printed at start-up. See Appendix I for where to get the files and what is and is not affected.

`PC_DISPLAY` (default `False`, v1.26+). `True` makes the device also write its four display fields to the USB serial log, as one short line each time something changes, for the **AeroMorse Display** program on the computer (section 5.2). Nothing is sent as keystrokes, and each line takes about 2 ms — far less than drawing the built-in screen. Leave it `False` if you do not use that program; the lines would only clutter the log.

`PC_SOUND` (default `False`, v1.27+). `True` makes the device also report each beep the moment it starts and stops, so the **AeroMorse Display** program can play it through the computer's speakers (section 6.2). It is the way to get audio feedback from a device that has no speaker fitted. It needs `PC_DISPLAY = True` (that is how the program finds the device), and *Sound* must be switched on in the program's right-click menu. Each report takes about 1 ms. The pitches come from the `BEEP_*` / `*_FREQ` settings under Audio.

`PC_HANDOVER` (default `False`, v1.32+). `True` lets a program on the computer — **SwitchMorse Windows** — take over the Morse decoding. While that program is running and asking for it, the device types and clicks **nothing** itself: it only reports each sip and puff starting and ending on the USB serial log, and the display shows **PC MODE**. The program asks by sending a short line of plain text about twice a second (never Ctrl+C, so it cannot stop the device). If those lines stop for 2 seconds — the program is closed or crashes, or the computer goes to sleep — the device goes straight back to typing by itself. While it runs, the AeroMorse Display program cannot be open at the same time (only one program can use the device's port).

Appendix F — Other displays

The main guide covers three displays, all tested: the screen built into the #5691, the window on the computer, and the wireless display (section 5). This appendix holds every other screen. **Before buying one, consider the window on the computer** (section 5.2): it gives text of any size with no hardware at all.

> ⚠️ **Not tested by the project.** Unless a part of this appendix says otherwise, nobody on the project has built what it describes. It is written from the parts' documentation. The build the project uses and tests every day is the standard build in section 2. If you build one of these, please report how it went.

Which displays are tested

Only the built-in screen works straight out of the box. `code.py` drives the screen that is part of the board (`board.DISPLAY`) with no extra file. Every other display on this page is supported by CircuitPython's `displayio` system and *can* be used, by adding one small file — see "Using a separate display" below — but the project has not built or tested any of them.

Display	Status
Built-in screen on the **#5691** Reverse TFT Feather	✅ **Tested** — the author's daily device. Works as shipped, no code change
Built-in screen on the #5483 / #5300 TFT Feathers	Expected to work as shipped (same `board.DISPLAY`, same size) — not tested by the project
3.5" TFT FeatherWing #3651 on a #5477	Needs a `display_setup.py` file; step-by-step in **Appendix F** — written from the parts' documentation, not from a tested build
Other FeatherWings (#5872, #3315)	⚠️ **Untested** — needs a `display_setup.py` file
Standalone / breakout TFTs (#2050, #1743, #1770, #4311)	⚠️ **Untested** — needs a `display_setup.py` file
STEMMA QT OLEDs (#326, #938)	⚠️ **Untested** with the current firmware — needs a `display_setup.py` file; text is small
Wireless display (second board, ESP-NOW)	✅ **Tested** — has its own ready-made `receiver.py`; see below

If you want a display that simply works, choose the #5691 — and if you need a bigger or more distant screen, see "Want a bigger wireless display?" further down this section.

Using a separate display — `display_setup.py` (v1.26+)

A display that is not built into the board is set up by **one small extra file on the CIRCUITPY drive, `display_setup.py`**. You never edit `code.py` for a display, and updating AeroMorse to a new version does not undo your display set-up.

1. Copy `display_setup.example.py` from the repo to the CIRCUITPY drive and rename the copy `display_setup.py`.
2. Open it. It holds a ready-made block for each display family (3.5" and 2.4" FeatherWings, breakout TFTs, OLED). Keep the one for your display and check the pin names against your wiring.
3. Copy that display's driver library into `/lib` (named in the block, and in the table at the end of this section).
4. Save. The device restarts.

What you get:

- **The layout fits itself to the screen.** Text size, row positions and the pressure bar are worked out from the screen's width and height — about 20 characters across and 4 rows — so a 480×320 screen gets text twice the size of the built-in one, with nothing to adjust.
- **A mistake cannot stop the device typing.** If the file has an error, a library is missing or the display does not answer, AeroMorse falls back to the built-in screen (if the board has one) or runs without a screen, and prints the reason in its start-up log.
- **The same file works on a wireless-display board** — `receiver.py` looks for `display_setup.py` too.

- The validator checks `display_setup.py` for typing mistakes when it finds one in the folder.

The file's one rule: it must contain `def setup():` and `setup()` must return the display. The blocks in the example have **not been run on real hardware** — they are written from each display's documentation — so treat them as a starting point, and please report what works.

Boards with a built-in screen (#5691, #5483, #5300) must **not** have a `display_setup.py` — they need nothing.

Understanding the columns

- **Form factor:** a FeatherWing plugs directly onto the Feather with no wiring — **but only onto a Feather that has header pins soldered on**. A breakout / standalone display needs 7 wires, soldered at the display end.
- **Interface:** SPI displays have no I^2C address conflicts with the sensor.
- **Resolution:** Higher = more text/detail. The Morse buffer, group name, last action, and status line all fit comfortably on 320×240 or larger.

FeatherWing displays (plug directly on Feather — easiest wiring, requires headers)

> ⚠️ **These need a `display_setup.py` file, and none is tested.** The #3651 has a step-by-step walkthrough in Appendix F (not from a tested build). See "Using a separate display" at the top of this appendix.

Display	#	Size	Resolution	Interface	URL
3.5" TFT Resistive Touch	3651	3.5"	480×320 colour	SPI	https://www.adafruit.com/product/3651
3.5" TFT Capacitive Touch	5872	3.5"	480×320 colour	SPI	https://www.adafruit.com/product/5872
2.4" TFT Resistive Touch	3315	2.4"	320×240 colour	SPI	https://www.adafruit.com/product/3315

> **Header pins required on the Feather.** A FeatherWing has two rows of female header sockets on its underside that slide down onto the male pins on top of the Feather. If your Feather shipped as a bare PCB with empty pin holes, those holes must be filled with soldered male headers before a FeatherWing can be used — there is nothing for the wing to grip otherwise. The right part is the pair of **male pin strips that normally come loose in the bag with the Feather**; if yours are missing, any 0.1" break-away male header works (#392). (Not the #2886 "Feather header kit" — those are *female* sockets.) Adafruit's own pages for the #5477 and #5691 do not offer a pre-soldered version; some resellers do — look for "headers soldered" in the listing. If you cannot or would rather not solder, choose a STEMMA QT OLED (#326 / #938) or the built-in TFT on #5691 instead — both are plug-and-play with no headers.
>
> FeatherWings have a touchscreen controller that uses I²C. You will not use touch input with AeroMorse — the display portion works via SPI independently and will not conflict with the pressure sensor.

Recommended for table mounting: #3651 — largest FeatherWing, bright colour, plugs straight on (once headers are in place) with no additional wiring.

Standalone / breakout displays (7 wires required)

> ⚠️ **Untested with AeroMorse — needs a `display_setup.py` file.** Nothing in this table has been built and tried by the project; see "Which displays are tested" and "Using a separate display" at the top of this appendix before buying one.

Display	#	Size	Resolution	Driver	URL
3.5" TFT with Touch	2050	3.5"	480×320 colour	HX8357D	https://www.adafruit.com/product/2050
3.2" TFT with Touch	1743	3.2"	320×240 colour	ILI9341	https://www.adafruit.com/product/1743
2.8" TFT with Touch	1770	2.8"	320×240 colour	ILI9341	https://www.adafruit.com/product/1770
2.0" IPS TFT	4311	2.0"	320×240 colour	ST7789	https://www.adafruit.com/product/4311

Standalone displays are useful when you want the display mounted separately from the Feather — for example, a display on an arm at eye level with the Feather tucked out of the way.

OLED displays (STEMMA QT plug-and-play, monochrome, no colour)

Display	#	Size	Resolution	Driver	URL
0.96" Monochrome OLED	326	0.96"	128×64	SSD1306	https://www.adafruit.com/product/326
1.3" Monochrome OLED	938	**1.3"**	128×64	SSD1306	https://www.adafruit.com/product/938

Both connect via STEMMA QT — no soldering. Both use the same SSD1306 driver, the same library, and the same I²C address (0x3C), so the two are interchangeable: whatever works for one works for the other.

#326 (0.96") is compact and easy to mount on a tube holder. **#938 (1.3")** is 35% larger with identical wiring and code — a better choice if you want a slightly bigger display without the complexity of a TFT.

> **These need a `display_setup.py` file** (see "Using a separate display" at the top of this appendix; the example file has an OLED block). The layout shrinks itself to 128×64, which makes the text small — about 2 mm tall on the 0.96" screen. (The very first AeroMorse code, "v2", was written for the #326 OLED; it was retired in v1.14.) An OLED has **not been tested** with the current firmware. If you just want it to work, use the #5691.

Parts for a separate display

> **If you chose the #5691 Reverse TFT Feather, skip this section entirely — the display is already part of that board.**
>
> ⚠️ **Every display in this table needs a `display_setup.py` file, and none has been tested by the project** (the #3651 has written instructions in Appendix F). Read "Which displays are tested" above before ordering.

Qty	Item	Adafruit #	URL
1	3.5" TFT FeatherWing Resistive (largest; **Feather needs header pins**)	#3651	https://www.adafruit.com/product/3651
—	*or* 3.5" TFT FeatherWing Capacitive (**Feather needs header pins**)	#5872	https://www.adafruit.com/product/5872
—	*or* 2.4" TFT FeatherWing Resistive (**Feather needs header pins**)	#3315	https://www.adafruit.com/product/3315
—	*or* 3.5" TFT Breakout (standalone, needs 7 soldered wires)	#2050	https://www.adafruit.com/product/2050
—	*or* 2.8" TFT Breakout (standalone)	#1770	https://www.adafruit.com/product/1770
—	*or* 1.3" OLED — STEMMA QT (larger OLED; plugs in, no header pins)	#938	https://www.adafruit.com/product/938
—	*or* 0.96" OLED — STEMMA QT (compact; plugs in, no header pins)	#326	https://www.adafruit.com/product/326

If you choose the OLED #326, you also need:

Qty	Item	Adafruit #	URL
1	STEMMA QT 4-pin cable 400 mm	#5385	https://www.adafruit.com/product/5385

Connecting a separate display

> The #5691's built-in screen needs no connecting. The displays below are **untested with AeroMorse** and also need a `display_setup.py` file before they show anything (Appendix F "Using a separate display").

If you chose a FeatherWing display (#3651, #5872, or #3315)

The Feather must already have its header pins soldered on. A bare Feather cannot be attached to a FeatherWing.

1. Hold the FeatherWing with the display facing up.
2. Line up the Feather's two rows of header pins with the two rows of sockets on the back of the FeatherWing.
3. Press firmly and evenly until the two boards sit flush. The pins should go in all the way.

That is all — no wires needed.

If you chose the OLED #326

1. Plug one end of the **100 mm STEMMA QT cable** into the STEMMA QT port on the Feather. The connector is keyed and only fits one way — do not force it.
2. Plug the other end into either STEMMA QT port on the **LPS33HW sensor**.
3. Plug one end of the **400 mm STEMMA QT cable** into the remaining port on the LPS33HW.
4. Plug the other end into the STEMMA QT port on the **OLED** (#326).

Chain: **Feather → 100 mm cable → LPS33HW → 400 mm cable → OLED**

If you chose a standalone TFT breakout (#2050, #1770, etc.)

You need 7 short wires, one for each row of the table. **This is a soldering job:** the wires are soldered at the display (breakouts arrive with their pin strip loose), and at the Feather they are soldered to the pads — or, if the Feather has header pins, pushed onto them with female jumper wires:

Display pin	Feather pin
SCK	SCK
MOSI (or SI)	MOSI
CS	D9
DC	D10
RST	D11
GND	GND
VIN	3V

> Pin labels vary slightly between breakout boards. Match by function name, not physical position. Refer to the Adafruit product guide for your specific board.

Display driver library required per display chip

Chip	Library	Used by
Built-in (board.DISPLAY)	*(none — built in)*	#5691 Reverse TFT Feather — uses the repo root `code.py`
SSD1306	`adafruit_displayio_ssd1306`	#326 (0.96" OLED) and #938 (1.3" OLED)
ILI9341	`adafruit_ili9341`	#3315, #1770, #1743
HX8357D	`adafruit_hx8357`	#3651, #5872, #2050
ST7789	`adafruit_st7789`	#4311

> **Note:** Any display other than a built-in one needs a `display_setup.py` file naming its driver (see "Using a separate display" at the top of this appendix). Apart from the built-in screen, these combinations are **untested**.

A bigger wireless display

The wireless display is the easy place to get a bigger screen: it is a separate board, so nothing you try there can affect typing. Since v1.26 `receiver.py` sizes its text to whatever screen it finds. Your choices, from least to most effort:

Choice	Screen	What you need	Status
The computer's own monitor	any size	The "AeroMorse Display" program above and `PC_DISPLAY = True` — no hardware	✅ Tested
A board with a larger built-in colour screen	2" and up	Any ESP32-S3 board whose screen CircuitPython drives by itself (`board.DISPLAY`) and whose circuitpython.org page lists the `espnow` module. Copy `receiver.py` as `code.py` exactly as for a second #5691 (Step 9.4) — no display file needed	⚠️ Untested — should work, please report
A Feather plus a FeatherWing TFT (e.g. #5477 + 3.5" #3651)	up to 3.5"	Header pins soldered on the Feather, the display's driver in `/lib` , and a `display_setup.py` (see "Using a separate display" above) next to `receiver.py`	⚠️ Untested

Before buying a board for the second row, open its page on circuitpython.org and check two things under "Built-in modules available": `espnow` is listed, and the board description says the display is built in. An ESP32-S2 or ESP32-S3 chip is required for ESP-NOW.

Text size follows the screen: 240×135 gives the size you see on the #5691, 320×240 the same size with more room around it, 480×320 twice that size.

A 3.5" screen on a #5477, step by step

A consolidated walkthrough for builders pairing the bare-Feather **ESP32-S3 Feather #5477** with the **3.5" TFT FeatherWing Resistive Touch #3651** — the largest readable display AeroMorse supports (480×320 colour). Same chip family as the recommended #5691, so all behaviour (USB HID, ESP-NOW, etc.) is identical; only the display path differs.

The cost of this build relative to #5691 is **one extra file on the drive** (`display_setup.py` , v1.26+) and one extra library. The text enlarges itself to suit the bigger screen. Everything else (sensor, speaker, AT switches, ESP-NOW wireless display) is identical to the recommended build.

F.1 When this build is the right pick

Want	This combo
Big, glance-able display with much more text area than #5691's 240×135	✅ 480×320 is ~4.4× the pixel area
Bright, colour, backlit display	✅ HX8357D TFT with built-in backlight
Same input + ESP-NOW capabilities as #5691	✅ ESP32-S3 chip family is identical
Built-in display (no separate display to buy)	❌ — display is a separate FeatherWing
Display + Feather as a single PCB (panel-mount friendly)	❌ — two PCBs stacked together
Cheaper than #5691	❌ — #5477 + #3651 costs more than #5691 alone

Pick this combo if you want the **biggest display** and don't mind a two-board stack or one code-edit step. Pick #5691 instead for the simplest single-board build.

F.2 Parts list

The hardware-side parts list is the standard sensor / speaker / switch options from §7 with these display-path changes:

Qty	Item	Adafruit #	Notes
1	**ESP32-S3 Feather 4MB/2MB PSRAM** — with its header pins soldered on	#5477	The pins come loose in the bag and must be soldered — see §F.3
1	3.5" TFT FeatherWing Resistive Touch	#3651	The display itself; resistive touch is not used by AeroMorse
—	*(everything else from §7 Sensor / Speaker / Input choices)*	—	Sensor (#4414 + STEMMA QT cable), speaker (S1/S2/S3), AT switches if not using sensor

You do **not** need any extra wires, cables, or breakouts beyond the two boards above — the FeatherWing plugs directly onto the Feather's headers.

F.3 Header pins on the #5477 — required

#5477 ships as a bare PCB with empty through-hole pads. The FeatherWing has female sockets on its underside that need male header pins on the Feather to grip. The "build without header pins" path documented in Appendix M does **not** work with FeatherWings — the wing's holes can't grip flat pads, and gripping the through-holes from below is not mechanically reliable.

Three ways to get headers on:

Path	Cost	Effort
Buy a #5477 **with headers already soldered** from a reseller that offers it (Adafruit's own page does not)	Varies	None
Bring the #5477 and the loose pin strips from its bag to a makerspace / library maker lab / electronics shop	Usually free or a few dollars	Drop off, pick up
Solder them yourself	Cost of an iron if you don't have one	~30 minutes for a beginner

If you don't already own a soldering iron, a **pre-soldered board from a reseller** or the **makerspace** path are the practical options. See Appendix M for makerspace pointers.

F.4 Hardware assembly

Once headers are on the #5477:

1. **Mount the #3651 onto the #5477.** Hold the FeatherWing above the Feather with the display facing up. Line up the two rows of holes on the FeatherWing with the two rows of male header pins on the Feather. Press firmly and evenly downward until the wing sits flush. The pins should click through all the way.
2. **Connect the sensor** (sip-and-puff builds). The Feather's STEMMA QT port sits on the side edge of the Feather PCB. The FeatherWing's PCB is **above** the STEMMA QT port, so you can plug a STEMMA QT cable from the side without removing the wing. Run the cable to the LPS33HW #4414 as in section 2.

3. **Connect the speaker.** A0 / 3V / GND are passed through to the FeatherWing's top-side pin headers. The STEMMA Speaker #3885 + #4046 cable plugs onto those header pins exactly the same way it would on a bare Feather. See Appendix D.

4. **Connect AT switches** (switch-mode builds). D5 and D6 are also passed through. Wire as in section 4.2.

USB-C, the Reset button, and the user button on the Feather all remain accessible. The FeatherWing has cutouts and pin pass-throughs in the standard FeatherWing layout.

F.5 Software setup

Libraries on /lib

The standard /lib set from §9.2 *plus* the HX8357D driver:

```
/lib/
├── adafruit_hid/                    (always)
├── adafruit_bus_device/             (always)
├── adafruit_register/               (always)
├── adafruit_display_text/           (always)
├── neopixel.mpy                     (always)
├── adafruit_lps35hw.mpy             (sip-and-puff builds)
└── adafruit_hx8357.mpy              (THIS COMBO — for the #3651 driver)
```

Project files

The same `boot.py`, `code.py`, `config.py`, and `morse_map.py` as the standard build, **plus one more file:** **`display_setup.py`**. No file is edited by hand.

Add `display_setup.py`

The #5477 has no built-in display, so `code.py` on its own finds no screen and runs without one (it still types). To use the #3651:

1. Copy **`display_setup.example.py`** from the repo to the CIRCUITPY drive.
2. Rename the copy **`display_setup.py`**.

That is all — the block that is active in the example file is the one for this FeatherWing:

```
def setup():
    import busio
    import fourwire
    from adafruit_hx8357 import HX8357
    displayio.release_displays()
    spi = busio.SPI(board.SCK, MOSI=board.MOSI, MISO=board.MISO)
    bus = fourwire.FourWire(spi, command=board.D10, chip_select=board.D9, reset=None)
    return HX8357(bus, width=480, height=320)
```

The `D9 = CS` / `D10 = DC` pin assignment is Adafruit's standard FeatherWing convention — those are the pins the #3651's CS and DC traces wire to on the FeatherWing PCB. `DISPLAY_ROTATION` and `DISPLAY_BRIGHTNESS` in `config.py` are applied afterwards as usual.

The layout sizes itself

Nothing needs retuning. `code.py` works out the text size from the screen: on this 480×320 panel the four rows are drawn at **twice** the size used on the #5691 (text scale 4 instead of 2 — characters 24 px wide, 20 across), centred on the screen, with a 16 px pressure bar along the bottom edge.

> Before v1.26 this build needed two hand edits inside `code.py` (a display set-up swap and a table of label sizes) that had to be redone after every update. If you made those edits to an older `code.py`, simply install the current `code.py` unchanged and add `display_setup.py`.

F.6 Configuration

Once the display is up, configuration is identical to any other build: edit `config.py` per §10 (and Appendix E for the deep notes). Set `USE_SENSOR`, thresholds, switch mode, and so on the same way you would on a #5691. The display layout edits above don't interact with any of those settings.

F.7 Verifying it works

A healthy first boot on this build should show:

```
Baseline: <pressure> sip<...> puff>...
Calibration complete — ready for input.
Input mode: 2-switch  (1-sw input = dot, 3-sw accept = long_dash)
Code repeat: off  long-press cycles group: True
Baseline auto-zero: 30s time constant
ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False
```

...with one extra line near the top of the log:

```
Display: from display_setup.py (480 x 320)
```

...and the 3.5" TFT should display the standard four-row layout with the pressure bar at the bottom, at the larger scale.

If the display stays blank, read the start-up log (Appendix N — mind the warning there about Thonny). A line beginning `display_setup.py` names the problem:

- `display_setup.py could not load (no module named 'adafruit_hx8357')` — the driver library is missing from `/lib`.
- `display_setup.py failed (...)` — the display did not answer; check the FeatherWing is pressed fully onto the header pins.
- No such line at all, and `No board.DISPLAY on this board` instead — the file is not on the drive, or is not named exactly `display_setup.py`.

In every one of these cases the device still types; only the screen is missing.

If text rotates the wrong way, change `DISPLAY_ROTATION` in `config.py` (valid values: `0`, `90`, `180`, `270`).

> This appendix is written from the parts' documentation; the combination has not been built and tested by the project.

Appendix G — AutoHotkey: starting programs with a Morse code

AeroMorse types keys; it cannot start a program, or put the pointer at a place on the screen, by itself. **AutoHotkey** is a free Windows program that fills the gap: it watches for a key combination — say Ctrl+Alt+D — and does whatever a small script tells it to. Give that key combination a Morse code, and one code does the job.

The project supplies the script ready to use, **`AeroMorse.ahk`**, with three hotkeys, and the supplied `morse_map.py` already has a code for each:

Hotkey	What it does	Morse code	Group
Ctrl+Alt+D	**AeroMorse Display** window (section 5.2) on / off	`.-.-.-`	0 — works from every group
Ctrl+Alt+H	**AeroMorse Help** — your cheat sheet with a tab per group — on / off	`..-..-`	0 — works from every group
Ctrl+Alt+Home	Mouse pointer to the middle of the screen	`-..--`	2 — Mouse

Setting it up takes three small steps, once. AutoHotkey is Windows-only. Until the script is running the three codes do nothing useful — they send the key combination and nothing answers it.

G.1 Get AutoHotkey

Download **AutoHotkey v2** from **https://www.autohotkey.com** and run the installer (the defaults are fine). It is free and open-source.

G.2 Put the script in your work folder

Copy `AeroMorse.ahk` from the project into your **AeroMorse work folder** — `Documents\AeroMorse`, the folder set up in Usage Guide §1 — next to `AeroMorse Display.exe` and `AeroMorse Help.html`. The script looks for those two files in its own folder, so **there is nothing in it to edit**.

Double-click `AeroMorse.ahk` to start it. A green **H** icon appears near the clock and the three hotkeys work. If one of the two files is not in the folder, the hotkey shows a short note saying so, which closes by itself.

Keeping the script in the work folder puts everything for your AeroMorse in one place. It lives on the computer, not on the device, so **a backup of the device does not include it** — when you back up, copy the work folder too.

G.3 Make it start with Windows

Press **Win+R**, type `shell:startup` and press Enter — a folder opens. Put a **shortcut** to `AeroMorse.ahk` in that folder — a shortcut, not the script itself, which stays in your work folder (right-click the file → *Show more options* → *Create shortcut*, then move the shortcut there). From the next sign-in the hotkeys are always ready.

G.4 The three hotkeys

Ctrl+Alt+D — AeroMorse Display on / off. Opens the window if it is closed and closes it if it is open. It comes back at the size, place and see-through level you left it.

Ctrl+Alt+H — AeroMorse Help on / off. `AeroMorse Help.html` is the cheat sheet made for looking things up while you work: the legend stays at the top with one **tab per group** under it, and only the group you pick is shown — no scrolling through pages.

- Change group: click a tab, press the group's number (`0` – `9`), or use the left / right arrow keys.
- Change size: the **+** and **–** buttons in the top right corner, or the `+` and `-` keys. Only the page grows; the buttons stay where they are.
- It remembers the tab and the size you left it at, and fits the tabs and the columns to the width of the window.

It opens in a plain Microsoft Edge window of its own, with no browser tabs or address bar; the same hotkey closes it. The file is made next to `AeroMorse Cheat Sheet.pdf` every time the cheat sheet is built (*Build my*

cheat sheet.bat, or `python build_pdfs.py --folder` with your work folder), from **your** `morse_map.py`, so it always matches your map. The copy that comes with the project shows the supplied map.

Ctrl+Alt+Home — pointer to the middle of the screen. For when the pointer is lost in a corner or far from where you want it. The device cannot do this by itself — a USB mouse only says "move a little this way" and never knows where the pointer is — but AutoHotkey can. With more than one monitor it goes to the middle of the main one.

Finding the pointer needs no AutoHotkey: the Mouse-group code `...-` taps Ctrl, and Windows draws circles round the pointer if *Show location of pointer when I press the CTRL key* is ticked (Control Panel → Mouse → Pointer Options).

All three hotkeys were tried on the author's computer (Windows 11, Microsoft Edge), sent from an AeroMorse.

G.5 The Morse codes

These lines are already in the supplied `morse_map.py`. To use a different code, change the pattern on the line; `morse_map_report.txt` lists the free ones. A code in **Group 0** works from every group.

```
g0[6][0b010101] = Keycode.CONTROL, Keycode.ALT, Keycode.D      # .-.-.-  AeroMorse Display on / off
g0[6][0b001001] = Keycode.CONTROL, Keycode.ALT, Keycode.H      # ..-..-  AeroMorse Help on / off
g2[5][0b10011]  = Keycode.CONTROL, Keycode.ALT, Keycode.HOME   # -..--   centre pointer
```

Changing the map is an ordinary edit: back up, edit the work copy, run the validator, and copy the file to the device only after it says PASS (Usage Guide §3).

G.6 Your own hotkeys, and changing the script

The same idea works for any program. Open `AeroMorse.ahk` in Notepad and add a line at the end; this one opens Notepad with Ctrl+Alt+N:

```
^!n:: Run("notepad.exe")
```

In a hotkey `^` means Ctrl, `!` means Alt and the letter is the key — change the letter if that combination is already used by something else on your computer. Then give the new combination a Morse code, as in the lines above.

After editing the script, right-click the green **H** icon near the clock and choose *Reload Script* — the change takes effect at once.

If a hotkey does nothing: check the green **H** icon is there (the script is running), that `AeroMorse.ahk` is in the same folder as the program or page it opens, and that no other program already uses the same key combination.

When the project's `AeroMorse.ahk` is updated, hotkeys you added yourself are not carried over — keep a note of them, or keep them in a second script of your own.

(This appendix is repeated in the Usage Guide as §7.)

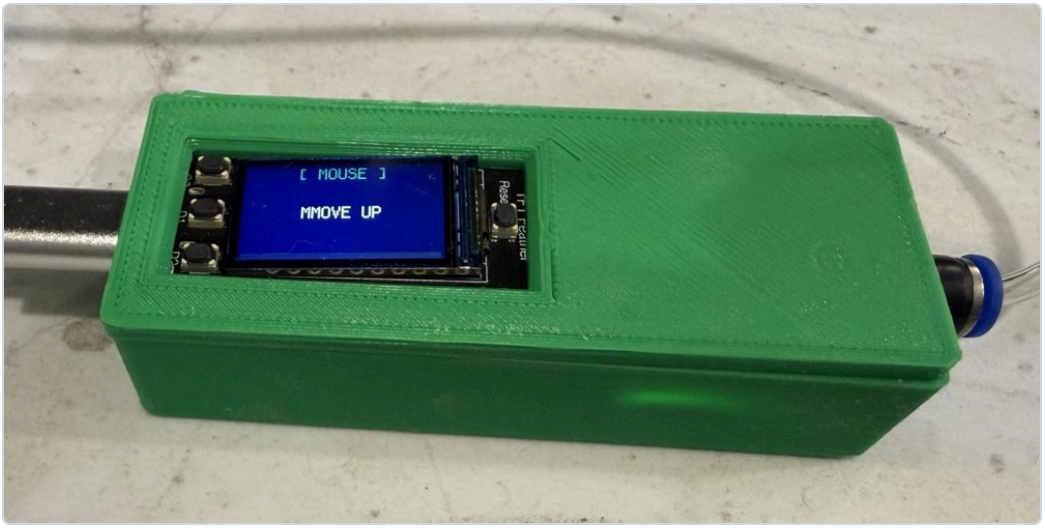
Appendix H — Enclosure and air-tube strain relief

Everything earlier in this guide gets you a working AeroMorse: a bare circuit board, a sensor on a short cable, and a tube pushed onto the sensor. That is fine on a bench. For daily use two things are still missing, and **neither is included in the parts lists or the prices in this guide**:

- **An enclosure** — something to hold the boards so they are not loose on a desk or a wheelchair tray, where they can be knocked, shorted against metal, or pulled by their cables.
- **Strain relief for the air tube** — the important one. The tube sits on a port only a few millimetres tall on the sensor board. Every tug on the tube (moving your head, the tube catching on something, a helper tidying up) pulls directly on that port. Without something else taking the pull, the tube works loose or comes off — and with it your access to the computer — or the sensor board is dragged around by its thin cable.

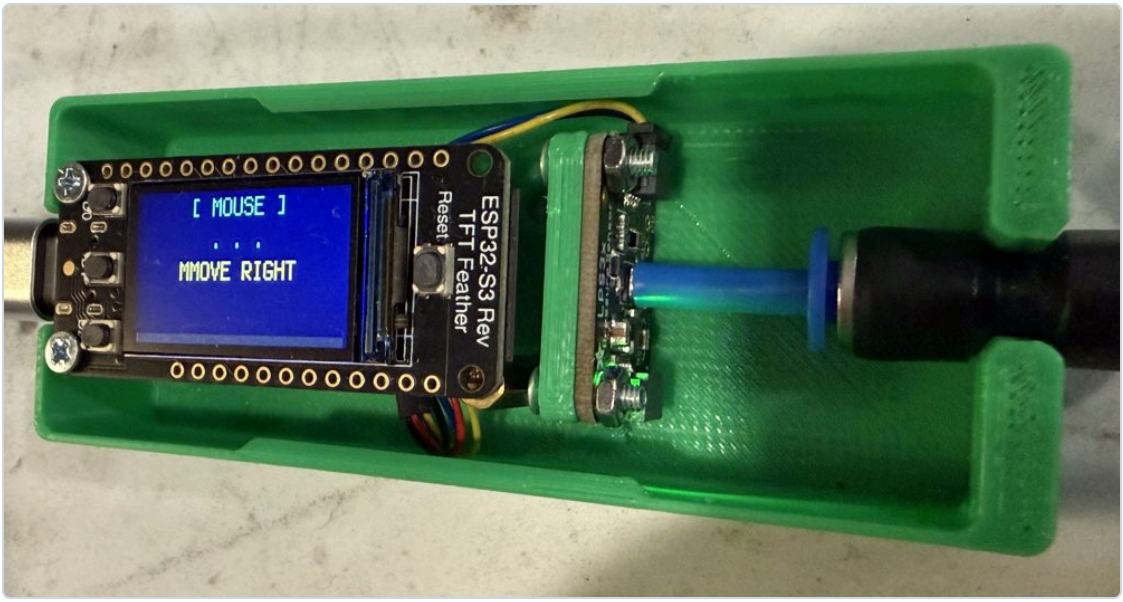
A good enclosure solves both at once. If you have no enclosure yet, do the strain relief anyway (H.3).

H.1 The author's enclosure — the ATMakers "HoseBox"



Closed: the screen and the three buttons show through the lid. USB cable on the left, tube fitting on the right.

The author's two devices sit in a 3D-printed box designed by **ATMakers** (Bill Binko, the maker of AirTalker, which inspired AeroMorse). ATMakers built the first one; the second was printed by an online printing service from the same design files.



Lid off. Left to right: USB cable, the Feather (screen up), the sensor board screwed to an inner wall, a short tube, and the tube fitting held by the end of the box.

How it is laid out:

Feature	What it does
Two parts: a base and a snap-on lid, about **96 × 37 × 23 mm**	Small enough to mount on a tube holder, tray or arm
Window in the lid	The #5691's screen and its three buttons show through. This is the "panel mount" use the Reverse TFT Feather is made for (Appendix L)
Feather on four nylon stand-offs	Held clear of the base, screen facing the lid
Inner wall with two screw holes	The sensor board is screwed to it, port pointing at the end of the box
Slot in one end	The USB-C cable comes out here
Clip in the other end holding a push-in tube fitting	**This is the strain relief.** A short piece of tube runs *inside* the box from the fitting to the sensor port. Your long tube plugs into the fitting on the *outside*. A tug on the tube pulls on the box, not on the sensor — and the tube can be unplugged and replaced without opening anything

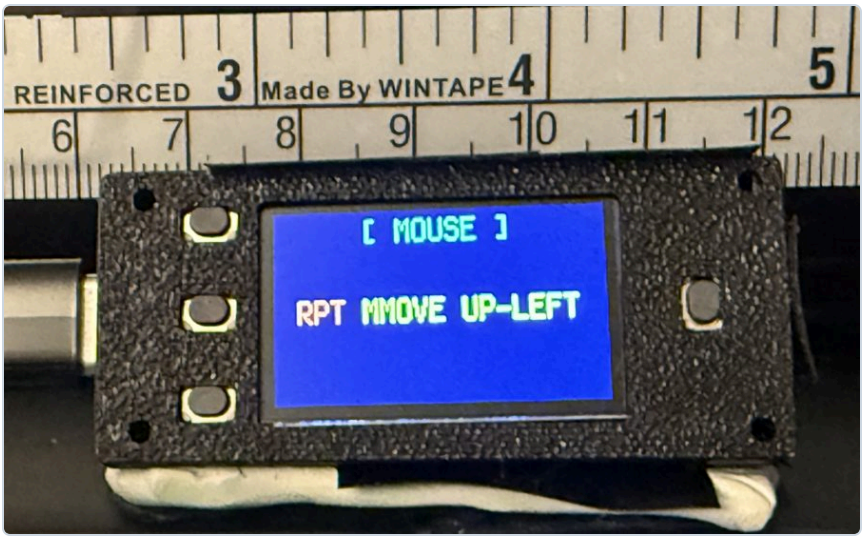
Besides the printed parts it uses small nylon screws and stand-offs (an M2.5 set — #3299 black or #3658 white, see §7) and one push-in ("push-to-connect") pneumatic tube fitting. The fitting's exact size is not recorded here yet — match it to your tube.

An earlier version is published. ATMakers' original sip-and-puff box, made for AirTalker, is on Thingiverse: Sip And Puff Interface by atmakers, with a video walk-through of the design, ATMakers Sip and Puff

Enclosure Design. That version is for a Feather **without** a built-in screen and for an older pressure sensor whose wires had to be soldered, so it does not fit the #5691 and the plug-in LPS33HW used in this guide as it stands — but it shows the same idea, and is a starting point if you can adapt a 3D model. The HoseBox described here is the later version, reworked for the #5691's screen and the STEMMA QT sensor.

Getting the HoseBox design. The files are ATMakers' work and are **not in the AeroMorse repository yet**. A reworked, adjustable version of them (in OpenSCAD) has been drawn but has **not been test-printed**, so it is not published either. If you would like to print this enclosure, open an issue at https://github.com/jlubin2001/AeroMorse and ask; this section will be updated when files can be shared.

H.2 A case for the wireless display



The wireless display in its case — about 52 × 23 mm.

The author's wireless display — a second #5691 — is in a small two-part printed case: a tray underneath and a bezel on top with openings for the screen, the three buttons and the reset button. The two halves are meant to be held together by four small screws through the Feather's mounting holes. It has no tube, so it needs no strain relief; the case is only for protection and for mounting it where it can be seen.

This design is published and free to download: Adafruit ESP32-S2 Reverse TFT Feather Case by **badabby**, on Printables.

- It was made for the ESP32-**S2** Reverse TFT Feather. The S3 board used in this guide (#5691) has the same outline, and the case fits it — the author's is on an S3.
- Print the two files named `feather-tft-rev-case-front.stl` and `feather-tft-rev-case-back.stl`. The same page also has a deeper version with room for a small battery (#3898).
- It takes four M2 × 10 mm screws (the deeper version up to six). The designer's advice: print without supports, and clean out the screw recesses with a Phillips screwdriver. **Order the screws with the print** — M2 × 10 mm is not a size most people have to hand, and they do not come with the Feather. The author did not have screws long enough and simply wrapped a strip of black electrical tape around the two halves, as in the photo above. The tape is holding.

- **Mounting it.** The author's display is stuck to the laptop, just below the screen, with a strip of reusable adhesive putty (the white strip under the case in the photo). It holds the display where it can be seen, peels off cleanly, and needs no bracket.
- **Licence: Creative Commons Attribution-NonCommercial 4.0.** You may print it and adapt it for your own use, with credit to the designer; it may not be sold. For that reason the files are not copied into the AeroMorse repository — download them from the designer's page.

A newer case, made for the S3 board — not tried yet. Adafruit ESP32-S3 reverse tft by **iamatulsingh**, on Printables (published September 2026), is designed for the #5691 itself and is also a two-part case: a base and a lid (`adafruit_esp32s3_reverse_tft-base.stl` and `-lid.stl`). The designer suggests 0.2 mm layers, 3 walls, 15–25 % infill, in PLA or PETG. Its licence is Creative Commons Attribution-ShareAlike 4.0, which also allows commercial use. The author thinks it looks the better of the two but **has not printed or used it**, so whether it needs screws, and how well it fits, is for you to find out — reports welcome.

H.3 No enclosure yet? Do the strain relief anyway

These need no printed parts, only a couple of small cable ties. The first two are from Adafruit's own sip-and-puff guide (LPS33 sip-and-puff); the third is ordinary good practice.

1. **Tie the tube onto the port** (Adafruit guide, "Assemble the sensor"). A small cable tie tightened around the tube where it sits on the port stops it sliding off and keeps it sealed. Cut the tube end square first, push it on with a gentle twist, then tighten the tie. Adafruit notes this is not what the port was designed for — it is made to seal against an enclosure with an O-ring — but it works.
2. **Tie the tube to the USB cable — this is the strain relief** (Adafruit guide, "Plug it all together", under *Optional but Suggested*). Plug the USB cable into the Feather, fold the tube back so it lies alongside the cable, and hold the two together with a second cable tie. A tug on the tube is then taken by the sturdy USB cable instead of by the little sensor port. Two things to watch:
   - leave the tube a **gentle bend**, not a kink, between the sensor and the tie, so air still flows; a slight twist helps it curve clear of the screen;
   - make the tie **snug, not tight** — firm enough to hold, not so hard that it squashes the tube shut.
3. **Fix the sensor board to the Feather.** Do not let the sensor hang from its STEMMA QT cable. Adafruit's guide screws its boards together through their mounting holes with M2.5 nylon screws and nuts; a small plate or box does the same job. With the boards joined, a pull that does get through cannot drag the sensor or unplug its cable.

The author has not used methods 2 and 3 day to day — his devices are in the enclosure above — so treat them as Adafruit's recommendation rather than something tested here.

Check the tube now and then for moisture, and that it is still fully on the port — a tube that has crept half off is a common cause of missed dots and dashes (§12).

H.4 Getting something 3D-printed without a printer

You do not need to own a 3D printer. Online printing services print from design files and post the parts to you (the author's second box was made this way), and many public libraries, schools and makerspaces print

small parts for the cost of the plastic. Prices vary widely with the service, the material and the size; **budget for it separately** from the electronics.

Any small project box also works if you would rather not print: what matters is that the boards are held still, the screen can be seen, and the tube is anchored to the box rather than to the sensor.

Appendix I — Keyboard layout: when symbols come out wrong

The symptom. Letters and numbers are right, but some symbols are not: you enter the code for `@` and get `"`, `#` gives `£`, `\` or `|` give something else — or, on a French or German computer, some letters are swapped too (`a` and `q`, `y` and `z`).

The cause. A USB keyboard does not send characters. It sends *which key was pressed*, and the computer decides what that key means from its own keyboard-layout setting. AeroMorse presses the keys that produce each character on a **US** keyboard. On a computer set to another layout the same keys mean something else. Nothing is faulty; the two simply disagree.

There are two ways to make them agree.

I.1 Easiest: set the computer to a US keyboard

If you can, add the "English (United States) — US" keyboard in the computer's language settings and select it while using AeroMorse. Nothing on the device changes. This suits anyone who types mostly in English.

I.2 Or: tell AeroMorse which layout the computer uses (v1.28+)

1. **Get the layout file.** Layout files for CircuitPython are published by the community project **Neradoc/Circuitpython_Keyboard_Layouts** (free, MIT licence). On its *Releases* page download the zip that matches your CircuitPython version — for CircuitPython 9.x, `circuitpython-keyboard-layouts-9.x-mpy-….zip`.
2. **Copy one file to the device.** In the zip, find the file for the computer's layout and copy it into the `lib` folder on the CIRCUITPY drive. Only that one file is needed.

Computer's keyboard layout	File	Setting
United Kingdom	`keyboard_layout_win_uk`	`"win_uk"`
German	`keyboard_layout_win_de`	`"win_de"`
French (AZERTY)	`keyboard_layout_win_fr`	`"win_fr"`
Spanish	`keyboard_layout_win_es`	`"win_es"`
Italian	`keyboard_layout_win_it`	`"win_it"`
Danish	`keyboard_layout_win_da`	`"win_da"`
Swedish	`keyboard_layout_win_sw`	`"win_sw"`
Portuguese	`keyboard_layout_win_po`	`"win_po"`
Brazilian	`keyboard_layout_win_br`	`"win_br"`
Czech	`keyboard_layout_win_cz`	`"win_cz"`
Hungarian	`keyboard_layout_win_hu`	`"win_hu"`
Turkish	`keyboard_layout_win_tr`	`"win_tr"`
French, on a Mac	`keyboard_layout_mac_fr`	`"mac_fr"`
US Dvorak	`keyboard_layout_us_dvo`	`"us_dvo"`

The `win_` files are made for Windows; other systems usually agree with them for the common symbols, but not always. 3. **Name it in `config.py`:**

```
python KEYBOARD_LAYOUT = "win_uk"
```

4. Run the validator, copy `config.py` to the device, and try the symbols that were wrong.

If the file is missing, misnamed or faulty, the device still types — it goes back to the US layout and prints the reason in its start-up log (a line beginning `KEYBOARD_LAYOUT:`). When it works the log says `Keyboard layout: keyboard_layout_win_uk`.

I.3 What the setting does and does not change

- **Changed:** everything written in `morse_map.py` as a character or as text — `'@'`, `'a'`, a phrase, a secret. Each character is typed with the key (and Shift or AltGr) that produces it on the chosen layout. A letter or digit sent with an armed Ctrl / Alt / Win also uses the right key, so Ctrl+A is still Ctrl+A on a French keyboard.
- **Not changed:** entries written as key names — `Keycode.ENTER`, `Keycode.EQUALS`, `Keycode.CONTROL`, `Keycode.ALT`, `Keycode.D`. Those name a key *position* and always press that position. `Keycode.Q` is the key where a US keyboard has Q, which is A on a French one. For anything layout-sensitive, write the character instead of the key name.

- **Not possible:** a character that is not on the chosen layout at all (for instance `£` on the US layout) is skipped, not typed wrongly.

> **Tested how far?** On the project's test device the UK and French files were loaded and the keys sent for a set of characters were checked against what those layouts expect — `@` and `"` swap on UK, `a` and `q` swap on French, Ctrl+A follows — and the fall-back to US was checked for a missing and a faulty file. It has **not** been tried on a computer actually set to a non-US layout, and the layout files themselves are the community project's work. If a symbol still comes out wrong, please report it.

Appendix J — More media, browser and app-launcher keys

This appendix has moved. The table of extra media, browser and app-launcher keys, how to add one to a group, and what to expect from each on Windows and other systems are now in the **Reference** (`AEROMORSE_REFERENCE.md`), in the section "More media, browser and app-launcher keys".

Appendix K — Solderless build: Cytron Maker Feather AIoT S3 with two switches

A complete two-switch AeroMorse with **no soldering at all**: a Cytron Maker Feather AIoT S3 bought with its headers already fitted, two 3.5 mm jack boards, and four push-on jumper wires. There is no screen and no pressure sensor; the beeps come from the board's own buzzer — or from a STEMMA Speaker (#3885) if you want them louder — and the display is the **AeroMorse Display** window on the computer (§5).

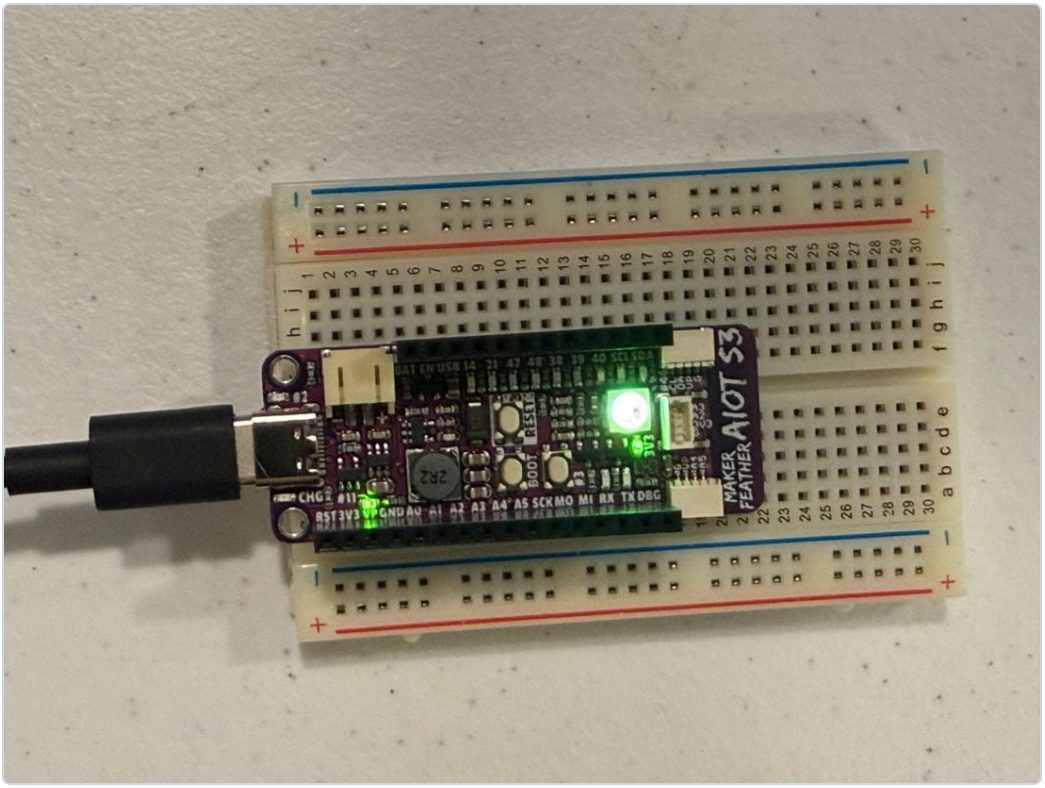
> **Buying in the USA?** The Adafruit Metro ESP32-S3 #5500 also comes with its headers fitted, and the same switches-and-speaker build works on it without the Cytron's shipping and customs costs. It has been built and tested with two switches and with the pressure sensor — see "A solderless switch build on the Metro ESP32-S3" in Appendix L.
>
> **Tested:** built and checked on 2026-10-10 with CircuitPython 10.3.1 and AeroMorse v1.28 plus the changes that became v1.29. Both switches type, and sound was checked both ways: the board's own buzzer, and a STEMMA Speaker #3885. The wireless display (§5) also works from this board. The pressure sensor (§K.8) has **not** been tried on it.

K.1 When this build is the right pick

Want	This build
No soldering, no tools	✅ every connection is a push fit
Two AT switches (dot and dash)	✅ one 3.5 mm jack each, ordinary mono switch plugs
Sound without buying a speaker	✅ buzzer is on the board
Louder sound, with a volume control	✅ optional STEMMA Speaker #3885 — three push-in wires
A screen on the device	❌ none — use the AeroMorse Display window on the computer
Sip-and-puff	❌ not tested on this board — use the recommended #5691 build

K.2 Parts

Qty	Item	Notes
1	**Cytron Maker Feather AIoT S3, with pre-soldered headers** — cytron.io	Order the pre-soldered version. Its headers are *stacking* headers: sockets on top, long pins underneath. Cytron ships from Malaysia: to the USA, allow for DHL shipping and a separate customs bill on top of the price shown — see "Cost to the USA" in Appendix L
1	Half-size breadboard (400 tie points)	Only a stand for the board. No wire goes into the breadboard in this build
2	3.5 mm stereo jack board with a 5-pin header (pins numbered 1–5). Sold as "3.5mm TRS Audio Jack PCB Adapter to 5Pin Header" or "3.5mm Stereo Audio Socket Adapter Boards (with pre-soldered headers)" — the ones used here: amazon.com/dp/B0G7GSQDRR	One per switch. Buy the kind with the header pins **already soldered on**. The board is marked "3.5mm Audio Jacks" and measures 25 × 16 mm
4	Male-to-female jumper wires	Two colours help: one colour for ground, one for signal
1	USB-C **data** cable	A charge-only cable will not work
2	AT switches with 3.5 mm mono plugs	
1	*Optional:* STEMMA Speaker #3885	Louder than the buzzer, with a volume screw
1	*Optional, with the speaker:* JST PH 3-pin to male header cable #3893	Its three pins push straight into the Cytron's sockets



The board on its breadboard. The labels along the lower header name the sockets used in §K.6.

K.3 What is different about this board

Feature	On this board	What it means for AeroMorse
Processor	ESP32-S3-WROOM-1-N8R8 module: 8 MB flash, 8 MB PSRAM	Same chip family as the recommended #5691, with more memory
Screen	None	`USE_DISPLAY = False`; keep `PC_DISPLAY = True`
Buzzer	Built in, on `board.BUZZER` (GPIO12), with a small mute slide switch on the **underside** of the board	`AUDIO_PIN = board.BUZZER`; no speaker to wire. A STEMMA Speaker on socket `A0` is the louder alternative
Ground sockets	**Only one** `GND` on the headers	The two jacks each borrow a spare socket as their ground (§K.5), which leaves `GND` free for a speaker
Sensor bus pull-up resistors	None on the board	Needs `code.py` **v1.29 or later**: an older one stops at start-up when nothing is plugged into the sensor port (§K.4)
Sensor / Maker port power	Switched by `board.VP_EN` (GPIO11)	Only matters if you add a sensor (§K.8)
Maker Ports	Three, compatible with Qwiic and STEMMA QT (and Grove with a conversion cable): two horizontal ones powered from VPeripheral, one vertical one powered from 3.3 V	Not used in this build. The two horizontal ports only have power while `board.VP_EN` is high (§K.8)
Bootloader drive name	`MFAS3BOOT`	Where the CircuitPython `.uf2` is copied

The dot and dash pins are named differently on the silkscreen than in `config.py`:

Socket label on the board	GPIO	Name in `config.py`
A0	10	`board.A0`
A1	9	`board.A1`
A2	6	`board.A2` or `board.D6`
A3	5	`board.A3` or `board.D5`
A4	4	`board.A4`

K.4 Software

1. **CircuitPython 10.3.1.** Download the `.uf2` from the board's page, circuitpython.org/board/cytron_maker_feather_aiot_s3. Double-press the board's **RESET** button; a drive

called `MFAS3BOOT` appears. Copy the `.uf2` onto it. The board restarts and a `CIRCUITPY` drive appears. (The board used here arrived with CircuitPython 8.0 from 2022, which is too old — the update is not optional.)

2. **Libraries.** From the **10.x** library bundle (§9.2), copy into `CIRCUITPY/lib`: `adafruit_hid`, `adafruit_display_text`, `adafruit_bitmap_font`, `adafruit_bus_device`, `adafruit_register`, `adafruit_lps35hw.mpy` and `neopixel.mpy`.
3. **AeroMorse files.** Copy `boot.py`, `code.py`, `morse_map.py` and `config.py` as in Step 9.3. **Step 4 — unplug the USB cable and plug it back in** so `boot.py` takes effect.

> **This board needs AeroMorse v1.29 or later.** An older `code.py` stops here at start-up with `RuntimeError: No pull up found on SDA or SCL`, because the board has no pull-up resistors on its sensor port.

K.5 `config.py` settings

Change these lines in `config.py` (everything else stays as it is):

```
DEVICE_NAME          = "AeroMorse Cytron"   # any name you like
USE_SENSOR           = False                # two switches, no pressure sensor
DOT_PIN              = board.D5             # socket A3
DASH_PIN             = board.A1             # socket A1
AUDIO_PIN            = board.BUZZER         # the buzzer on the board
USE_DISPLAY          = False                # no screen
USE_WIRELESS_DISPLAY = False                # unless you have a wireless display
PC_DISPLAY           = True                 # show the display on the computer
```

With a STEMMA Speaker use `AUDIO_PIN = board.A0` instead of `board.BUZZER`. The buzzer on the board then stays silent.

For no sound from the device at all set `USE_SPEAKER = False` (v1.29+).

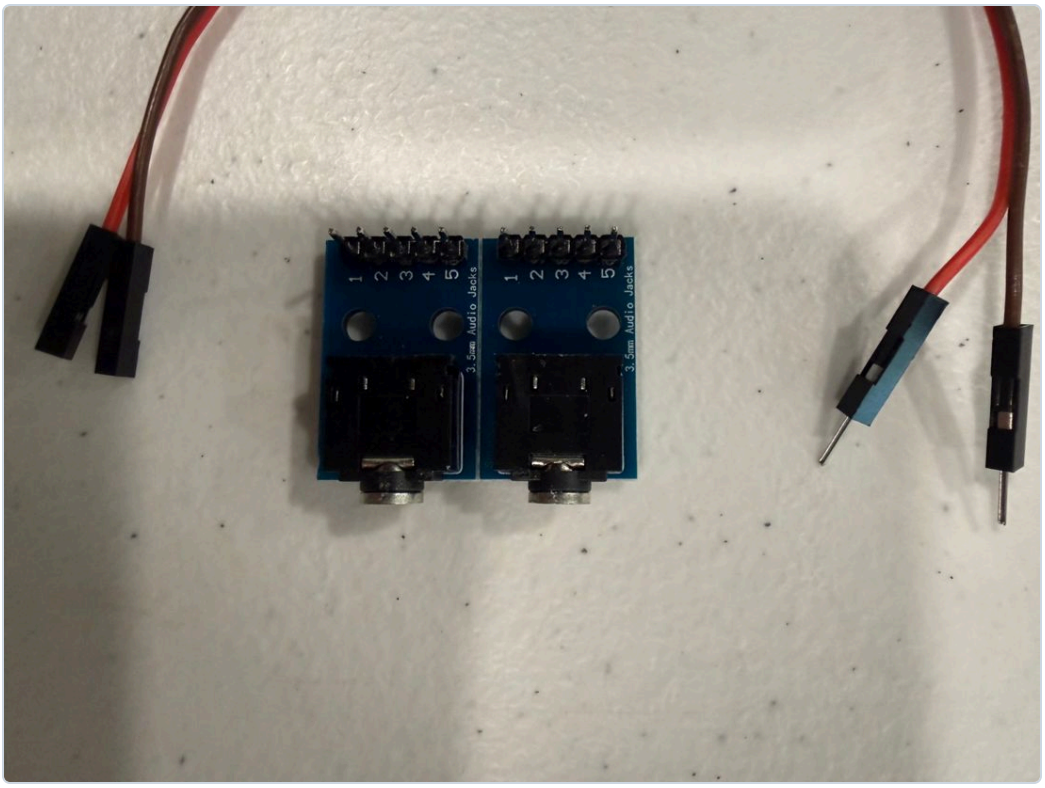
Then add these lines just below the `DASH_PIN` line. The board has only one `GND` socket and each socket takes one wire, so these lines hold sockets **A4** and **A2** at 0 V to serve as the grounds for the two jacks:

```
import digitalio as _dio
_gnds = []
for _p in (board.A4, board.A2):
    _g = _dio.DigitalInOut(_p)
    _g.switch_to_output(value=False)
    _gnds.append(_g)
```

A switch contact draws far less than a milliamp, so a pin is perfectly good as its ground. A **speaker is different** — it must use the real `GND` socket, which this arrangement leaves free for it.

K.6 Wiring

> ⚠️ **Unplug the USB cable before you touch the wires.** The `3V3` and `VP` sockets sit right beside `GND`; a wire pushed into one of them by mistake while the board is powered can damage it.



The two jack boards and one pair of male-to-female jumper wires.

On these jack boards **pin 1 is the sleeve (ground)** and **pin 5 is the tip (the switch signal)**. Pins 2, 3 and 4 stay empty.

Each wire's **socket end** pushes onto a jack-board pin; its **pin end** pushes down into a socket on top of the Cytron's **long** header — the one labelled `RST 3V3 VP GND A0 A1 A2 A3 A4 A5 …`.

Jack board	Jack pin	Wire	Cytron socket
A — **dot** switch	1	ground colour	`A2`
A — **dot** switch	5	signal colour	`A3`
B — **dash** switch	1	ground colour	`A4`
B — **dash** switch	5	signal colour	`A1`

Optional — STEMMA Speaker #3885. Click the #3893 cable's white plug into the speaker board, then push its three pins into the same header:

Speaker cable wire	Cytron socket
Black (ground)	`GND`
Red (power)	`3V3` — **not** `VP` next to it
White (signal)	`A0`

With everything fitted, the sockets read like this, counting from the USB end:

Socket	Wire
1st — `RST`	empty
2nd — `3V3`	speaker red
3rd — `VP`	empty
4th — `GND`	speaker black
5th — `A0`	speaker white
6th — `A1`	dash jack signal
7th — `A2`	dot jack ground
8th — `A3`	dot jack signal
9th — `A4`	dash jack ground

Without a speaker, `3V3`, `GND` and `A0` stay empty too.

Check before plugging the USB back in: nothing in `RST` or `VP`, and nothing in the short header on the other side of the board (`BAT EN USB 14 21 …`).

> Any free `A` socket works for a signal wire as long as `config.py` names the same one. If a wire ends up one socket off, it is often easier to change `DOT_PIN` / `DASH_PIN` than to move the wire.

The speaker's loudness is set with the small **Volume** screw on the speaker board — turn it gently with a small screwdriver.

K.7 First test and troubleshooting

Click into an empty Notepad window, then plug in the USB cable, then one switch at a time. A short press of the dot switch beeps and types `e`; a short press of the dash switch gives a lower beep and types `t`. Holding either for more than a second changes group (Appendix A).

What happens	Cause	Fix
Nothing at all, and the serial log (Appendix N) shows `No pull up found on SDA or SCL`	`code.py` is older than v1.29	Download the current files from the repo (Step 9.3)
One switch works, the other does nothing	That jack's signal wire is one socket off, or its ground wire is not in `A2` (dot) / `A4` (dash)	Count the sockets again from the USB end, or change `DOT_PIN` / `DASH_PIN` to the socket the wire is really in
Neither switch does anything	The ground lines are missing from `config.py`, so `A2` and `A4` are not acting as grounds	Add the lines from §K.5
A beep or a letter repeats non-stop the moment a switch is plugged in	The signal wire is on the jack's ring pin, which a mono plug connects to ground	Move that wire from pin 5 to pin 2 (or the other way round)
Typing works but there is no sound (buzzer)	The buzzer's mute slide switch is off. It is on the **underside** of the board, hidden while the board sits in a breadboard	Unplug the USB cable, lift the board out, slide the switch the other way, and press the board back into the same holes. Sound through the computer also works: `PC_SOUND = True` and the AeroMorse Display program (§5)
Typing works but there is no sound (STEMMA Speaker)	`AUDIO_PIN` is still `board.BUZZER`, the white wire is not in `A0`, the red wire is not in `3V3`, or the Volume screw is turned right down	Check each in turn
The AeroMorse Display window does not find this board	An `AeroMorse Display.exe` from before v1.29 only looks for Adafruit boards	Download the current one from the repo. Or start the old one with the port named: `"AeroMorse Display.exe" --port COM16` (use the board's own COM number, from Device Manager)
The AeroMorse Display window shows your other AeroMorse instead	It remembers the device it showed last	Right-click the window → **Show device** → pick it by name
The computer shows the old board name	Windows remembers it	Remove the device in Bluetooth & devices and replug

K.8 Adding the pressure sensor later (not tested)

The LPS33HW sensor (#4414) should plug into the Cytron's I2C Maker port with a STEMMA QT cable, but this has not been tried. Two changes to `config.py` are known to be needed:

1. Change the existing line `USE_SENSOR = False` to `USE_SENSOR = True`.

2. Switch the port's power on **before** the sensor starts. Add these three lines **directly below the ground lines from §K.5** — the block that ends with `_gnds.append(_g)`, a little below the `DASH_PIN` line:

```
import digitalio as _dio2
_vp = _dio2.DigitalInOut(board.VP_EN)
_vp.switch_to_output(value=True)
```

Start each of the three lines at the left edge, with no spaces in front. If your `config.py` has no ground lines (you are not using switch jacks), put them just below the `DASH_PIN` line instead. Anywhere below the `import board` line near the top of the file works; this spot keeps the board-specific lines together.

If you build this, please report how it goes.

AeroMorse is open-source. Project files at: https://github.com/jlubin2001/AeroMorse

---

Appendix L — Other boards

The standard build uses the Adafruit ESP32-S3 Reverse TFT Feather, #5691. The Cytron Maker Feather AIoT S3 is the other board the project has built on (Appendix K). This appendix lists the rest: boards that should work, boards that cannot, and why — including a no-soldering switch build on the Adafruit Metro ESP32-S3.

> ⚠️ **Not tested by the project.** Unless a part of this appendix says otherwise, nobody on the project has built what it describes. It is written from the parts' documentation. The build the project uses and tests every day is the standard build in section 2. If you build one of these, please report how it went.

Why not every Feather works

AeroMorse uses **USB HID** — the USB standard that lets keyboards and mice talk to computers. Only Feather boards with **native USB** support this. Classic ESP32 boards (including Adafruit #5900 Feather ESP32 V2) use a separate USB-to- serial chip that cannot emulate a keyboard. Those boards will not work.

Bluetooth — not supported

AeroMorse sends **USB HID only**; it does not work as a Bluetooth keyboard. The chip can do Bluetooth, but a Bluetooth keyboard on these boards was tested in September 2026 (CircuitPython 10.3.1) and did not stay connected. For a phone or tablet, use a **USB-C cable** — AeroMorse works as a wired keyboard and mouse. The test record is in BLE_STATUS.md.

Boards that work

Board	Adafruit #	USB HID	PSRAM	WiFi	Notes
ESP32-S3 Reverse TFT Feather	#5691	✅	✅ 2MB	✅	Built-in 1.14" TFT — screen faces down (panel mount). **Recommended for panel-mount enclosures — see below.** Not great on a breadboard once headers are soldered (TFT ends up pressed against the breadboard).
ESP32-S3 TFT Feather	#5483	✅	✅ 2MB	✅	Built-in 1.14" TFT — screen faces up. **Best built-in-TFT option if you plan to use a breadboard with headers.**
ESP32-S3 Feather 4MB/2MB PSRAM	#5477	✅	✅ 2MB	✅	No built-in display — pair with a STEMMA QT OLED (#326 / #938) or a FeatherWing TFT (Appendix F), or use the AeroMorse Display window on the computer. **Tested** with the pressure sensor (see below).
ESP32-S2 TFT Feather	#5300	✅	✅ 2MB	✅	Built-in 1.14" TFT — older S2 chip

A non-Adafruit board that works — and needs no soldering

Board	Where to buy	USB HID	PSRAM	WiFi	Notes
Cytron Maker Feather AIoT S3, with pre-soldered headers	Cytron	✅	✅ **8MB**	✅	No built-in display. Buzzer built in. **Sold with its headers already fitted**, so a switch build needs no soldering — every wire pushes in. Tested as a **two-switch** device; not tested with the pressure sensor. Full build in Appendix K.

What you get, and what to know before ordering:

- **Processor and memory:** an ESP32-S3-WROOM-1-N8R8 module — 8 MB flash and 8 MB PSRAM, more than the 4 MB / 2 MB of the Adafruit Feathers above.
- **Headers:** Cytron sells it two ways. Order the version **with pre-soldered headers**; the "Basic" version has none. The headers are sockets on top with long pins underneath, so the board stands in a breadboard and jumper wires push into the sockets.
- **Sound:** a piezo buzzer on the board (GPIO12), with a mute switch on the **underside** of the board. A STEMMA Speaker #3885 can be added for louder sound.
- **Maker Ports:** three small sockets compatible with Qwiic and STEMMA QT (and Grove with a conversion cable). Two lie flat and are powered from "VPeripheral", which the software can switch on and off; one stands upright and always has 3.3 V.
- **Wireless display:** works — the ESP-NOW broadcast (§5) was tested from this board.
- **CircuitPython:** 10.3.1, from circuitpython.org/board/cytron_maker_feather_aiot_s3.
- **Cost to the USA:** Cytron ships from Malaysia by DHL. The checkout shows the shipping charge, and asks you to accept that any import duty is yours to pay — but it does not say how much that will be. The

duty, with DHL's own fee for handling it, is billed separately by DHL after the parcel ships. On the author's order (September 2026, $33 of parts) shipping was about as much as the parts, and the DHL customs bill was about as much again — roughly three times the price of the parts in all. Allow for that when comparing prices.

- **Settings:** it needs a few board-specific lines in `config.py` — all in Appendix K.

Metro form factor boards

Metro boards use the Arduino Uno footprint (68×53 mm) — larger than a Feather. They work for AeroMorse but with one key difference: **FeatherWing displays will not plug directly onto them.** Use a standalone TFT breakout (SPI wires) or the STEMMA QT OLED instead. The pressure sensor plugs straight in via STEMMA QT with no wiring change.

Board	Adafruit #	USB HID	PSRAM	WiFi	Notes
Metro ESP32-S3	#5500	✅	✅ **8MB**	✅	**Most memory** (16 MB flash + 8 MB PSRAM); built-in LiPoly charging + battery monitor. **Comes with its female headers already soldered**, so a switch-and-speaker build needs no soldering — see below. **Tested** with two switches and with the pressure sensor

> The Metro ESP32-S3 #5500 has the most memory in this guide (16 MB flash, 8 MB PSRAM). Choose the Metro for the extra memory or the larger form factor.

A solderless switch build on the Metro ESP32-S3

The same no-soldering idea as the Cytron build in Appendix K, on an Adafruit board sold and shipped from the USA: two switches — AT switches or push buttons — and an optional speaker, every wire pushed into the Metro's own sockets or clamped under a screw. No screen and no pressure sensor; the display is the AeroMorse Display window on the computer (§5).

> **Tested:** built and checked on 2026-11-09 with CircuitPython 10.3.1 and AeroMorse v1.33 — two AT switches through 3.5 mm jacks on `D5` and `D6`, the STEMMA Speaker on `A0`, and then the pressure sensor instead of the switches (see "Using the pressure sensor on the Metro" below). The push-button option is written from the Metro's documentation and the tested Cytron build.
>
> **A step-by-step guide for this build** — with photos, a wiring drawing, a wiring checklist and the tube joiner — is in AeroMorse_Metro_no_breadboard_guide.pdf. It is written for someone who has never built electronics before.

The exact parts of the tested build — no breadboard; every wire goes straight from a jack board or the speaker into the Metro:

Qty	Part	Link	Notes
1	Metro ESP32-S3	#5500	Sockets fitted at the factory
1	LPS33HW pressure sensor	#4414	For sip-and-puff
1	STEMMA QT cable, 50 mm	#4399	Sensor to the Metro's `QT I2C` socket
1	STEMMA Speaker	#3885	Volume screw on the board
1	Speaker cable, JST PH 3-pin to male pins	#3893	Sold separately from the speaker
2	3.5 mm jack board with **bent** pins (5-pack; choose "With bent needle")	Amazon B0FG24PBJJ	Pin 1 = sleeve (ground), pin 5 = tip — confirmed on the tested boards
4	Female-to-male "extension" jumper wires	—	Socket end onto a jack-board pin, pin end into the Metro; about 20 cm (8 in) reaches
1	USB-C **data** cable	—	A charge-only cable will not work

Wiring of the tested build: jack A pin 5 → `D5` (dot), jack A pin 1 → `GND`; jack B pin 5 → `D6` (dash), jack B pin 1 → `GND`; speaker white → `A0`, red → `3.3`, black → `GND`; sensor → `QT I2C`. One `GND` socket each, and nothing in `VHI` or `VIN`. The speaker runs fine from the `3.3` socket.

Plug-in test switches (tested, optional). Two 30 mm arcade buttons, each wired to a 3.5 mm **mono** plug adapter with screw terminals (marked `+` and `–`), make two test switches that plug into the jacks like a real AT switch. A caregiver can use them to check the box works, or to tell whether a fault is in the box or in the user's own switch. On each button, the cable's two spades go on the click switch's `C` (common) and `NO` (normally open) tabs; cut off the small white connector at the other end, strip about 5 mm (¼ in) of each wire, and clamp one under each terminal screw — either way round. Check with a multimeter: tip to sleeve should beep only while the button is pressed.

How it differs from the Cytron build

	Cytron (Appendix K)	Metro ESP32-S3 #5500
Headers	Stacking headers, ordered pre-soldered	Female sockets, **soldered at the factory**
Ground sockets	Only one — two spare pins stand in as grounds	**Three** (two on the power header, one beside `D13`) — no extra lines in `config.py`
Sound	Buzzer on the board, or a STEMMA Speaker	**No buzzer** — add a STEMMA Speaker (recommended), or use sound through the computer (`PC_SOUND`, §6.2)
Pins for the switches	`A3` and `A1`	`D5` and `D6` — the `config.py` defaults
Size	Feather (51 × 23 mm)	Arduino Uno (68 × 53 mm)
Where to buy	Cytron, Malaysia (shipping and customs on top)	Adafruit, USA

Parts — every Metro build

Qty	Item	Notes
1	Adafruit Metro ESP32-S3 #5500	Headers already fitted
1	USB-C **data** cable	A charge-only cable will not work
1	*Optional:* mounting plate #275	Holds the Metro (and a half breadboard) on rubber feet

Parts — the two switches: pick one option

Option	Parts	Good for
M1 — AT switches, jack boards with pins	2 × 3.5 mm stereo jack board with a 5-pin header, pins already soldered on (the same boards as §K.2, or the bent-pin boards of the tested build above); 4 × **male-to-female** jumper wires; 2 AT switches with 3.5 mm mono plugs	A user's own AT switches. Everything pushes in, no tools. **Tested**
M2 — AT switches, screw-terminal jacks	2 × TRRS jack terminal block #2915; **male-to-male** jumper wires #1957 (4 needed); a small flat screwdriver; 2 AT switches with 3.5 mm mono plugs	A user's own AT switches, all parts from Adafruit. The #2915 has only screw terminals, so each wire needs a pin at **both** ends: one clamped under a screw, one pushed into the Metro
M3 — push buttons	STEMMA wired push-button pack #4431 (five buttons, each with its own cable; two are used)	Trying AeroMorse, or someone who presses buttons by hand. No jacks and no extra wires: each button's cable pushes straight into the Metro

Parts — sound: pick one option

(A piezo on a plug — Option S2 in Appendix D — is left out here on purpose: on this board the speaker is louder, has a volume control, costs about the same, unplugs at its own cable, and needs no jack, wires or screwdriver.)

Option	Parts	Notes
STEMMA Speaker — recommended	Speaker #3885 + JST PH 3-pin to male header cable #3893 (**sold separately** — the speaker does not include it)	A real speaker with its own small amplifier, so AeroMorse's beeps come through clearly, and a **volume screw**. Its three pins push straight into the Metro — no jack, no tools. Tested on the Metro and on the Cytron build
None on the device	—	Beeps through the computer instead: `PC_SOUND = True` and the AeroMorse Display window (§6.2)

The Metro's three `GND` sockets are exactly enough: one for each switch and one for the sound.

Software

1. **CircuitPython 10.3.1** from circuitpython.org/board/adafruit_metro_esp32s3. Double-press the Metro's **RESET** button; a drive whose name ends in `BOOT` appears. Copy the `.uf2` onto it. The board restarts and a `CIRCUITPY` drive appears. The Metro has 16 MB of flash, so the bootloader change that 4 MB boards need for CircuitPython 10 does not apply to it.
2. **Libraries and AeroMorse files** — the same as §K.4 steps 2 and 3. Then unplug the USB cable and plug it back in so `boot.py` takes effect.

`config.py` **settings** — the pins are already the defaults, so only these change, whichever options you chose:

```
DEVICE_NAME          = "AeroMorse Metro"    # any name you like
USE_SENSOR           = False                # two switches, no pressure sensor
USE_DISPLAY          = False                # no screen
USE_WIRELESS_DISPLAY = False                # unless you have a wireless display
PC_DISPLAY           = True                 # show the display on the computer
```

`DOT_PIN = board.D5`, `DASH_PIN = board.D6` and `AUDIO_PIN = board.A0` stay as they are. With no sound on the device, set `PC_SOUND = True` for beeps through the computer, or `USE_SPEAKER = False` for no sound at all.

Wiring — ⚠️ **unplug the USB cable first.** Whatever the option, the dot switch's signal goes to `D5`, the dash switch's to `D6`, the sound's signal to `A0`, and each part's ground to its own `GND` socket.

Option M1 — jack boards with pins. On these boards **pin 1 is the sleeve (ground)** and **pin 5 is the tip (signal)**; pins 2, 3 and 4 stay empty. Each wire's socket end pushes onto a jack-board pin, its pin end into the Metro:

Jack board	Jack pin	Metro socket
A — **dot** switch	1	`GND`
A — **dot** switch	5	`D5`
B — **dash** switch	1	`GND`
B — **dash** switch	5	`D6`

Option M2 — #2915 screw-terminal jacks. The terminals are printed `GND`, (blank), `Right`, `Left`. **`Left` is the tip** — the switch signal — and `GND` is the sleeve. Loosen the screw, push one end of a male-to-male wire in, and tighten it until the wire does not pull out:

#2915 jack	Terminal	Metro socket
A — **dot** switch	`Left`	`D5`
A — **dot** switch	`GND`	`GND`
B — **dash** switch	`Left`	`D6`
B — **dash** switch	`GND`	`GND`

`Right` and the blank terminal stay empty.

Option M3 — STEMMA push buttons. Each button's cable ends in three pins:

Button wire	Dot button	Dash button
White (signal)	`D5`	`D6`
Black (ground)	`GND`	`GND`
Red (power)	**not connected** — tape it over	**not connected** — tape it over

The red wire is not needed: AeroMorse turns on the Metro's own pull-up for the switch pins, and a press connects the white wire to ground. Leaving it out also keeps the Metro's only `3.3` socket free for a speaker. **Never put a red wire in `VHI` or `VIN`** — those carry 5 V, more than the Metro's pins can take.

STEMMA Speaker (recommended): click the #3893 cable's white plug into the speaker board, then push its three pins into the Metro:

Speaker cable wire	Metro socket
Black (ground)	the remaining `GND`
Red (power)	`3.3` (3.3 V) — not `VHI` or `VIN`
White (signal)	`A0`

Set the loudness with the small **Volume** screw on the speaker board — turn it gently with a small screwdriver.

Then test as in §K.7: a short press of the dot switch types `e`, of the dash switch `t`.

Using the pressure sensor on the Metro (tested)

The Metro also works as a sip-and-puff device, still with no soldering. It has a **STEMMA QT** port, the same small 4-pin plug as the LPS33HW pressure sensor board.

1. Unplug the USB cable.
2. Connect the sensor to the Metro's STEMMA QT port with a STEMMA QT / Qwiic cable that has the small plug **on both ends**. Short is best: the tested build used the 50 mm #4399; up to about 20 cm (8 in) is

fine. The plug only fits one way.
3. In `config.py`, set `USE_SENSOR = True`. The switch pins are then not used.
4. Plug the USB cable back in.

Everything else — sound, the AeroMorse Display window — stays as above.

Boards that do NOT work for USB HID

Board	Reason
Feather ESP32 V2 #5400	No native USB — uses CP2102N serial chip
Feather ESP32 Huzzah #3405	No native USB
Feather ESP8266 #2821	No native USB

Recommended board: Adafruit ESP32-S3 Reverse TFT Feather #5691

Why this one:

- **Display already built in** — the 240×135 colour TFT is soldered to the board, facing down for panel mounting; no separate display to buy or wire
- Has **PSRAM** (2 MB extra RAM) — handles all display and sensor work without running out of memory
- Has a **STEMMA QT** port — the LPS33HW pressure sensor plugs straight in with no soldering
- **WiFi + ESP-NOW** built in — supports the wireless remote display option (section 5.3) without any extra hardware on the main board
- USB HID works out of the box — appears as keyboard and mouse to any computer
- CircuitPython support is excellent and actively maintained
- Available from Adafruit: https://www.adafruit.com/product/5691

> **Watch out — #5691 is not breadboard-friendly once you solder headers.** The TFT is on the back of the board, so once header pins are soldered and the Feather is pressed into a breadboard, the screen ends up facing the breadboard and is unreadable. #5691 is designed for **panel-mount enclosures** (a cutout in a case with the screen showing through), not for prototyping on a solderless breadboard. If you plan to develop on a breadboard before moving to an enclosure, use **#5483** (built-in TFT facing up — screen still visible when the Feather is on a breadboard) or **#5477** with an external display.

If you want a built-in TFT and plan to use a breadboard (or no enclosure):

The **ESP32-S3 TFT Feather #5483** is electrically identical to #5691, but its TFT faces the same side as the components, so the screen still faces up when the board stands in a breadboard. **A breadboard needs header pins on the Feather** — they come loose in the bag and have to be soldered on first (see "Building with no header pins" below). Without a breadboard the #5483 needs no pins at all, exactly like the #5691.

Everything else — PSRAM, STEMMA QT, ESP-NOW, USB HID — is the same.
https://www.adafruit.com/product/5483

If you want a bare Feather (no built-in display) for the cleanest breadboard or for maximum display flexibility:

The **ESP32-S3 Feather #5477** is the same chip without a soldered-on TFT, so — **once header pins are soldered on** — it sits flat in a breadboard. Like most Feathers it arrives as a bare board with the pin strips loose in the bag. Pair it with one of:

- **STEMMA QT OLED #326 (0.96") or #938 (1.3")** — plug-and-play via the STEMMA QT chain (the same chain that carries the sensor). Needs a `display_setup.py` file on the drive (v1.26+, example provided — see Appendix F). Monochrome 128×64, small text. Untested.
- **FeatherWing TFT (#3651, #5872, or #3315)** — no wiring, but **the Feather must have header pins soldered on**: the wing has sockets that the Feather's pins push into, and a bare Feather has nothing to push in. Also needs a `display_setup.py` file (Appendix F). Larger, colour, much more screen area than the OLED; the text grows to fit by itself. For the full **#5477 + #3651** combo see **Appendix F** at the end of this guide. Untested.
- **No screen on the device at all** — the display is the AeroMorse Display window on the computer (§5). Nothing to solder: the sensor plugs into the Feather's STEMMA QT port with a cable that has the small plug on both ends.

> **Tested:** a #5477 with the pressure sensor on its STEMMA QT port and no screen on the device (the AeroMorse Display window instead) was built and checked on 2026-11-09 with CircuitPython 10.3.1 and AeroMorse v1.33. In `config.py` set `USE_DISPLAY = False` (no screen on the board) and `PC_DISPLAY = True`, and give it a `DEVICE_NAME` of your choice.
>
> **If CircuitPython 10 will not install:** the test board arrived with an older bootloader (TinyUF2 0.10.2). Copying the CircuitPython 10 `.uf2` onto its `FTHRS3BOOT` drive did nothing — the drive simply came back. The fix is to install the newer bootloader first: TinyUF2 0.35.0 for `adafruit_feather_esp32s3`, from github.com/adafruit/tinyuf2/releases, written with esptool or Adafruit's WebSerial ESPTool (`combined.bin` at address 0). This erases the board, so do it before copying any AeroMorse files. Then copy the CircuitPython 10 `.uf2` as usual.

https://www.adafruit.com/product/5477

If you want the most memory (Metro form factor):

The **Metro ESP32-S3 #5500** has the most memory in this guide (8 MB PSRAM, 16 MB flash). Like the Feathers above, it supports USB HID.

The #5500 has no built-in display — connect the STEMMA QT OLED or a standalone TFT breakout. The ESP-NOW wireless display (section 5.3) works on it identically to the Feather boards.
https://www.adafruit.com/product/5500

A note on PSRAM:

PSRAM is extra RAM soldered alongside the main chip. The ESP32-S3 chip itself has 512 KB of built-in RAM; a colour TFT display needs a *framebuffer* (a block of RAM holding every pixel) that can easily exceed that. Every Feather and Metro board still listed above has at least **2 MB of PSRAM** (the #5500 Metro has 8 MB), so the framebuffer lives in PSRAM and the built-in 512 KB stays free for code. Any supported display size — 128×64 OLED through 480×320 FeatherWing — fits comfortably.

Quick decision guide:

I want…	Choose
Simplest build, colour TFT included, USB only	#5691 Reverse TFT Feather
Same but screen faces up	#5483 TFT Feather
To use it with a phone or tablet	Any board here, over a **USB-C cable** — Bluetooth is not supported (see above)
Most memory (8 MB PSRAM)	#5500 Metro ESP32-S3

Boards with a screen built in

Board	#	Display size	Resolution	Screen orientation	URL
ESP32-S3 TFT Feather	5483	1.14"	240×135 colour	Faces up — same side as components	https://www.adafruit.com/product/5483
ESP32-S3 Reverse TFT Feather	5691	1.14"	240×135 colour	Faces down — opposite side to components	https://www.adafruit.com/product/5691
ESP32-S2 TFT Feather	5300	1.14"	240×135 colour	Faces up — same side as components	https://www.adafruit.com/product/5300

All three have the display soldered onto the board and accessed via `board.DISPLAY` — no wiring, no extra purchase, no library to add.

Which orientation to choose:

- **#5483 (standard)** — screen faces the same direction as the USB port and pins. Best for handheld use or when you want to see the display while working with the board.
- **#5691 (reverse)** — screen faces the opposite side. Designed for mounting behind a panel with a cutout so only the screen shows through. Useful if you want to build AeroMorse into an enclosure.
- **#5300 (ESP32-S2)** — same orientation as #5483 but older chip. Only choose it if #5483 and #5691 are both unavailable.

All three are functionally identical for AeroMorse.

> **Code file:** #5483, #5691, and #5300 all use the **repo root** `code.py` (which drives the built-in TFT via `board.DISPLAY`).

The pressure sensor on a board with no STEMMA QT socket

If your board has no STEMMA QT port (e.g. the Feather nRF52840 Express), wire the sensor to the board's I²C pins instead. Which cable depends on the Feather:

- **Feather with header pins, not in a breadboard:** a STEMMA QT cable with **female sockets** (#4397) — the sockets push onto the pins.
- **Feather standing in a breadboard:** a STEMMA QT cable with **male pins** (#4209) — the pins push into the breadboard rows beside the Feather.
- **Bare Feather, no header pins:** cut one end off a STEMMA QT cable and solder its four wires to the pads.

LPS33HW (STEMMA QT wire)	Feather pin
Red — power	**3V**
Black — ground	**GND**
Blue — SDA (data)	**SDA**
Yellow — SCL (clock)	**SCL**

No config change is needed. From firmware **v1.4**, `code.py` uses `board.STEMMA_I2C()` when the board has a STEMMA QT port and otherwise falls back to `board.I2C()` on the SDA/SCL pins — the serial console prints a note when it does. (Earlier firmware crashed at startup on such boards.)

Appendix M — Header pins

Feather boards arrive bare, with two strips of pins loose in the bag. The standard build and the three choices in the main guide need none of them soldered on. They matter for some of the hardware in Appendices D, F and L, and this appendix says which.

Header pins. Most Feathers arrive as a bare board with two male pin strips loose in the bag — nothing is soldered. Whether you need them soldered on depends on what you choose below:

You choose	Header pins on the Feather
#5691 + sip-and-puff sensor (STEMMA QT cable)	**Not needed** — nothing is soldered at all
AT switches on the #2915 terminal block, speaker S1 / S2	Not needed, but then 2–3 wires are soldered to the Feather's pads. With pins, jumper wires push on instead
A FeatherWing display	**Required**
A breadboard (switch Option B1)	**Required**

Building with no header pins — what works

Many Feather boards ship as bare PCBs with no header pins soldered. If you don't want to solder headers (and your supplier didn't pre-install them), some build paths still work via STEMMA QT plug-and-play, some require soldering a few wires directly to the Feather's PCB pads, and some fundamentally need headers.

Component / option	Works without headers?	What's needed
#5691 built-in TFT display	✅ Plug-and-play	Display soldered to board — nothing to wire
STEMMA QT OLED (#326, #938)	✅ Plug-and-play	STEMMA QT cable, no Feather wiring
STEMMA QT sensor (#4414 LPS33HW)	✅ Plug-and-play	STEMMA QT cable
FeatherWing displays (#3651, #5872, #3315)	❌ Not possible	The wing's holes can't grip bare pads — headers required
Standalone TFT breakouts (#2050, #1770…)	⚠️ Solder 5–7 wires	Solder display wires directly to Feather pads
AT switches — Option B1 breadboard	❌ Not possible	Feather must sit *in* the breadboard via headers
AT switches — Option B2 TRRS breakout	⚠️ Solder 3 wires	Solder breakout output wires to D5/D6/GND pads
AT switches — Option B3 #2915 Terminal Block	⚠️ Solder 3 wires	Solder terminal-block wires to D5/D6/GND pads
Speaker — #3885 STEMMA Speaker	⚠️ Solder 3 wires	Cut JST PH plug off, solder to A0/3V/GND pads
Speaker — Option S2 piezo (#2915 + #2790 + #1740/#1739)	⚠️ Solder 2 wires only if no headers	If Feather has headers: 2 F-F jumpers from #2915 → A0/GND (zero soldering). If no headers: solder 2 wires from #2915 → A0/GND pads. Piezo side is screw-terminal either way.
Speaker — PAM8302 amp (#2130)	⚠️ Solder 3 wires	Solder amp board to A0/3V/GND pads

STEMMA QT does not solve every problem. STEMMA QT carries I²C, 3 V, and GND only — that is enough for sensors and I²C displays (OLEDs), but **not for audio** (analog signal must come from A0) or **arbitrary switches** (digital inputs need GPIO pins).

> An I²C GPIO expander board (AW9523 #4886, MCP23017 #5346, PCF8575 #5611) *could* host AT switches via STEMMA QT, but you still need to wire jacks to the expander's GPIO pads — and `code.py` would need a new library and rewritten input routines. **More work, no benefit** vs. soldering three wires directly to D5/D6/GND on the Feather.

Recommended fully-header-less build:

- Feather: **#5691** (built-in TFT — no wiring)
- Sensor: **#4414 LPS33HW** via STEMMA QT (or skip if using AT switches)

- AT switches: **#2915 Terminal Block** + 3 wires soldered to D5/D6/GND pads
- Speaker: 3 wires soldered to A0/3V/GND pads (or skip — display alone is fine)

Total soldering for the full build: **3–6 wires to Feather pads** (skip the sensor row if you use switches, skip both if you don't want a speaker). Each pad is ~1.5 mm — much easier to solder than the pin holes for headers.

> **No soldering at all?** Skip the speaker and use the **#5691 + sensor only** combination. The OLED/TFT shows every dot and dash visually. Cost: a #5691 + #4414 + STEMMA QT cable, about $38 (electronics only — no enclosure, see Appendix H), zero solder joints. Add a third-party 3.5 mm Y-splitter and your existing AT switches won't work in this config — sensor mode is the no-solder path.

Getting header pins soldered on

Do you need header pins on the Feather? Only for some builds. The recommended build — #5691 with the sip-and-puff sensor on a STEMMA QT cable — needs **no header pins and no soldering**. Header pins are *required* for a FeatherWing display or a breadboard, and they turn the 2–3 soldered wires of the switch jack and the speaker into push-on jumper wires. If your build needs them and your board came bare (most do), the pins must be soldered on before you start; many local makerspaces or electronics shops will do this for free if you ask.

If your Feather already has header pins soldered: No special tools needed — the pins press straight into the breadboard.

If your Feather has no header pins (bare holes): You have two options:

Option	What to do	Cost
Get headers soldered	Take the Feather and the included loose header strip to a local makerspace, library maker lab, or electronics repair shop — most will solder headers for free or a few dollars. This is the most reliable long-term solution.	Free–$5
Order with headers pre-installed	Some resellers sell Feather boards with the headers already soldered — look for "headers soldered" in the listing. Adafruit's own pages for the #5691 and #5477 do not offer this.	Varies

> **Why not test hook clips?** Test hook clips (EZ-hook / Goupchn style) need something cylindrical to hook around — a wire lead, header pin, or IC leg. They cannot reliably grip a flat through-hole pad: there is nothing protruding for the hook to bite, and the hook is too large to engage inside the 0.7–1.0 mm through-hole. They will slide off with the lightest tug. Use one of the two options above instead.

Appendix N — Seeing the device's error messages (Thonny)

You do not need this appendix for a normal install. Come here when something does not work and Troubleshooting (section 12) asks what the device says at start-up.

Before copying the AeroMorse project files in Step 9.3, install **Thonny** — a free, beginner-friendly code editor that doubles as a serial console. The serial console is how the Feather reports errors when it tries to load `code.py`, the libraries, or `morse_map.py`. **Without watching the console, a problem in Step 9.3 looks like "nothing happens" instead of a specific error message you can act on.**

> ⚠️ **Connecting Thonny STOPS AeroMorse.** When Thonny connects to the Feather it interrupts whatever program is running, so `code.py` stops and the device **no longer types or moves the mouse** until it is restarted. During a first build that is harmless — you are using an ordinary keyboard and mouse. But **if AeroMorse is how you operate this computer, connecting Thonny to it locks you out**: the `>>>` prompt in the Shell means AeroMorse is *stopped*, not ready.
>
> **To get it going again:** have a helper unplug the Feather's USB cable and plug it back in (or click in Thonny's Shell and press Ctrl+D, which needs a working keyboard or mouse). Nothing is lost or damaged.
>
> **If AeroMorse is your only access:**
>
> - Do not connect Thonny to it unless a helper, or a second keyboard / mouse you can use, is at hand.
> - You do not need Thonny to change settings. `config.py` and `morse_map.py` can be edited straight on the CIRCUITPY drive with any text editor (Notepad, Notepad++, ...); saving restarts the device by itself.
> - To only *watch* what the device prints, **untick "Interrupt working program on connect"** in the same window (*Run → Configure interpreter...*, or *Tools → Options → Interpreter*) **before** you choose the port and click OK. Thonny then just listens: the Shell shows each code as you enter it, and AeroMorse keeps typing (confirmed on a device in daily use). Thonny remembers the setting. Even then, the Run button, Ctrl+C in the Shell, and *File → Open → CircuitPython device* all stop AeroMorse, and the Stop / Restart button interrupts it (see below).
> - When the device restarts (`devicereset`, or a replug) Thonny's Shell prints **"Connection lost — Use Stop/Restart to reconnect"** and then shows nothing more. That message is only about Thonny: AeroMorse itself has restarted and is working normally. To make Thonny listen again, click **Stop / Restart**: on our test device this restarted the program on the Feather (Shell shows `soft reboot`, then the start-up lines — a few seconds of calibration, so do not sip or puff) and AeroMorse carried on working. Because that button does interrupt the program first, have a helper nearby the first time you try it on a device you depend on; if the Shell ends at `>>>` instead, AeroMorse is stopped and needs a replug.
> - The same applies to any other serial program (PuTTY, `screen`, Mu): sending Ctrl+C to the device stops it.

N.1 Download and install Thonny

1. Go to **https://thonny.org** and click the download button for your operating system (Windows, macOS, or Linux).
2. Run the installer. No special options are needed — accept defaults.
3. Launch Thonny after installation completes.

> Mu Editor was previously recommended by Adafruit but has announced end-of-life in 2026. Thonny is the current recommended alternative.

N.2 Confirm the Feather is plugged in

The Feather should still be connected from §9.1 / §9.2. The **CIRCUITPY** drive must be visible in File Explorer (Windows) or Finder (Mac). If it isn't, unplug and replug the USB-C cable.

N.3 Point Thonny at CircuitPython

1. In Thonny, click **Run** → **Configure interpreter...** (or click the interpreter name shown at the bottom-right of the Thonny window).
2. In the top dropdown, select **CircuitPython (generic)**.
3. In the **Port** dropdown, select the port your Feather is on:
   - **Windows:** a COM port, e.g. `COM3` or `COM7` — try each if unsure
   - **macOS:** something like `/dev/cu.usbmodem14101`
   - **Linux:** something like `/dev/ttyACM0`
4. Click **OK**. **This stops any program running on the Feather** — on a device that already has AeroMorse on it, typing and the mouse stop working at this moment (see the warning at the top of this appendix).
5. The bottom **Shell** panel should now show `>>>` — this is the REPL prompt, confirming Thonny is talking to the Feather. While `>>>` is showing, AeroMorse is not running; press Ctrl+D in the Shell (N.5) or replug the USB cable to start it again.

> **Can't find the right port?** Windows: open Device Manager → Ports (COM & LPT) — the Feather appears as "USB Serial Device" or "CircuitPython CDC Control". Mac: open Terminal and type `ls /dev/cu.*` — look for `usbmodem`.

N.4 Leave the Shell panel visible

The Shell panel at the bottom of the Thonny window will display everything `code.py` prints (errors, calibration messages, debug output). Keep this panel visible while you complete Step 9.3. If a file copy fails, or a library is missing, the Shell will print an exact error message naming the file and line number — much easier to act on than guessing.

A healthy startup after Step 9.3 looks like this in the Shell:

```
Calibrating — do not sip or puff ...
Baseline: 1013.241  sip<1008.241  puff>1018.241
Calibration complete — ready for input.
```

If you see a red `Traceback (most recent call last):` line instead, the last line of the traceback names the problem (e.g. `ImportError: no module named 'adafruit_lps35hw'` — means that library file is missing from `/lib`; re-do §9.2). Note: in some Thonny versions tracebacks render in plain text rather than red.

N.5 Capturing a fresh boot trace (Ctrl+D)

The Shell panel only shows messages that fire *while Thonny is listening*. The first time you connect, the device has usually already booted — the `Calibrating…` and `Baseline:` lines you want to see have already scrolled past. You don't need to physically replug the cable or hit a reset button to recapture them.

Click in the Shell panel, then press Ctrl+D. That sends a CircuitPython "soft reset" — the device re-runs `boot.py` and `code.py` from scratch and prints everything to the Shell as it happens. Use it any time you:

- want to see the boot output again,
- have edited `code.py` and want a clean restart (saving normally triggers an auto-reload, but Ctrl+D gives you a confirmed clean run),
- need to capture a traceback for diagnosing a problem.

A healthy Ctrl+D in sensor mode looks like:

```
soft reboot
Auto-reload is on. ...
code.py output:
Calibrating — do not sip or puff ...
Baseline: 1013.241  sip<1008.241  puff>1018.241
Input mode: 2-switch  (1-sw input = dot, 3-sw accept = long_dash)
Code repeat: off  long-press cycles group: True
ESP-NOW: disabled by USE_WIRELESS_DISPLAY = False
```

> If `ESP-NOW: init failed (ESP-NOW error 0x3065)` shows up, that's a known CircuitPython 9.0.3 bug — see §12 "Wireless display" for the workaround.

Useful Thonny shortcuts

Action	Shortcut
Save file (and trigger Feather restart)	Ctrl+S / Cmd+S
Stop running program / go to REPL — **AeroMorse stops typing**	Ctrl+C (in Shell)
Restart program from REPL — AeroMorse works again	Ctrl+D (in Shell)
Open file from CIRCUITPY drive	File → Open → CircuitPython device

> Further Thonny details (REPL examples, alternative editors like VS Code) are in `TOOLS_AND_GUIDES.md`.